

The Symbol Grounding Framework Book Series

Volume 2: The Third Protocol

*A Blueprint for Zero-Integration Machine-to-Machine
Communication*

James Lee Stäkelum

THE SYMBOL GROUNDING COMPANY

Volume 2: The Third Protocol
A Blueprint for Zero-Integration Machine-to-Machine Communication

Copyright © 2026 James Lee Stäkelum

All rights reserved.

Published by
THE SYMBOL GROUNDING COMPANY
MMXXVI

ISBN: 979-8-9909162-6-5

SymbolGroundingCompany.ai

Contents

Machines That Can Meet Cold	v
The Substrate in One Pass	viii
This Architecture is Yours	xii
I The Missing Layer	I
1 Probably Understood Is Not a Protocol	2
2 The Babel Tax	8
3 Meaning Must Be Admissible	14
4 The Grammar, Not the Dictionary	21
II Shared Ground	27
5 The Core Lexicon as Shared Bedrock	28
6 The Canonical ID as Machine Interlingua	37
7 The Synapse as Machine Pivot	44
8 The Validity Window	51
III The Wire Protocol	58
9 HFF: Meaning as Portable Context	59
10 The Stranger in the Envelope	65
11 Knowledge Packs	71
12 AFP: The Speech-Act Layer	78
13 The Capability Exchange	85

14	The Sidecar Path	92
----	----------------------------	----

IV	Federation and Admission	100
-----------	---------------------------------	------------

15	Federated Queries Without a Central Brain	101
16	The Security Envelope	107
17	Profiles and the Admission Pipeline	114
18	The Receiver Is Sovereign	122
19	Governance of the Stable Layer	127
20	How the Protocol Evolves	134
21	Selective Disclosure	139
22	Trust Lenses	146
23	The Evidence Gate	151

V	The Protocol in the World	156
----------	----------------------------------	------------

24	What One Image Knows	157
25	Walkthrough: The Emergency Broadcast	165
26	Walkthrough: The High-Risk Command	172
27	Walkthrough: The Regulated Transaction	179
28	Walkthrough: The Commitment Graph	187
29	Software Agents and Commerce Agents	194
30	The Preparation / Decision Boundary	200
31	The Swarm That Can Refuse	205
32	The Global Ark	213
33	The World That Opens	219

APPENDIX	221
-----------------	------------

A	The SGF Architecture	222
B	HFF Envelope Reference	232

C	AFP Act Types and Confusion Pairs	233
D	Capability Manifest Reference	235
E	Profiles and Receiver Admission Checklist	236
F	Mini-Lexicon Dependency Bundle	238
G	Governance, Versioning, and Extension Policy	239
H	Glossary	241
I	Complete HFF Message Specimen	242
J	AFP Transition Matrix	247
K	Receiver Outcome Taxonomy	250
L	Sidecar Adoption Ladder	252
M	Minimal Conformance Tests	254
N	Residual Risks	256
O	The SGF OS Architecture	258
Index		263

Machines That Can Meet Cold

The first protocol taught machines to share a wire. The second taught them to share documents and services. The Third Protocol lets machines that have never met, and do not share a human language, exchange admissible meaning without a custom integration between them.

For half a century, every serious system has carried its own private world: schemas, codes, status fields, and authority rules that are perfectly reasonable inside the boundary and opaque outside it. Two systems can expose clean interfaces and still disagree, silently, on what a single value means. The bridges between them are hand-built treaties: adapters, mappings, prompts, and one-off assumptions that work until the next upstream change breaks them. Each new participant multiplies those treaties. The cost grows faster than the network. The risk is buried in code.

In parallel, machines have acquired an internal language rich enough to describe the world they act in. Models can read and summarize, detect patterns, classify events, and propose actions in fluent text. Grounding architectures have turned that fluency into structure: events with verbs at the center, roles for who did what to whom, lexicons where each sense has a stable address, and graphs where every claim carries its sources, time, and framing. Meaning stops being a string and becomes an object the machine can inspect.

When those two facts meet—local worlds everywhere, grounded meaning inside the machine—the missing layer becomes obvious. The hard question is no longer how to connect endpoints or serialize data. The hard question is what must travel with a piece of meaning so that a receiver does not have to guess. A machine that can move vehicles, schedule surgery, transfer funds, or reconfigure a swarm must know, for every incoming message, what is being claimed, what act is being attempted with that claim, who is making the move, how fresh it is, and whether local rules permit action. If any of those

conditions are missing, the receiver is being asked to improvise where it should be governed.

Probably understood is not a protocol. Transmission is not admission.

The Third Protocol answers that gap by changing what a message is.

On the wire, meaning becomes a grounded structure carried in a disciplined envelope. Each message arrives as a sealed unit that contains the claims themselves, the minimal lexicon material needed to hydrate unfamiliar terms, proof traces back to sources, integrity and freshness checks, and declarations that bind the content to a declared context. The receiver does not see a naked string and a field name; it sees stable identifiers it can follow into its own lexicon, Synapses whose roles obey a fixed grammar, and frames that mark each claim as fact, testimony, allegation, rule, hypothesis, or fiction. Ambiguity does not disappear. It is surfaced explicitly, where it can be refused or quarantined.

Over that content, an act layer makes the move explicit. A message declares itself as a notification, advice, a request, a query, a command, a proposal, an acceptance, a refusal, a cancellation, a confirmation, an acknowledgement, or an error. The underlying meaning does not change; the illocution does. The same grounded event can arrive as something to know, something to consider, or something to do. The receiver's obligations differ accordingly. From the receiver's point of view, every message becomes a proposal that must pass an admission procedure: the terms must be grounded, the act type declared, the authority checkable, the content fresh within tolerance, and the requested effect compatible with local safety and governance. Only then does it enter the action path. If any condition fails, the receiver refuses within the protocol, in structure, not in silence. Transmission is not admission.

Once that gate exists, machines that share this substrate and protocol can do something they have not done before. They can meet cold.

A system built for one human language and another built for a different language can still coordinate, because they are not meeting in English, Japanese, or German. They are meeting in grounded identifiers, role structure, acts, and procedures. A warehouse robot and a drone from another manufacturer do not need a custom mapping of field names. They need a shared way to say "this object," "this event," "this act," "this authority," "this time window," and "this safety boundary" in inspectable form. The protocol provides that

surface. Below it, each keeps its own data models. Above it, each keeps its own judgment.

The economic shape of coordination changes. Instead of building a new semantic bridge for every pair of systems, each participant maps once from its local world to a reusable protocol face. Sidecar components let existing services present that face without rewriting their internals. Capability exchange lets unfamiliar participants negotiate versions, encodings, act vocabularies, profiles, lexicon releases, trust anchors, and limits before any meaningful traffic flows. Where there is a safe overlap, they can start acting together without a custom integration project. Where there is not, they learn that fact at the boundary, before damage accumulates. The quadratic integration tax of pairwise semantic treaties becomes a linear cost at each edge.

On top of that surface, a different kind of network becomes possible. Knowledge is no longer glued permanently to the applications that first produced it. It can travel as signed, grounded packs that any conforming system can admit, inspect, or refuse under its own lenses. Graphs in different organizations can answer precise questions for each other without centralizing their data, because they meet at stable addresses for meaning rather than shared tables. Swarms of agents and robots can be deployed into environments where not every interaction can be planned in advance, because each member can receive, inspect, and, when necessary, refuse the meanings proposed by others.

This protocol does not claim to make machines wise, honest, or benevolent. Those properties belong to the systems and institutions that adopt it, not to the wire. The claim is narrower and load-bearing: consequential machine-to-machine communication needs a layer where meaning arrives as structure, acts are declared, authority is explicit, time is bounded, safety is checked at the gate, and refusal is a normal move. The Third Protocol is one concrete realization of that layer—specific enough to implement, strict enough to audit, and open enough to extend or replace.

The gate it describes is about to become necessary in every domain where machines act on each other's words. This work arrives early enough that its terms can still be shaped, and late enough that improvising without a gate is already a liability.

The Substrate in One Pass

The protocol layer does not move empty packets. It moves SGF objects. HFF, AFP, capability exchange, sidecars, federated queries, and swarms depend on the substrate those mechanisms carry.

The substrate begins with a simple refusal: machines should not act on ungrounded strings when grounded structure is available. A string can be fluent, plausible, and dangerous. A string can say “bank” without telling the receiver whether it means a financial institution, a river edge, a storage action, or an aircraft maneuver. A string can say “he signed it” while hiding who he is, what was signed, when it happened, and whether signed means physically signed, approved, or cryptographically sealed.

SGF turns such fragments into structured meaning.

The Core Lexicon

The Core Lexicon is the shared grounding floor. It is not a flat dictionary. It is a navigable graph of entries that machines can reference, hydrate, compare, and reason over. A useful entry carries a lemma, a microgloss, a part of speech, a language code, a namespace or source scope, embeddings or profile-declared comparison aids, content fingerprints where applicable, IS_A links, optional HAS_PART structure, and optional descriptive Synapses.

The microgloss is the small but crucial move. It does not need to distinguish a word from every other term in the lexicon. It needs to distinguish lemmamates. `bank.financial_institution` must be distinguished from `bank.r-river_edge`, not from every concept in the world. That makes disambiguation buildable.

The Core Lexicon is shared, but it is not total. Real messages contain private terms, document entities, local device IDs, business categories, legal terms, route segments, mission objects, and domain vocabulary. Those non-core terms travel through mini-lexicons, dependency bundles, and Knowledge

Packs. They tether to Core where possible, but they do not pretend to already be global.

Canonical IDs

The Canonical ID is the machine address of meaning. It is not the whole meaning. It is the place where meaning can be inspected.

Human systems often identify things with local names: PCN, fuel_cell_A17, priority_customer, left_lane_segment_77. Those names may be clear inside one organization and useless or dangerous outside it. The Canonical ID gives a receiver an address it can hydrate through the lexicon and related graph material.

This is why the Canonical ID functions as a machine interlingua. The interlingua is not a beautiful sentence between human languages. It is a stable address between machine worlds. English, Spanish, images, audio, barcodes, local database codes, and sensor events can all point toward the same grounded entry if the binding evidence supports it.

Content fingerprints

A content fingerprint is a compact, profile-declared comparison aid derived from semantic content, often from an embedding or related representation. It should not be treated as proof that two meanings are identical. It is a signal that helps machines compare semantic neighborhoods, cache entries, detect drift, and align terms across systems.

The important rule is declaration. A fingerprint means little without the profile that produced it. Different embedding models, dimensions, encodings, and update policies produce different comparison behavior. A receiver must know which profile it is looking at before deciding how much weight to give the fingerprint.

The fingerprint helps find or compare a door. It is not the authority to open it.

Synapses

The Synapse is the atomic unit of claim-bearing structure. It is a hub-and-spoke object: a hub expresses the event, relation, state, or act being represented; the spokes attach participants through fixed semantic roles such as agent, patient, theme, time, location, source, destination, instrument, reason, cause, and attribute.

The Synapse solves the fragment problem. A prose sentence can hide its structure in grammar and context. A triple can shatter an event across many statements. A Synapse keeps the event together so the receiver can inspect the whole structure as one unit.

This does not mean every Synapse is true. A Synapse is a structured claim or assertion-bearing object. It can be sourced, disputed, inferred, stale, hypothetical, or refused. Its power is that its structure can be examined. It makes error visible instead of hiding error inside prose.

GLEAN

GLEAN is the process that turns prose and other inputs into structured SGF material. It should be understood as admission control for meaning, not as mind reading. GLEAN does not magically know everything the source meant. It extracts candidate facts, resolves references, chooses senses, binds entities, records provenance, identifies uncertainty, and produces Synapses or refusal/gap reports when structure cannot be safely minted.

GLEAN's discipline matters because every later protocol layer depends on the quality of the structure it receives. HFF cannot repair an ungrounded payload by wrapping it. AFP cannot make an inferred act safe if the underlying terms are wrong. The admission pipeline cannot evaluate a claim whose provenance disappeared before the message was packaged.

Mini-lexicons and dependency bundles

The Third Protocol assumes that receivers will meet strangers. Those strangers will use terms not yet in the receiver's graph. A non-core term must therefore arrive with enough dependency material to be hydrated. That dependency material may include a canonical ID, lemma, microgloss, language

code, part of speech, namespace, Core tether, HAS_PART links, descriptive Synapses, source, version, and fingerprints.

This is the Stranger Rule: non-core meaning must not cross the boundary naked.

The receiver may still refuse. Hydration is not belief. It is the minimum condition under which belief, refusal, quarantine, or further query can become disciplined.

Why this substrate matters to the protocol

HFF carries SGF objects across trust boundaries. AFP declares what act is being performed with those objects. Capability exchange declares what the other participant can understand. Sidecars map legacy systems into this structure. Federated queries ask other graphs for structured answers. Trust Lenses decide what claims a receiver is willing to see. Evidence Gates test whether mined claims deserve admission.

All of that depends on the substrate.

The protocol moves meaning under discipline.

The substrate is the meaning being moved.

This Architecture is Yours

I started this project writing code. But as the system grew more complex, I realized what I really should give you is the architecture. The architecture isn't the code. The architecture is the ideas. The ideas can live in any language, and have an infinite number of ways to code it, as long as it conforms to the blueprint. I stopped coding to write a book explaining the architecture. Soon what started as one book turned into a series.

This book is part of that series. The series describes an architecture called the **Symbol Grounding Framework**, or SGF: a way of organizing knowledge so that machines can reason about meaning the way humans do, a protocol for machines to communicate that meaning to each other, a file format for storing it, one or more grammars for expressing it, one or more operating-system designs for running systems built on it, and the supporting concepts, methods, and patterns that hold all of it together. Some of this is described in this volume. Some is described in other volumes in the series.

The architecture is yours.

I am not claiming ownership of any of it. Not the concepts, not the protocols, not the formats, not the grammars, not the operating-system designs, not anything else in the architecture as it appears in this volume or in any other volume of the series. The whole architecture is placed in the public domain, irrevocably, as it is published. Anyone may use it. No one needs my permission. No fee will ever be charged. I hold no live patent applications and no granted patents on the architecture, and I commit never to file any. No patent claim will ever be made, now or in the future, by me or by anyone acting on my behalf.

The same is true of any code I publish that demonstrates how to build with this architecture. Reference implementations will be released under the Apache 2.0 license. That license lets anyone use, modify, and build on the code, including in commercial products. It comes with a built-in patent peace clause: anyone who uses the code is, by that act, joining a community that has agreed not to use patents as weapons against each other. That mutual peace is the foundation on which everything else is built.

But you don't need my code. The logic in these volumes is complete enough for you to create your own implementations in whatever language you prefer.

What I keep is the books. The words you are reading, the diagrams, the specific way the ideas are explained. These are mine, copyrighted in the ordinary way. Buy the book, read the book, lend the book to a friend, quote the book in your own writing. The standard rules of copyright apply. The architecture inside the book does not depend on the book; you can describe it in your own words, draw your own diagrams, write your own implementation, build your own products, found your own company, and never cite this book at all. You owe me nothing.

Why am I doing this? Because I want this architecture to become a standard for how machines reason about meaning. Standards happen when many people build with them and no single person controls them. Tim Berners-Lee gave away the protocols of the web. Vint Cerf and Bob Kahn gave away the protocols of the internet. Linus Torvalds released the Linux kernel under terms that prevented any single company from owning it. Every foundational technology that became universal did so because its creator chose adoption over ownership. I am making the same choice. The canonical home of the architecture is *SymbolGrounding.io*.

What I will do, going forward, is write more books in this series, teach what I know, and consult with teams adopting the architecture through my practice, *The Symbol Grounding Company*.

But I will not just be consulting. I intend to build. Through my company, I will write reference code, launch applications, and work to build commercial products that run on SGF. When you see me doing this, you should interpret it as a signal of exactly how much faith I have in this foundation. I want to prove that this architecture can power real, profitable systems. I want to see a thriving global ecosystem, and I want to be an active participant in it.

I do not know yet if SGF will achieve the global scale I envision. Adoption is not something I can dictate; it is something the community must choose. But I know that if this architecture does take root, it will need a governance model that outlives its creator.

I make you this permanent promise: the foundation will always remain neutral, and it will remain unified.

To guarantee this, we must move past the old era of open-source governance. The world does not need SGF to fracture into a dozen proprietary dialects the way early database languages did. Nor should the integrity of a global standard rely on human maintainers bending to the influence of well-resourced players.

Instead, the evolution of the **specification itself** will be governed by an algorithmic gauntlet. Anybody can write their own source code in whatever language they choose. What the gauntlet protects is the architecture—the upgrades from SGF 1.0 to SGF 1.1.

Whether a new semantic role or an Omega primitive is proposed by a human developer or generated by an AI, it must survive an impartial, rigorous evaluation by an ensemble of frontier language models. These models will score every proposal strictly against the published canon of this book series. Does it break the 15-role grammar? Does it violate the foundational primes? Does it introduce a geometric collision?

The proposals, the algorithmic reasoning, and the human debates over edge cases will all be public. In the beginning, I will shepherd this process, but

I will not override its logic on a whim. This ensures that no entity—no matter how large or heavily funded—can bypass the rules to force a proprietary advantage into the standard. The architecture will be protected by verifiable logic, not by backroom politics.

My commercial products will sit on the exact same open-domain bedrock available to you. We will share the grammar, and we will compete on merit.

The ultimate protection for any standard is the freedom to leave it. If anyone, including me, ever tries to capture this architecture, bypass the gauntlet, or betray its neutrality, the community is invited to **fork the specification**. You can take the public-domain blueprint, ignore my books, and build your own consensus without asking my permission.

I wrote this because the world needs an architecture for symbol grounding more than I need to own one.

Build something.

—James Lee Stäkelum

PART I

The Missing Layer

Why meaning needs a protocol

CHAPTER 1

Probably Understood Is Not a Protocol

Interpretation is a guess; admission is a gate. A machine that mistakes the former for the latter has already surrendered its sovereignty to the void.

A warehouse vehicle receives a message: “clear the left lane.” The message is grammatical. The sender may have been authorized. The intent may be urgent. None of that is enough.

The receiving machine must answer five questions before it can act. Which lane, under which coordinate system? Who sent the message, and under what authority? Is the message fresh, or a replay of an instruction issued two minutes ago in a different context? Does acting violate a local safety policy? And is the message a command, a suggestion, an informational update, or a test frame that should not trigger action at all? Prose does not answer any of these questions. It raises them.

This situation has a name. It is not an edge case in the history of machine communication. It is the central problem.

The Tempting Answer

Every autonomous machine shipped in the last decade carries an onboard language model capable of parsing and generating natural language. Fleet operators use natural language to configure vehicle behavior. Logistics systems accept prose from operators and translate it into routing decisions. The models

are capable enough that the obvious objection appears decisive: if machines can already read and write fluent prose, why build a separate protocol at all?

The answer turns on a distinction that the premise ignores: the distinction between interpretation and admission.

Interpretation asks what a message probably means. A language model interprets well. Given sufficient context, a capable model will identify the likely intent behind “clear the left lane”—the lane probably referred to by local convention, the clearance probably expected within some reasonable window, the authority probably established by the sender’s role in the system. The model’s interpretation may even be correct most of the time.

Admission asks a different question: whether the meaning of a message may alter system state, create a commitment, trigger a safety policy, move physical assets, or change the behavior of a machine operating under liability. Admission is not a question about what a message probably means. It is a question about whether the receiver is permitted to act on that meaning, and under what procedure.

The difference between interpretation and admission is the difference between a transaction that usually clears and a transaction that must clear. Banking systems interpret payment requests all day. They also run separate admission procedures—authorization, balance checks, fraud detection, audit logging—that cannot be probabilistic. A payment that “probably succeeds” is not a payment. A lane clearance that “probably applies” to the left lane at the current intersection is not a lane clearance command.

The moment a message can cause a machine to act in the physical world, probably is not a protocol.

The Hidden Burden of Prose

Natural language appears to communicate more than any formal notation can hold. That appearance is created by shared context between human speakers: prior knowledge, common reference frames, situational awareness, and the social accountability that comes with face-to-face speech. Strip those supports away and the apparent richness collapses into a list of unresolved questions.

A machine receiving “clear the left lane” must resolve five categories of ambiguity before it can act safely.

The first is entity grounding. “The left lane” is not an entity. It is a relational description requiring a reference point, a coordinate system, and a mapping to the machine’s current environment. The message carries none of this. The machine must infer which physical segment is meant, and any inference introduces error.

The second is act type. “Clear” may be a command, a request, an advisory, a warning, or a test. These categories are not interchangeable. A command from an authorized controller requires immediate compliance. A request may be deferred pending local conditions. An advisory may be logged and ignored. A test frame must be dropped without action. A machine that cannot distinguish among these types will either refuse too much or comply too much.

The third is authority. Not every sender has the right to instruct every receiver. A message from a traffic management system carries different authority than a message from a peer vehicle, which carries different authority than a message from an unauthenticated source. The prose carries no declaration of authority. Inferring authority from prose is, again, probabilistic.

The fourth is freshness. A message cleared through a queue in a congested network may arrive seconds or minutes after it was relevant. Acting on a stale clearance instruction in a live traffic environment produces errors that look like failures of the receiving system, not failures of the protocol.

The fifth is safety scope. The receiving machine operates under local safety policies: minimum following distances, maximum deceleration rates, no-go zones, emergency preemption rules. A clearance instruction that conflicts with a local safety constraint cannot be admitted, regardless of the clarity or authority of the sender. The machine must be able to refuse without exiting the protocol.

None of these categories are exotic. They are the routine burden of any machine acting in a shared physical or financial environment. Natural language does not carry declarations for any of them. A language model can guess at their resolution, but guessing is not adequate when the action is irreversible.

The Action Boundary

There is a threshold at which communication becomes consequential. Below the threshold, interpretation errors are self-correcting: the human re-reads, the system retries, the context clarifies. Above the threshold, interpretation errors compound: the vehicle occupies the wrong lane, the fund transfers to the wrong account, the drone enters a restricted zone, the actuator engages the wrong valve.

The action boundary is the line between those regimes. Below it, natural language is efficient and appropriate. The conversation is the thing. Above it, the interpretation layer cannot be the admission layer. The two layers have different requirements, and conflating them produces systems that are brittle in exactly the scenarios where brittleness is most dangerous.

This is not an argument against natural language as part of a machine communication stack. Language models are excellent for translating human intent into structured proposals, for explaining structured decisions in prose that humans can review, for rendering machine-readable outputs in forms that operators can audit. The claim is narrower: natural language cannot serve as the admission layer for high-stakes machine action, because the admission layer requires declared structure that prose does not carry.

A language model may propose a structured action. That proposal must then be validated, grounded, authorized, and checked against safety policy before it enters the action path. The model sits outside or around the protocol. It does not replace it.

Admission, Not Interpretation

A protocol for machine meaning must do more than convey intent. It must carry a procedure. When a message arrives at a receiver capable of taking action, the receiver must be able to run a sequence of checks: Is the meaning grounded in shared, traversable terms? Is the act type declared, not inferred? Is the sender's authority asserted in a verifiable form? Is the message timestamped and fresh within the receiver's tolerance? Does the proposed action satisfy the receiver's local safety policy? And if any check fails, does the protocol specify a structured outcome—refusal, quarantine, error, or request for clarification—rather than leaving the receiver to improvise?

These requirements define a class of protocol that is distinct from both natural language exchange and from conventional API design. APIs make systems callable; they do not make meaning admissible. Cryptographic signing makes identity verifiable; it does not make intent declared. Semantic ontologies make terms traversable; they do not constitute an admission procedure.

The missing layer is a procedure for making meaning admissible: a set of structural requirements that a message must satisfy before it may enter the action path. That procedure is not a restriction on expression. It is the precondition for safe, auditable, autonomous action at scale.

Coordination between autonomous machines is possible. The obstacle is not the complexity of machines or the richness of language. The obstacle is that communication and admission have been treated as the same problem, and they are not.

Probably understood is not a protocol. The Third Protocol begins where interpretation ends.

What the Receiver Has to Infer

The weakness of prose becomes clearest when the receiving machine has to list what it must infer before acting.

The message “clear the left lane” looks simple to a human reader. It is not simple to a machine that may move two tons of metal at highway speed. The receiver must identify the sender. It must determine whether the sender is describing a condition, issuing advice, making a request, or asserting command authority. It must decide which lane is meant, which segment of road is meant, which vehicles are included, which time window applies, whether the request is still fresh, whether the request came from a trusted emergency authority, whether the requested maneuver is physically possible, and whether local sensor state permits the action.

None of that burden appears in the sentence.

The sentence relies on shared context. Human drivers carry that context in bodies, habits, laws, and social expectations. Machines do not get to assume the same background. A machine that treats the sentence as self-sufficient is not being intelligent. It is smuggling in unstated assumptions.

This is why the first chapter is not an attack on language models. A language model may be excellent at interpreting the sentence. It may produce the right explanation. It may render the meaning in clean prose. It may even produce the correct structured candidate most of the time. But the receiver still needs the action path to depend on declared structure, not on a fluent interpretation that happened to be plausible.

The protocol does not replace language because language is weak.

It replaces language at the action boundary because language is generous.

Generosity is a virtue in human conversation. It is a liability in machine action. Human beings repair ambiguity through social judgment. Machines need the ambiguity removed, bounded, or refused before action. They need the sender's meaning to arrive as a structured act with grounded references and admission gates.

The receiver is not asking, "What did that probably mean?"

It is asking, "What am I allowed to do with it?"

CHAPTER 2

The Babel Tax

Bridges are fossilized negotiations between systems that cannot speak the same language. Meaning becomes admissible only when the protocol forces the treaty into the light, replacing hidden translation with explicit inspection.

Every system that handles meaning carries a local model of it. The model is embedded in schemas, identifiers, category hierarchies, business rules, and assumptions about authority. Those models are not documented in any form the next system can read. They are the sediment of operational decisions made by teams who no longer work there, encoding distinctions that made sense at the time and may be invisible to any outside party.

Two systems can expose clean APIs and still be semantically isolated. The APIs make the systems callable. They do not make the systems mean the same thing.

Coordina- tion pattern	What each new partner requires	Failure mode
Pairwise integration	New semantic bridge, new mapping, new assumptions	N-squared coordination cost

Continued on next page

(Continued)

Coordination pattern	What each new partner requires	Failure mode
Shared protocol layer	Capability exchange, profile selection, dependency hydration	Conformance and governance burden
Sidecar deployment	One local bridge to a reusable protocol face	Local mapping must be explicit and audited

Local Worlds

A field named *priority* appears in dozens of enterprise systems. In a hospital's triage system, it encodes clinical urgency: four levels, with specific clinical criteria attached to each. In a logistics platform, it encodes shipping tier: three levels, with contractual implications. In a financial exchange, it encodes order execution preference: time-priority within a price tier. In a public safety dispatch system, it encodes response level: five levels, with resource allocation rules attached.

These four uses of *priority* share a word and share nothing else. The integer 3 in the hospital system and the integer 3 in the logistics system encode distinctions that are not comparable, not interchangeable, and not safely routed through a generic adapter that copies values across systems.

The problem is not terminology disagreement. The problem is that each system's meaning is local by design. The hospital's triage categories encode medical judgment that its clinical teams developed over decades. The logistics platform's shipping tiers encode commercial agreements that its sales team negotiated. Neither system needs to match the other's model to do its own work correctly. The problem arrives only when a message must cross the boundary and be admitted by a system whose model is different.

This is not a pathological condition. It is the normal condition of any large-scale distributed system. Every organization that develops software for long enough accumulates local meaning. The meaning is rational, well-maintained, and correct within its boundary. It becomes a liability the moment it needs to interoperate.

Why Bridges Multiply

A direct integration between two systems requires someone to produce a mapping from one local model to the other. Which fields correspond? What value transformations are needed? Which authority declarations are compatible? How are missing fields handled? The work is bounded for a single pair of systems, tedious but completable.

The cost structure changes with the third system. The first two systems now each need a separate mapping to the third. The third needs separate mappings to both. The cost is not linear in the number of systems. For every new participant, a new set of mappings must be produced for every existing participant. The total mapping burden grows with the square of the participant count.

This is the Babel Tax: the quadratic cost of making local meaning admissible across system boundaries through pairwise translation.

The standard response is middleware: a centralized integration platform that accumulates mappings and manages transformations. Middleware reduces some duplication. A well-maintained integration hub can apply a mapping once rather than once per consumer. The architectural improvement is real. The semantic problem is unchanged. The middleware contains the same pairwise mappings; they are stored in one place rather than distributed across endpoints. When a new system joins, the middleware must learn a new mapping. The quadratic growth is deferred, not eliminated.

Language models reduce the labor of producing mappings. A capable model can propose a field correspondence between two systems' schemas with high accuracy in many cases. The proposal still requires review, validation, and testing. More critically, the mapped values are still probabilistic: the model may correctly identify that `hospital.priority.3` likely corresponds to `logistics.priority.urgent`, but "likely" is the same problem under another name. A mapping that is right most of the time does not satisfy the admission requirements of a system that needs to know whether to dispatch an ambulance.

The Shape of Semantic Isolation

The gap between field matching and semantic matching is the precise location of the problem. Field matching is what adapters and middleware provide: a value in one system is transformed and written to a corresponding field in another. The receiving system can parse the value. It cannot verify that the concept behind the value is the one the sender intended.

Consider a message crossing from a hospital's patient-management system to a pharmaceutical-dispensing system. The hospital system sends a `patient_id`, a `medication_code`, a dose, and a `priority`. The dispensing system receives these fields and appears to understand them. But the `medication_code` in the hospital system is drawn from a drug database maintained by the hospital's pharmacy department, using a code list agreed between the pharmacy and the hospital's IT team. The dispensing system uses a different drug database maintained by a national formulary. The two code lists overlap extensively but not completely. There are drugs in the hospital's list that have no exact match in the formulary, drugs with multiple matches, and drugs where the code structures correspond but the dose units do not.

The adapter can map the structural fields. It cannot resolve the semantic gap without a declared procedure for grounding the medication code in a shared reference. Without that procedure, the receiving system is making an inference from a field that means something different in its sender's model. The inference may be right. In a dispensing context, it must be correct.

API documentation does not resolve this. Documentation is prose. It describes what a field means in the sender's system, addressed to a human engineer who will read it and write code. The code the engineer writes embeds a translation decision that is not expressed in the protocol, is not visible to the receiving system at runtime, and cannot be audited without inspecting the integration code.

The Adapter Illusion

An integration built with a language model at its core replaces the human engineer's reading of the documentation with a model's reading of it. The model produces a mapping function rather than a human. The function is generated faster and may be more consistent. The fundamental structure is

the same: a probabilistic translation decision is embedded in an artifact that runs between two systems, outside the protocol, invisible to the receiver.

The receiver cannot verify that the adapter was given accurate documentation. It cannot verify that the model's translation decision was correct. It cannot verify that the adapter is current if one system's semantics changed after the adapter was built. It cannot verify any of these things because the translation is not part of the message it receives. The message arrives looking like a well-formed API call. The semantic work happened upstream, in a process the receiver cannot inspect.

This is not an argument against adapters. Adapters are necessary wherever systems exist that were not designed for the same protocol. The argument is that adapters are not a solution to the semantic admission problem. They move the translation out of sight. They do not make meaning admissible.

The New Definition

The Babel Tax has a standard definition: the quadratic cost of pairwise integration. That definition is correct but incomplete. The full cost includes not just the labor of building bridges, but the ongoing liability of translation decisions that are embedded in artifacts rather than expressed in protocols.

A message that carries its own semantic grounding—terms tied to shared, traversable reference structures, acts declared in terms the receiver can evaluate, authority expressed in a form the receiver can verify—does not require a translation decision to be made outside the protocol. The meaning is admissible on arrival, or it is declared inadmissible and the sender can be informed. Either way, the decision is made within the protocol, at the boundary where it belongs, not in middleware that both systems depend on but neither owns.

The Babel Tax is the cost of making local meaning admissible somewhere else. The quadratic bridge-building is the visible cost. The invisible cost is every integration that works until the underlying semantics shift, and no one knows it broke until the error surfaces in a downstream system three steps removed from the cause.

That is the tax. The Third Protocol is the mechanism for reducing it at the boundary where it is incurred: not by standardizing all vocabulary, but by making the meaning a message carries inspectable at the point of admission.

What the Bridge Really Contains

The phrase “custom integration” sounds like code. In practice it is usually code wrapped around unstated human agreements.

Two teams meet. They decide which fields correspond. They decide which statuses are equivalent. They decide which codes are safe to translate and which require exceptions. They decide what happens when a value is missing, when a status is stale, when a counterparty uses an older schema, when a field has two possible meanings, when one system has a distinction the other lacks. Then an engineer writes the adapter.

The adapter is therefore more than software. It is a fossilized negotiation.

This is why pairwise integration is so expensive. Each bridge contains a private treaty between two systems. The treaty may be partly documented, partly embedded in code, partly remembered by the team that built it, and partly discovered only when the bridge fails. When a third system arrives, the treaty cannot simply be reused. It encoded assumptions about the first two systems. A new treaty must be written.

The Third Protocol does not eliminate all local mapping. It changes where the treaty lives. Instead of burying assumptions inside an adapter, the protocol requires the message to carry grounded terms, declared acts, dependency material, authority claims, profile requirements, and refusal paths. The receiver can inspect the treaty at the boundary.

This is a different economic shape. The cost does not vanish. It becomes reusable, testable, and visible.

The old bridge says: “Trust this adapter.”

The protocol says: “Inspect this crossing.”

CHAPTER 3

Meaning Must Be Admissible

Admission is the sovereign act of the receiver, not the reflex of a sensor. A machine that cannot refuse is a machine that cannot govern.

A message can arrive. A message can be parsed. A message can be cryptographically verified, syntactically valid, and semantically rich. None of this is sufficient. The receiver must still decide whether the message may enter.

That decision is admission.

Receipt Is Cheap

Networks deliver messages continuously. A receiver in any connected system processes hundreds or thousands of incoming messages per second, most of which require no deliberation: they are telemetry updates, heartbeat signals, status confirmations, log entries. The receiver processes them, routes them, and moves on. The cost is the cost of parsing and storage.

High-stakes messages are different not because they are rarer but because acting on them changes things that cannot easily be changed back. A telemetry update that is slightly wrong costs nothing. An instruction to reroute a vehicle that is slightly wrong puts the vehicle in the wrong place. An instruction to transfer funds that is slightly wrong moves money. An instruction to engage a physical actuator that is slightly wrong produces a physical outcome. The wrongness of such an instruction cannot be corrected by receiving the next message. The damage is in the action, not the record.

Receipt is the beginning of the problem, not the end. The system has received the instruction. Now it must decide whether to act on it. That decision

requires structure that natural language does not carry and that API field values do not carry. It requires a procedure.

The Admission Decision

Admission is a receiver-side judgment about whether a message may alter state. The judgment is not a guess and not an interpretation. It is a structured evaluation of the message against a set of declared requirements.

A receiving machine making an admission decision must evaluate the message on several independent dimensions.

The first is term grounding. The entities and concepts named in the message must map to shared, traversable reference structures. A message that says “reroute to dock 7” admits the question: which dock 7? The answer must not require the receiver to infer from context. The message must carry or reference a grounded identifier that resolves unambiguously in the receiver’s lexicon and graph. If it does not, the receiver cannot evaluate the instruction, and admission must be refused.

The second is act declaration. The message must declare what kind of action is being proposed. Is this a **COMMAND**, requiring authority validation before any execution path can open? A **REQUEST**, admissible if local conditions permit? A **PROMISE**, creating a commitment record? An **INFORM**, updating the receiver’s world model without requesting action? These categories have different downstream consequences, and a receiver that must infer which category applies from prose is a receiver that will sometimes infer wrong.

The third is authority. The sender must declare their basis for making this request. Authority in a distributed system is not a global property. A controller has authority over the vehicles in its designated zone. A logistics coordinator has authority over the shipments assigned to their account. A safety system has authority to override other instructions within declared safety constraints. None of this authority is visible in a message that carries only a sender identity and a payload. The authority must be declared, checkable, and scoped to the specific act being requested.

The fourth is freshness. A message must arrive within the temporal window in which it is relevant. An instruction to hold at an intersection while emergency vehicles pass has a validity window that may be measured in seconds.

After that window, acting on the instruction produces the wrong outcome—the intersection is now clear, and the vehicle that holds is blocking traffic. The message must carry a declared validity window, more than a timestamp.

The fifth is safety. The receiving machine operates under local safety constraints that it is not required to reveal to senders and is not permitted to override on sender request. An instruction that would violate a safety constraint must be refused. The refusal must be communicable without the receiver explaining its safety policy, because safety policies may themselves be sensitive. The protocol must support structured refusal with a reason code that gives the sender enough information to retry correctly or escalate appropriately.

These five dimensions define the admission gate. A message that satisfies all five may be admitted. A message that fails any one of them requires a declared outcome: refusal, quarantine, partial admission, or a request for supplementary material.

The Gates in Sequence

The admission gate is not a single check. It is a sequence of checks, each of which must succeed before the next is evaluated. This ordering is not arbitrary.

Term grounding comes first because a receiver cannot evaluate an instruction about an entity it cannot identify. An instruction to “close the emergency valve at pump station 14B” cannot be evaluated for authority or safety if the receiver does not have a grounded representation of “emergency valve at pump station 14B.” The grounding check gates everything else.

Act declaration comes second because the downstream meaning of passing the remaining checks depends on act type. The authority required to issue a `COMMAND` is different from the authority required to issue an `INFORM`. The safety evaluation of a `COMMAND` is different from the safety evaluation of a low-risk `REQUEST`. Without a declared act type, the receiver cannot evaluate authority or safety correctly.

Authority comes third because authority is act-type-specific. Checking authority before act type has been declared produces a check against the wrong authority requirements.

Freshness comes fourth because a stale message that passes all earlier checks is still inadmissible. Checking freshness last, after all other checks, avoids the cost of freshness validation for messages that would fail earlier checks.

Safety comes last because it is the constraint that the receiver may not negotiate. Even a message that passes all other checks must satisfy local safety policy. The safety check is a veto, not a condition. Its position at the end of the sequence reflects its unconditional authority: if safety fails, the message is refused regardless of what came before.

Refusal as Protocol Behavior

A system that cannot refuse is not participating in a protocol. It is subject to one.

Refusal is normal protocol behavior, not an error condition. When a receiving machine refuses an instruction because the act type is undeclared, that is the protocol working correctly. The sender now knows that it sent a structurally incomplete message and can correct it. When a receiver refuses because the requested action is outside the declared authority scope, the sender now knows it was directing the instruction to the wrong receiving agent, or that it needs to escalate its authority before proceeding. When a receiver refuses because the safety check failed, the sender knows that the instruction conflicts with local constraints and must be modified.

Refusal without structure is not useful. A receiver that responds with a generic error leaves the sender unable to correct the problem. Structured refusal carries a reason code, a description of which check failed, and enough information for the sender to formulate a valid retry or route the instruction elsewhere. This requires that the protocol define the vocabulary of refusal: what codes exist, what each code means, and what corrective action each code implies.

A swarm of machines that can only comply is a swarm that cannot defend itself against incorrect or malicious instructions. The ability to refuse, with structure, is what separates autonomous coordination from remote control. Autonomous coordination requires that each participant evaluate incoming

instructions against its own state, constraints, and policy—and act on that evaluation, including the decision not to act.

The Protocol Proper

The Third Protocol is not a wire format. It is not a serialization standard. It is not primarily a language for expressing machine meaning, though it provides one.

The Third Protocol is the procedure by which machines make meaning admissible to one another.

That procedure encompasses the structure of messages, the structure of identifiers, the declaration of act types, the assertion and verification of authority, the specification of validity windows, the definition of safety constraint categories, the vocabulary of refusal, and the rules for what happens when any check fails. It encompasses the lexicon and graph structures that allow terms to be grounded, the profile system that allows different deployment contexts to specialize the procedure, and the capability exchange that allows machines to declare their admission requirements before message exchange begins.

None of those components is central individually. The procedure is central. The components exist to make the procedure executable at scale.

A message that has been admitted has passed the procedure. The receiver has evaluated it against its state, constraints, and policy and determined that acting on it is appropriate. The sender can know this, through receipt confirmation or through observation of the receiver's behavior, and can build on the basis of that admission. The action is auditable: every step in the admission process is logged, attributable, and replayable.

This is the object the architecture proves. The Third Protocol is an admission procedure. The question it answers is not what the message means. The question is whether the meaning may enter.

Admission Is a Verb

The word admission can sound static, as if a receiver checks a box and lets a fact pass through a gate. That is not enough. Admission is an act performed by the receiver. It has inputs, policy, state, timing, and consequence.

A receiving system may admit a message into memory but not into action. It may admit a claim for display but not for automated execution. It may admit a Knowledge Pack for query but not for command. It may admit a source under one Trust Lens and reject it under another. It may admit a fact as historical but refuse to use it as current. It may admit a request as understood and still refuse the requested action.

These distinctions matter because machine systems often collapse them. A system retrieves a document and treats it as evidence. It parses a sentence and treats it as a fact. It receives a signed message and treats it as authority. It finds an API shape match and treats it as semantic agreement. Each collapse is a small category error. At scale, those category errors become infrastructure failures.

Admission is the procedure that keeps the categories apart.

It asks: admitted where?

Into storage?

Into a view?

Into an inference chain?

Into a commitment graph?

Into a physical action?

Into a legal or financial record?

The same incoming meaning may receive different answers for each destination. A receiver may store a disputed claim, show it to a human, prevent it from influencing automated decisions, and attach a receipt explaining why. That is not indecision. It is disciplined admission.

The most dangerous systems are often not the systems that know too little. They are systems that admit too much into the wrong place.

The Honest Objection

The honest objection to admissible meaning arrives early: fixed grammar sounds like a new rigidity. The history of computing is full of schemas that were elegant at birth and intolerable five years later. A fixed grammar can look like another attempt to make the world sit still.

That objection is serious. The answer is not that the grammar magically anticipates every future domain. The answer is that the grammar fixes only the part of meaning that must be common for machines to coordinate. It does not freeze every term, every domain object, every local category, every policy, or every application profile. It fixes the structural slots through which participants can be understood.

The distinction is the difference between a road system and a list of destinations. Roads need stable rules: lanes, signs, signals, right-of-way, maps, addresses. Destinations can change. New buildings appear. Businesses close. Streets are renamed. The road system survives because the rules of navigation are not rewritten every time a new destination exists.

The Third Protocol makes the same bet. It does not try to list the world. It gives the world a stable way to be addressed, described, acted upon, refused, and audited. The vocabulary can grow because the grammar remains disciplined.

The claim is not that all meaning has been solved. The claim is that boundary-crossing machine meaning needs a stable structural surface. Without that surface, every new vocabulary becomes another private dialect. With that surface, private vocabulary can travel with its grounding material.

CHAPTER 4

The Grammar, Not the Dictionary

Vocabulary is a local, time-bound artifact that fractures under the weight of change. Grammar is the immutable bedrock that allows meaning to travel across the boundaries of domain and time.

Every attempt to build a universal vocabulary for machines has failed by trying to build too much. The failure pattern is consistent across a century of attempts: a committee produces a vocabulary, the vocabulary covers the domain as the committee understood it at the time, the domain changes, the vocabulary cannot keep pace, and the systems built on it are either frozen or forked.

The Third Protocol does not build a universal vocabulary. It constrains a universal grammar.

The False Solution

The obvious path to machine interoperability is vocabulary agreement. If every system uses the same term for the same thing, no translation is required. Two machines that both refer to the same physical object by the same canonical name can exchange facts about that object without ambiguity. The coordination cost goes to zero at the point where the vocabularies match.

This is correct as a local observation. Within a domain governed by a single authority, shared vocabulary is achievable and enormously valuable. A hospital network that standardizes on a single drug coding system across its facilities reduces transcription errors, enables cross-facility analysis, and makes clinical decision support systems viable. The benefit is real.

The problem is the scope assumption. Shared vocabulary within a governed domain is achievable. Shared vocabulary across all domains, all organizations, all languages, all future states of technology is not achievable. Vocabulary is infinite. Every new disease requires new diagnostic codes. Every new financial instrument requires new taxonomies. Every new class of autonomous device requires new operational categories. Every new language brings its own structures for categories that may not correspond to the categories encoded in a vocabulary designed elsewhere.

The vocabulary problem is not a problem of insufficient effort or insufficient standardization bodies. It is a structural problem: vocabulary is a local, time-bound, cultural artifact. Systems that depend on shared vocabulary inherit all of its limitations, including the limitation that it cannot absorb new territory without organizational decisions that slow the velocity of vocabulary expansion to the velocity of committee processes.

Why Vocabulary Keeps Escaping

A standards body defining terms for medical device communication will produce a vocabulary that feels complete at the moment of publication. Within three years, new device classes will have appeared, new sensing modalities will have been developed, and new operational categories required by regulators will have been created. The vocabulary has three choices: extend through an amendment process, allow manufacturer extensions that fragment the standard, or leave the new territory undefined and let implementers improvise.

All three choices reproduce the problem. Amendment processes are slow relative to technology development. Manufacturer extensions produce the pairwise integration problem the standard was designed to solve. Undefined territory produces improvisation, which is another name for local meaning.

Cultural difference compounds the technical problem. A vocabulary designed by teams working in English will encode distinctions that English-speaking cultures find natural and may not encode distinctions that other languages find natural. This is not a problem of translation—machine-readable terms do not need to be beautiful in any language—but a problem of conceptual coverage. A category system built around one culture's ontological assumptions will be extended by other cultures through approximation and

workaround, degrading the shared meaning that the vocabulary was designed to create.

Time is the most reliable destroyer of shared vocabularies. The vocabulary of computing in 1995 contained no grounded terms for containerized microservices, transformer architectures, or autonomous vehicle coordination. Any standard built in 1995 for machine communication in computing would require either wholesale revision or constant extension to remain useful. The extension process itself fractures shared meaning, because different adopters extend in different directions.

The Grammar Move

The Third Protocol takes a different approach: constrain the structure of meaning rather than its content.

A Synapse—the unit of meaning in the protocol—has a fixed grammar: a hub verb, a set of typed role slots, grounded filler identifiers, provenance, and optional frame data. The grammar specifies what slots exist and what types of things may fill them. It does not specify what the hub verbs are. It does not specify what entities fill the roles. It does not specify what frames mean within a given domain. Those are vocabulary questions, and vocabulary is left free.

This is the same architectural move that TCP/IP made for network communication. The packet header specifies a fixed set of fields: source address, destination address, sequence number, flags. The payload is unconstrained. TCP/IP does not specify what messages may be sent. It specifies the structure within which messages must be wrapped to travel over the network. Every subsequent application—web, email, video, voice, financial transactions—rides the same structural layer without requiring changes to it.

The Third Protocol specifies what a Synapse must contain to be admissible. It does not specify what Synapses may say. A Synapse carrying a command to a surgical robot and a Synapse reporting a sensor reading from an atmospheric buoy use the same grammar. The entities, acts, and roles differ. The structure is constant.

A closed grammar enables federation. When two systems share a grammar, any Synapse generated by one system is structurally parseable by the other,

regardless of whether the vocabulary is known. The receiving system may not be able to evaluate a Synapse whose hub verb or role filler it has never seen. But it can identify the structure, extract the roles, and determine what additional information it would need to evaluate the Synapse fully. That is enough to begin the resolution process. A completely foreign message that shares no vocabulary at all can still be identified as a Synapse, its structure can be traversed, and its dependencies can be requested.

An open grammar—one that allows any predicate type, any edge label, any structural form—produces the opposite behavior. Two systems may both claim to exchange structured data and still be unable to parse each other’s messages, because they have each used the structural flexibility to make different choices. The open grammar does not eliminate pairwise incompatibility. It relocates it from vocabulary to structure.

Open Vocabulary with Tethers

Constraining the grammar does not eliminate the vocabulary problem. Machines still need to refer to entities, and entities require names. The protocol’s answer to vocabulary diversity is tethering rather than standardization.

The Core Lexicon provides a shared vocabulary for a large set of common concepts, sufficient to cover the semantic territory that most machine communication requires. Terms in the Core Lexicon have canonical identifiers, structured definitions, part-of-speech annotations, hierarchical relationships that allow traversal, and content fingerprints that allow comparison across versions. This vocabulary is not closed. It is not curated by a single authority. It is open and versioned.

Domain-specific terms that are not in Core are not forbidden. They are required to be tethered to Core. A term that is not in Core must include, in any message that uses it, sufficient definitional material for the receiver to understand its relationship to Core terms—at minimum, an `IS_A` link that places the new term within the Core hierarchy. A message carrying the term `calico_cat` must include material establishing that a calico cat is a domestic cat, which is a feline, which is a mammal, and so on until the chain reaches Core. The receiver that has never seen `calico_cat` can, after processing that

chain, evaluate any Synapse containing it in terms of what it knows about mammals, animals, or living things.

This is not vocabulary standardization. The term `calico_cat` does not need to be added to any central list. It needs to be traceable. The tether to Core is what makes a locally invented term admissible in a message to a receiver that has never seen it before. Without the tether, the receiver has a dangling pointer. With the tether, the receiver has a traversable path to shared meaning.

The grammar constraint and the tethering requirement work together. The grammar ensures that any Synapse can be structurally parsed. The tethering ensures that the entities named in any Synapse can be semantically evaluated. Together they allow a protocol that is simultaneously stable and extensible: stable in structure, extensible in vocabulary, tethered enough that the extension does not fragment the shared semantic ground.

What This Enables

A protocol that constrains grammar and tethers vocabulary rather than standardizing it has a specific capability profile.

Two machines that have never communicated can exchange Synapses and evaluate each other's messages, provided both have access to the Core Lexicon and both follow the tethering requirements for non-Core terms. No prior coordination is required. No shared dictionary was negotiated in advance. The grammar provides the structural scaffold. The tethered vocabulary provides the semantic resolution path.

The claim is not that any two machines can immediately understand any message from any other machine. The claim is that any two machines that follow the protocol can determine what is needed to evaluate any message, and can request that material through the protocol's dependency mechanisms. The protocol defines not just the message format but the resolution process: what to do when a message contains unfamiliar terms, how to request dependency material, how to declare admission pending resolution, and how to complete admission when the material arrives.

This is a weaker and more defensible claim than universal understanding. It is also more useful. A protocol that requires prior vocabulary agreement

before any message can be evaluated would require extensive coordination before any new pairing of systems could communicate. A protocol that defines the resolution process for unfamiliar vocabulary allows new pairings to begin communicating immediately, resolving unknowns incrementally as they arise.

The stable layer is the grammar. The grammar is what makes the protocol a protocol rather than a vocabulary recommendation. Vocabulary grows indefinitely. Grammar does not.

Why the Lexicon Must Be Navigable

A lexicon that only defines words is useful to a reader. A lexicon that machines can navigate is useful to a protocol.

The difference appears the moment a receiver encounters a term it has never seen. A definition alone may say what the term means in prose, but the receiver needs to know where the term sits. Is it a kind of animal, vehicle, legal instrument, medical condition, financial asset, place, tool, event, policy, or role? What parts does it have? What attributes normally attach? Which parent classes should be inherited? Which facts are definitional and which are empirical?

The `IS_A` relation gives the receiver an upward path. `HAS_PART` gives it internal structure. Descriptive Synapses give it facts that can be admitted, refused, or queried. Media references and embeddings give it surface recognition paths. Content fingerprints and profiles give it comparison handles. Together these make the lexicon navigable.

Navigation is the difference between recognizing a label and entering a library.

When the receiver hydrates a term, it should more than attach a string. It should gain a limited but inspectable semantic neighborhood. That neighborhood may be small. It may be partial. It may depend on a pack or profile. But it gives the receiver somewhere to stand.

PART II

Shared Ground

Lexicon, identity, Synapses, and validity

CHAPTER 5

The Core Lexicon as Shared Bedrock

A dictionary defines words with more words, creating a circle that never touches the ground. A lexicon anchors symbols to a shared, traversable graph, turning the infinite regress of explanation into an admissible stop. Meaning becomes infrastructure only when it ceases to be a definition and becomes a coordinate that any machine can inspect.

A lookup table is not a lexicon. A dictionary is not a lexicon. A graph is a lexicon, because a graph can be traversed.

When a receiving machine encounters an unfamiliar term in an incoming Synapse, it needs to do more than find a definition. It needs to determine what category of thing the term belongs to, what other things share that category, how specific the term is relative to general concepts the machine already knows, and whether the Synapse's claim about the entity is consistent with the type constraints the entity's category imposes. None of these questions can be answered by lookup. All of them require traversal.

The Core Lexicon is the shared bedrock because it is the graph that machines traverse to ground unfamiliar meaning.

Where Explanation Stops

Every protocol for meaning needs a place where explanation stops.

The regress problem begins with a simple demand. If A is accepted as true, A needs proof. Suppose B is the proof. But B cannot support A unless B is also true, so B needs proof. Suppose C is the proof of B. But C now needs proof

as well. The chain continues without end. If the demand is taken literally, nothing can ever be proven, because every proof arrives needing another proof behind it.

The logic looks clean. The conclusion is unusable.

The pattern is familiar. Achilles never catches the tortoise if motion is described as an infinite sequence of remaining intervals. First Achilles must reach where the tortoise was. By then the tortoise has moved. The reasoning can be repeated forever. The conclusion is false. Achilles arrives. The paradox does not abolish motion. It exposes a missing account of convergence.

The grounding regress has the same character. If every symbol must be grounded by another symbol, and that symbol by another, then no symbol can ever be grounded. Taken literally, that argument proves too much. It would make mathematics, chemistry, law, engineering, and ordinary language unusable. Their notations would point only to other notations, and no finite procedure would explain why action is warranted. Working systems avoid that trap by defining where their symbols stop, what evidence binds those stops, and what procedures govern use.

The mistake is to treat grounding as one thing. Human grounding includes embodiment, sensation, memory, pain, appetite, social training, and lived consequence. Machine-to-machine grounding does not require all of that. It requires enough shared structure for a receiver to know what is being referred to, what evidence supports the reference, what act is being performed, and whether local policy permits admission. A robot does not need to feel heat as pain to refuse a movement that violates a thermal safety threshold. A clinical system does not need to experience illness to bind a lab value to a medical category.

Operational grounding is not phenomenological grounding.

A machine receiving a boundary-crossing message receives symbols, references, fields, model outputs, and local identifiers. If those identifiers point only to more prose, the machine chases definitions. If they point back to themselves, the machine has learned a circle. If they stop at a private table, the receiver has been asked to trust a stop it cannot inspect.

The Core Lexicon is SGF's engineered stopping rule. It does not claim to contain all meaning. It provides shared addresses where meaning can be inspected. A non-core term may still travel, but it must bring its mini-lexicon

entry and its IS_A tether back toward Core. The receiver does not have to believe the claim. It does not have to admit the term. But it no longer has to chase language forever.

This is the answer to the overbroad impossibility claim. The regress problem does not prove that machine grounding is impossible. It proves that grounding cannot be left implicit. If every symbol is explained only by more symbols, the receiver has no admissible stopping point. If the chain stops at a private table, the receiver is being asked to trust a local convention. SGF makes a different move: it stops at shared, inspectable addresses, then requires non-core terms to bring their own path back to that shared ground.

The stop is not metaphysical certainty. It is protocol admissibility.

Machines do not need the totality of human meaning to coordinate. They need enough grounded structure to make the receiver's next act admissible.

Grounding is the difference between a definition and an admissible stop.

Dictionary vs Graph

A dictionary entry answers one question: what does this word mean? The answer is a definition, expressed in other words, addressed to a human reader who can use their own knowledge to interpret the definition. This works for human reading because the reader brings context that no dictionary can encode: familiarity with related terms, understanding of how the term is used in specific domains, sensitivity to connotation and implication.

A machine needs a different kind of entry. The machine needs traversal. When the machine encounters the term *thrombocytopenia*, it cannot use context in the way a human reader does. What it needs is a path from *thrombocytopenia* to terms it already knows. *thrombocytopenia* IS_A *hematologic_disorder*. *hematologic_disorder* IS_A *medical_condition*. *medical_condition* IS_A *condition*. The machine that knows nothing about *thrombocytopenia* but knows the category *medical_condition* now knows that *thrombocytopenia* is a medical condition, not a chemical, not a location, not a person. It can evaluate the Synapse using whatever it knows about medical conditions, even in the absence of specific knowledge about *thrombocytopenia*.

The traversal capability is what makes the Core Lexicon useful as shared ground. It is not a registry of all possible meanings. It is a navigable structure in which any new term can find a location, and in which that location inherits the properties and constraints of the term’s ancestors.

This is why a flat list fails as shared meaning. A flat list can answer “does this term exist?” It cannot answer “what kind of thing is this term?” without additional structure. The additional structure is the graph.

Entry Anatomy

A Core Lexicon entry is not a definition. It is a node in a graph with a specific set of required fields, each of which serves a distinct function in the admission procedure.

The canonical identifier is the node’s address. It is constructed from a language code, a lemma, a microgloss, a part of speech, and a container identifier. The identifier is human-readable, collision-resistant across systems that have never coordinated, and parseable by any system that understands the format. A system receiving `en.bank.financial_institution.noun.core` can extract the language, the base form, the disambiguating sense, the grammatical category, and the lexicon namespace from the identifier itself, without any lookup.

The lemma is the base form of the word or name in its language. For `en.bank.financial_institution.noun.core`, the lemma is `bank`. The lemma is the entry point for human-facing searches and for approximate matching in the absence of an exact identifier.

The microgloss is a short disambiguating phrase that distinguishes this sense of the lemma from other senses. `financial_institution` distinguishes this use of `bank` from `river_slope`, `aeronautic_turn`, and other lemmamates. The microgloss does not define the term completely. It does the minimum necessary: separate this node from other nodes with the same lemma.

The part-of-speech field identifies the grammatical category. This matters for the admission procedure because the grammar constrains what roles different parts of speech may fill in a Synapse. A noun fills entity roles. A verb fills hub roles. Knowing the part of speech of a term constrains how it may appear in a valid Synapse.

The embedding is a vector representation of the entry's meaning, generated from the lemma, microgloss, and definition. Embeddings allow approximate matching across terms that are semantically similar but lexically different. They provide a mechanism for identifying candidate matches when an exact identifier is not known, and for comparing the semantic neighborhood of a term across different lexicon versions or profiles.

The content fingerprint is a hash of the entry's definitional content: the microgloss, the definition text, the IS_A links, and the HAS_PART links if present. The fingerprint allows two systems to verify that they are working with compatible versions of an entry without exchanging the full entry content. If the fingerprints match, the entries agree. If they differ, the entries have diverged, and the systems should negotiate which version to use.

The IS_A links place the entry in the Core hierarchy. Every entry except the terminal semantic primitives must have at least one IS_A link pointing to a parent concept. The IS_A links are the traversal edges. They are what allows a machine to navigate from a specific term to its category, from a category to its parent category, and eventually to the terminal primitives that form the base of the hierarchy.

The HAS_PART links are optional but valuable for concrete entities. `automobile.vehicle.noun.core` HAS_PART entries for `engine`, `wheel`, `steering_system`, and so on. These links support reasoning about the components of entities, which is relevant for maintenance instructions, safety assessments, and capability declarations.

Descriptive Synapses are optional additional facts about the entry, expressed in the protocol's own grammar. An entry for `ambulance.vehicle.noun.core` might carry a descriptive Synapse asserting that an ambulance has `emergency_medical_transport` as its reason or intended service function, represented through the final role/frame vocabulary rather than through a custom role. These Synapses are not required for the entry to be valid, but they increase the richness of machine reasoning over the entry.

The Microgloss Trick

The microgloss is the buildability move that makes the Core Lexicon tractable.

A common criticism of large shared vocabularies is that disambiguation requires unbounded descriptive effort: to specify which sense of bank is meant requires distinguishing it from every other sense of bank, which requires knowing all the senses, which requires complete enumeration of the language.

The microgloss breaks this requirement. The microgloss does not need to distinguish a sense from the entire language. It only needs to distinguish it from other senses of the same lemma, the term's lemma-mates. `bank.financial_institution` only needs to be distinguishable from `bank.river_slope`, `bank.aeronautic_turn`, and other senses that share the lemma `bank`. The disambiguation work is bounded by the number of lemma-mates, not by the size of the language.

This bound is small in practice. Most lemmas have fewer than ten distinct senses that require separate entries. The lemma `bank` has perhaps five or six that a machine communication system would encounter with any regularity. The microgloss for each needs only to be distinct from the microglosses of the others, tested by embedding distance: two microglosses for the same lemma must be sufficiently distant in embedding space that a vector search for one will not return the other as a near match.

The lemma-mate restriction reduces an unbounded problem to a bounded one. It is why a Core Lexicon with millions of entries can be bootstrapped and maintained by a tractable process rather than requiring indefinite committee effort.

Versioning and Stability

The Core Lexicon is not static. Terms are added as new domains contribute to it. Existing entries are revised when definitions become imprecise or `IS_A` relationships are corrected. Entries become deprecated when the concepts they encode are superseded or merged into other entries.

Versioning is not optional. A Synapse that uses an entry from Core version 2.1 may refer to a concept whose definition changed between version 2.1 and version 2.3. Two machines communicating across a version difference must be

able to identify the difference, determine whether it is material to the Synapse under evaluation, and either resolve it or declare it unresolvable.

The profile system manages version diversity. A machine operating in a specific deployment context declares the Core version it uses and the profile under which it evaluates messages. When a message arrives from a machine using a different Core version, the receiving machine can use the version declaration to select the appropriate resolution strategy.

Content fingerprints per entry allow fine-grained version tracking. Two machines may be running different versions of Core overall but agree on the specific entries relevant to a particular message. If the fingerprints for those entries match, the message can be evaluated under shared meaning regardless of overall version differences. If they do not match, the machines can exchange the specific entries in question and determine whether the difference is material.

The versioning mechanism also handles the evolution of local vocabularies. A domain lexicon that tethers its terms to Core version 2.1 must be updated when Core version 2.2 changes the entries its terms tether to. That update process is discoverable: the entry fingerprints will differ, the difference will be detected at admission time, and the system's profile will specify how to handle the mismatch.

The Tether

The Core Lexicon's power is not that it contains all terms. It is that all terms can reach it.

A locally minted term—a device identifier, a mission-specific category, a private business classification—that carries an IS_A link to a Core entry can be evaluated by any receiver that has access to Core. The receiver does not need to know the local term in advance. The IS_A link places the local term in the Core hierarchy, giving the receiver enough context to evaluate Synapses that use the term, apply type constraints from the term's ancestors, and reason about relationships between the local term and other terms in Core.

Consider a mission term: `urban_search_sector.geographic_region.noun.mission_42`. This term is not in Core. It was minted for a specific operation. A receiving machine that has never participated in mission 42 does

not know this term. The dependency material for the term includes its IS_A link: `urban_search_sector` IS_A `geographic_region`, which IS_A `spatial_region`, which IS_A `region`. The receiver processes the IS_A chain and understands that `urban_search_sector` is a spatial region: a bounded area in physical space. The receiver can now evaluate Synapses using `urban_search_sector` in terms of what it knows about spatial regions—that they have coordinates, that things can be inside or outside them, that they can intersect other regions—without knowing anything specific about mission 42.

This is what makes the Core Lexicon a shared grounding layer rather than a shared dictionary. It does not need to contain every term. It needs to provide the ground to which every term can be anchored. The tether is the admission mechanism for locally invented meaning.

A fact that still needs its paragraph to be understood is not yet liquid knowledge. The Core Lexicon, combined with the tethering requirement, is the procedure by which a locally expressed fact becomes a fact that can travel. Any term that can be anchored to Core can be carried in a Synapse to a receiver that has never seen it, and the receiver can evaluate the Synapse with the help of the Core traversal. The meaning crosses the boundary because the grounding layer is shared, not because the vocabulary is shared.

That is the distinction the rest of the protocol rests on.

Why Not Just Use Translation?

A translation system can turn a German message into English and an English message into Japanese. It can do this fluently. The result may be good enough for a human reader. But human translation still leaves a machine with prose, and prose still leaves the receiver with interpretation.

If the translated sentence says “the medicine is contraindicated,” the receiver must still know which medicine, which patient, which contraindication, which source, which validity window, and which action follows. If the translation says “the road is blocked,” the receiver must still know which road segment, from when to when, under whose authority, and whether the message is advisory or commanding.

Translation can help a human understand a message. The Canonical ID helps a machine locate the meaning the message refers to.

This is why machine interlingua should not be judged by the beauty of its sentences. It should be judged by whether two machines can arrive at the same grounded address, inspect the same dependency material, apply their own policies, and produce consistent receipts. A translator produces prose. A Canonical ID produces a place in the graph where further discipline can begin.

The distinction becomes even sharper when the source is not language. A photograph of a calico cat cannot be translated into English in the same sense a Spanish sentence can. It can be captioned. It can be described. But for machine coordination, the useful crossing is from image evidence to Canonical ID. Once the ID exists, the receiver can use the same machinery it uses for text-derived meaning: Core Lexicon, Knowledge Packs, HFF, AFP, Evidence Gates, and local policy.

This is why the interlingua is an address rather than a sentence.

CHAPTER 6

The Canonical ID as Machine Interlingua

Translation is the art of approximation; addressing is the discipline of precision. Meaning is not a fluid to be poured, but a coordinate to be verified.

The dream of an interlingua is older than the machines that now need one. The old dream imagined a middle language: a sentence in one language would pass through the middle and emerge in another. English into French. French into Russian. Russian into Japanese. The bridge would be neutral enough to receive many tongues and stable enough to preserve meaning across them.

The Third Protocol does not fulfill that dream in the old human way.

It does something narrower and more useful for machines.

It turns meaning into an address.

Human translation must preserve more than reference. It must preserve surface, tone, idiom, register, implication, rhythm, style, and cultural force. Machines coordinating under constraint do not need all of that. They need to know which concept, entity, state, relation, event, or action is being referenced. They need to know where that meaning lives in a graph. They need to know whether the reference can be hydrated, whether it belongs to the Core Lexicon, whether it arrives from a domain pack, whether its dependencies are present, and whether local policy will admit it.

The Canonical ID is the machine interlingua.

Not because it is an ideal sentence. Because it is a stable address for meaning.

Surface forms can differ. *cat*, *gato*, *chat*, a camera embedding of a calico cat, a veterinary database entry, and a warehouse robot's local object class may all approach the same semantic place from different directions. The protocol does not require those surface forms to become one human word. It requires them to resolve, under declared profiles, to a shared or bridgeable Canonical ID.

Once the ID exists, the receiver can move. It can hydrate the entry in the Core Lexicon. It can follow the *IS_A* chain. It can inspect *HAS_PART* structure. It can retrieve descriptive Synapses. It can query a Knowledge Pack. It can ask a federated graph for external facts. It can apply a Trust Lens. It can decide whether the reference is admissible.

The interlingua was never an ideal sentence. It was a stable address for meaning.

The Address Is Not the Meaning

A Canonical ID is not magic. It is not truth. It is not proof. It is not the whole concept packed into a string. It is an address into a structured semantic neighborhood.

That distinction matters because IDs can be wrong. A perception model may bind an image of a fox to the ID for a dog. A parser may select the financial sense of *bank* when the river-edge sense was intended. A local namespace may mint a domain ID that is poorly tethered to Core. A malicious sender may use a legitimate-looking ID in a false claim. The ID does not eliminate admission. It makes admission inspectable.

The receiver can ask: Which namespace minted this ID? Which Core parent does it claim? Which microgloss distinguishes it from lemma-mates? Which profile generated the embedding or content fingerprint? Which dependencies arrived with it? Which source asserted it? Which Knowledge Pack does it belong to? Which Trust Lens is willing to surface it?

Without the ID, the receiver cannot ask those questions in a disciplined way. It is left with a string, a vector, or a local code.

A database key is usually private. It identifies a row in one system. A Canonical ID is designed to cross boundaries. It carries enough public structure

for another machine to know what kind of semantic object it is being asked to consider. It is still only an address, but it is an address made for federation.

The old interlingua dream tried to place a shared middle tongue between human languages. The Canonical ID places a stable coordinate between machine worlds.

The Role of Embeddings

Embeddings still matter. They help find the address. They place a word, phrase, image, sound, or sensor observation near candidate meanings. They can propose that an image resembles the lexicon's calico-cat examples. They can suggest that a local phrase in one language is close to a known entry in another. They can narrow search space.

But an embedding is not the interlingua.

An embedding is a compass. A Canonical ID is an address.

The difference is practical. A compass points toward a neighborhood. An address gives the receiver a place to inspect. The receiver can compare the proposed binding against the Core entry, the microgloss, the source profile, the dependency bundle, the Evidence Gate, and local policy. The receiver can accept the binding, reject it, quarantine it, ask for dependencies, or require another source.

This is the recurring discipline: proposal is not admission.

Embeddings propose.

Canonical IDs address.

Admission decides.

Cross-Language Without Prose Translation

A Japanese warehouse robot and a German drone do not need to speak each other's human language to coordinate. They need a shared act grammar, grounded payloads, and Canonical IDs that can be hydrated by both sides.

Suppose the robot's local term refers to a medical fuel cell. Its internal system may store the term in Japanese. The drone's operating context may render terms in German. The HFF message does not need to translate Japanese prose into German prose. It needs to carry the Canonical ID, the dependency

material for any non-core term, the role the item plays in the Synapse, the AFP act being performed, and the relevant profile.

The drone can render the result in German for a human operator if needed. That rendering is not the protocol's meaning. It is a display artifact.

This is how the interlingua dream becomes operational without pretending to solve literary translation. Machines do not need to agree on word order, conjugation, idiom, politeness, or style before they can coordinate. They need to agree on addresses, roles, acts, proofs, and refusal rules.

The Canonical ID is where languages, domains, and modalities stop being separate dialects and become different roads to the same place.

Boundaries and Refusal

The Canonical ID is also a boundary object. It crosses from one machine world into another, but it does not demand belief. The receiver remains sovereign.

A sender may say that a local term is an instance of a Core concept. The receiver may inspect the dependency chain and reject it. A sender may say that two IDs are equivalent. The receiver may store that as an external claim without merging local entities. A sender may include an image-derived ID. The receiver may admit the observation only as low-confidence or require raw evidence.

This is why the Canonical ID belongs in The Third Protocol rather than only in the substrate book. The substrate explains what the ID is. The protocol book explains what happens when the ID travels.

When it travels, it becomes a request for admission.

The receiver can open the door.

The receiver can ask for papers.

The receiver can refuse.

That is the machine interlingua at protocol scale: not one language ruling all others, but one disciplined address system that lets meaning cross boundaries without pretending every boundary has disappeared.

Four Arrivals at the Same Door

The easiest way to see the difference between translation and interlingua is to watch four different signals arrive at the same door.

A physician writes, in English, “The patient reports a penicillin allergy.”

A clinic in another country records the local equivalent of “PCN sensitivity.”

A pharmacy database stores a coded contraindication under an internal drug-safety vocabulary.

A scanned intake form contains a handwritten note that an optical recognizer reads imperfectly, then resolves through a medical Knowledge Pack.

In a prose translation system, these remain translation problems. Which phrase should become which English phrase? Is “sensitivity” an allergy, an intolerance, a side effect, or a warning? What if the handwritten note is ambiguous? What if the local abbreviation has two meanings?

In The Third Protocol, the important question is not which English phrase is prettiest. The important question is whether each signal resolves to a grounded medical concept that a receiver can inspect. The Canonical ID becomes the meeting place. The Knowledge Pack supplies the domain constraints. The evidence trail records the source. The receiver’s profile decides whether the claim is strong enough to affect prescribing.

The same pattern holds outside medicine.

A warehouse robot calls an item `fuel_cell_A17`. A supplier pack calls it `portable_medical_power_unit`. A customs graph calls it by a tariff classification. A logistics agent receives an image of the container label. These are not four meanings if they resolve, under evidence, to the same object or to explicitly related objects. They are four arrivals at the same door.

The door is the Canonical ID.

The door does not make them identical by force. It lets the receiver compare them under a procedure. One signal may be authoritative. One may be stale. One may be a loose synonym. One may be a misbinding. The ID is where the comparison can happen.

That is why the Canonical ID should not be understood as a mere convenience. It is the boundary object that lets machines enter the same semantic room without surrendering their local languages, local databases, or local evidence standards.

The Local Name Problem

Every organization has local names that are reasonable inside the organization and dangerous outside it.

A hospital says PCN. A clinician knows the abbreviation. A pharmacy system may know it too, but only if the abbreviation exists in the right local vocabulary and the context is clinical. A logistics system might interpret the same string as a product code. A general model may expand it differently depending on nearby words. A foreign-language system may not expand it at all.

The local name is not wrong. It is local.

The Canonical ID changes the problem. The hospital does not need to force every receiver to learn its abbreviation. It needs to map the abbreviation, under a medical profile and evidence trail, to the grounded concept it intends. The receiver can then hydrate the ID, inspect the allergy or sensitivity class, check the Knowledge Pack, and decide how strongly the claim should be admitted.

This pattern repeats everywhere. A factory calls a component A17. A supplier calls it `portable pump assembly`. A regulator calls it by a category code. A photograph of the component resolves through a vision model. These are all local approaches to the same semantic object, or to related objects that must not be confused. The protocol does not ask the world to stop using local names. It asks local names to show their papers when they cross boundaries.

That is the interlingua in practice.

Not one global vocabulary.

One disciplined way for local names to become inspectable addresses.

The 3x5 Card Test in Practice

The old research card had a simple discipline: a note should still make sense when separated from the book it came from. It had to carry enough context to be useful later. Page number, author, claim, quotation, topic, and perhaps a short comment. A loose sentence copied onto a card without context was not a note. It was a fragment waiting to confuse its future reader.

The Synapse applies the same discipline to machine knowledge.

“He signed it” is not liquid. The fact still needs the paragraph. The receiver must know who he is, what was signed, when, in what capacity, under what

source, and whether “signed” means physically signed, approved, cryptographically signed, or metaphorically endorsed.

A liquid version names the actor, the act, the object, the time, the role, the source, and the relevant frame. It may still be incomplete. It may still require provenance. But it no longer depends on the paragraph to supply its basic referents.

This is why liquid knowledge is more than small knowledge. A small fragment can be useless. A liquid fact is small enough to move and complete enough to survive movement.

That is the standard.

CHAPTER 7

The Synapse as Machine Pivot

A fragment is a debt of context that a machine cannot repay. The Synapse is the structural ledger that converts that debt into liquid, admissible knowledge, stripping away the noise of tone to leave only the hard geometry of action.

The Fragment Problem

A fact that cannot stand alone is not yet a fact. It is a fragment.

“He signed it” is a fragment. Which he? Which it? The surrounding paragraph resolves those questions for a human reader, who carries the referents silently from sentence to sentence. Remove the paragraph and the fragment collapses to noise. The pronouns become dangling pointers. The claim that seemed precise becomes semantically void.

Human communication is built on this compression. Context is efficient. Entire conversations run on shared background, borrowed reference, and ellipsis. The listener fills gaps from memory and convention. This works because the listener was present when the background was established, speaks the same dialect of assumption, and can ask a clarifying question when the gap becomes too large.

Machine coordination does not have access to these repair mechanisms. A receiver acting on “he signed it” cannot pause to ask who “he” is. In a high-stakes path, the receiver must either act or halt. Acting on an unresolved fragment is a bet that the context was correctly inferred. Halting because the context was not shipped is the safer behavior, and it is also what the protocol should enforce.

A fact becomes liquid when all its context-dependence has been made explicit. “Ambassador Chen signed the Northern Water Compact on behalf of the Coastal Authority” is liquid. Every participant is named. The action is typed. The object is identified. The role of the agent is declared. A machine can act on this claim without having been present for the conversation that preceded it.

The Synapse is the mechanism that converts fragments into liquid facts.

What the Synapse Refuses to Preserve

Human language carries more structure than the action path requires. It carries tone, register, politeness level, stylistic choice, cultural resonance, and rhetorical force. These qualities are real. They affect human communication in important ways. The Synapse does not capture them, and that omission is deliberate.

Consider the range of ways an instruction to pass the salt can be delivered across languages and registers. One is a blunt imperative. Another is a ceremonial flourish occupying several lines. Another is a minimalist gesture that functions as a request only within a particular cultural frame. Each of these utterances would produce the same Synapse: an action hub whose type represents physical transfer, with a filler identifying the item to be moved, a filler identifying the agent responsible for moving it, and a filler identifying the recipient. The surface that distinguished the utterances is not carried forward.

This is not a defect. It is a scope boundary. The Synapse captures the structure the protocol is allowed to act on. That structure is: which entity performed which action on which patient, where, when, through what instrument, for what recipient, under what authority, and with what provenance. Everything outside that inventory is either non-admissible or relegated to a trace sidecar that preserves it for audit and display purposes without allowing it to govern action.

The scope boundary matters because machines cannot be permitted to act on tone or register. A system that routes resources differently depending on whether an instruction arrived with a polite or an impatient surface is not

following policy. It is following noise. Protocol removes tone from the action path so that policy, evidence, and authority govern instead.

This is not the same as erasing surface form. Provenance fields in a Synapse can reference the original text, store the source document span, and link to the full shadow trace of the ingestion pipeline. Surface form is preserved for human review, legal audit, and display. It is quarantined from the decision logic. The separation is architectural, not incidental.

What Action Requires

The structure that machine coordination requires has a stable inventory. For any event or state change that a receiver must evaluate, the relevant questions are a finite set: who acted, what changed, who was affected, where this occurred, when it occurred, by what means, under what role or authority, and with what evidence.

These are not arbitrary choices. They represent the minimal grounded description under which a receiver can evaluate whether to admit a claim to its reasoning graph, route it for further processing, or reject it. A claim that answers none of these questions is a bare assertion with no foothold. A claim that answers all of them that apply is a structured fact.

The Synapse formalizes these questions as a closed grammar of semantic roles. Each role is a typed spoke attached to a hub node representing the event itself. The hub carries the event type, drawn from the Core Lexicon. The spokes carry the role fillers, which are grounded identifiers, not free strings.

The grammar closure is deliberate. An open grammar, where any team can invent a new role name for their domain, produces the same fragmentation problem the Synapse was designed to solve. If one system uses `HAS_AGENT` and another uses `HAS_PERFORMER` for the same structural slot, their graphs cannot be queried against each other without a bilateral mapping table. The mapping table is the Babel Tax appearing again at the schema level. A closed grammar holds the schema stable so that any receiver who shares the grammar can parse any message without prior coordination on slot names.

Closing the grammar does not limit what can be expressed. Domain specificity lives on the hub node, not on the roles. “Authorized the transaction,” “detected the intrusion,” “calibrated the sensor,” and “signed the treaty” are

all distinct event types in the Core Lexicon hierarchy. The roles remain stable across all of them. The semantic richness is concentrated where it can be organized and reasoned over, not distributed across an unbounded edge vocabulary that compounds over time.

The Pivot Object

The Synapse is defined by four components: the hub, the roles, the frames, and the provenance.

The hub identifies the event type. It is a grounded entry in the Core Lexicon, which means it carries a canonical identifier, a microgloss, a part-of-speech classification, and an IS_A chain leading to the lexicon root. A hub is not a free string. A free string cannot be compared to another free string from a different system without ambiguity. A hub with a canonical identifier can be compared across organizational boundaries because the identifier is constructed to be self-describing: it encodes language, lemma, disambiguation gloss, part of speech, and namespace in a human-parseable format that requires no lookup table to decode.

The roles are the twelve semantic spokes that anchor the hub to its participants. Not all roles fire on every Synapse. A simple observation may carry only a hub, a patient, a location, and a time. A complex transfer may carry agent, patient, theme, recipient, instrument, source, and destination. The role inventory does not change between these cases. What changes is which slots are filled. A receiver knowing the grammar can inspect any Synapse and immediately understand what each spoke represents without consulting documentation specific to that sender.

Frames are optional structures attached to a Synapse that carry metadata the protocol separates from the core claim. Temporal scope belongs in a frame: whether a state is eternal, bounded by a known interval, open-ended with a decay rate, or instantaneous affects how a reasoner queries it but does not affect what the claim asserts. Confidence, modality, and derivation tags also belong in frames. This separation keeps the core claim clean. A downstream system that does not support temporal reasoning can still parse and store the core claim; the frame degrades gracefully rather than causing a parse failure.

Provenance records the derivation path from source text to Synapse. Which document produced this claim? Which extraction pass? Which version of the pipeline? A Synapse without provenance is an assertion without evidence. It may be admitted to a reasoning graph under a low-confidence policy, but it cannot be cited, audited, or challenged because there is no chain to inspect. Provenance is not optional for claims in regulated or safety-critical domains.

The 3x5 Card Test

A useful operational test for Synapse completeness is whether the claim can be written onto a 3x5 index card and shipped independently of its source document.

A researcher citing a finding does not include the entire paper. They include the finding: who observed what, when, where, by what method, under what conditions, and citing which source. If the card requires the reader to fetch the paper to understand the claim, the card is not a citation. It is a reference to a citation.

The Synapse applies this discipline to machine memory. “The cargo manifest was certified” is not a complete Synapse. It is missing the agent, the document identifier, the timestamp, and the authority under which certification was performed. Once those fields are populated from grounded identifiers in the Core Lexicon or an accompanying dependency bundle, the claim can travel. A receiving system can evaluate it without access to the original document, the original system, or the original context of extraction.

A fact that still requires its paragraph is not liquid yet. Liquidity is the condition under which a fact can be recombined with other facts from other sources to produce valid reasoning. Liquidity requires that the context on which the fact depends has been made explicit in the fact’s own structure. The paragraph that originally contained the fact was the context delivery mechanism. The Synapse is the replacement that makes the context portable.

This is the pivot. Not a translation from one language to another, but a transformation from context-dependent fragment to context-explicit structure. The pivot is lossy on surface form. It is disciplined on action-relevant structure. That discipline is what makes the grounded fact admissible to a receiver who was not present when the sentence was written.

Where Surface Goes

Preserving surface form for machines serves different purposes than preserving it for humans, and the architecture keeps the two purposes separate.

For human review and legal audit, the full surface of a document should be traceable from any Synapse extracted from it. The provenance chain records the source document, the extraction span, and the version of the extraction pipeline. A human investigator following the chain from a contested claim arrives at the original text. The Synapse did not erase the evidence. It distilled the action-relevant structure from it.

For display and rendering, a system may reconstruct natural-language representations of Synapses. These representations are glosses, produced by a renderer that reads the grounded identifiers and generates readable text from them. The gloss is a display artifact, not the protocol truth. A gloss in English produced from a Synapse whose hub was grounded against a Japanese source document is a reader convenience, not a translation of the original. The protocol truth is the grounded structure. The gloss is a human-facing window onto that structure.

The distinction matters because display and action must never be confused. A receiver acting on the gloss rather than the Synapse is reading a rendering artifact and treating it as protocol. This failure mode is not hypothetical. It is the original failure of natural-language machine coordination: the rendering of intent into text, the transmission of text, and the re-interpretation of text at the receiving end, with probabilistic drift introduced at every step. The Synapse removes text from the action path. The gloss is for people. The structure is for machines.

Admissible Structure

The Synapse is the smallest unit of structure the protocol is allowed to act on.

This definition is both a lower bound and an upper bound. It is a lower bound because anything smaller, a bare identifier, a raw predicate string, a fragment without role structure, is not admissible to a reasoning graph that must check it against policy, authority, and prior state. It is an upper bound because anything larger, a document, a paragraph, a multi-clause sentence

treated as an atomic unit, does not localize provenance correctly. When a compound claim is treated as atomic, a dispute about one part of the claim forces the entire unit to be quarantined, even the parts that are not in dispute.

The Atomic Truth Principle follows: if two sub-claims within an apparent unit can have different sources or different truth values, they belong in different Synapses. Beethoven's birth year and birth city both come from "Beethoven was born in Bonn in 1770," but the year came from a biographer's inference and the city from a church record. They are different claims. They should be separable, so that if the year is later contested, the city claim remains undisturbed.

The protocol acts on admissible structure, not on sentences. A fact becomes admissible when its context-dependence has been resolved, its participants have been grounded to identifiers, and its structure has been expressed in the closed grammar. Before that transformation, the fact is available information. After it, the fact is liquid knowledge the receiver can inspect, evaluate, and route without any further request to the sender.

CHAPTER 8

The Validity Window

Freshness is a property of the transmission, but validity is a property of the claim. A message that arrives in the present is a relic if it lacks the temporal coordinates to survive the transition.

A message can be fresh and still be wrong.

That sentence appears contradictory only if freshness is treated as one thing. It is not. A packet has freshness. A fact has validity. A source has currency. A claim has conditions. A receiver has a query time.

The Third Protocol must keep these separate.

A signed HFF envelope may have been created three seconds ago. Its nonce may be valid. Its signature may verify. Its replay window may be open. The packet is fresh. But the payload inside may assert a fact that expired ten years ago, belonged only to a document's narrative time, or depends on conditions not currently true.

A fresh packet can carry a stale fact.

The Difference Between Message Time and Claim Time

Message time answers one question: when did this envelope cross the boundary?

Claim time answers another: when is the meaning inside the envelope valid?

These are not interchangeable. A receiver may receive a fresh HFF bundle containing a historical claim. The message is current, but the claim belongs to the past. A receiver may receive a fresh signed statement that a road is closed, but the closure window ended thirty minutes earlier. A receiver may

receive a fresh instruction based on a Knowledge Pack whose version has been superseded. A receiver may receive a fresh query result from a federated graph whose source document was correct when published but obsolete now.

Security freshness protects against replay. Validity protects against temporal confusion.

Both are required.

Consider a document written in 2015 that says a particular person is president. The extraction may be correct. The Synapse may be grounded. The source span may be hashed. The HFF bundle may be signed today. None of that makes the claim current today. It was a valid claim in a temporal frame. It is not a timeless property of the world.

The same problem appears in ordinary questions. “Is Tom married?” cannot be answered responsibly without a time. Was Tom ever married? Is Tom married now? Was Tom married at the time of the document? Was he married according to one source but divorced according to another? A receiver that treats the sentence as context-free without making the context explicit will answer the wrong question with confidence.

Kinds of Validity

Facts behave differently.

Some are definitional under a frame. “A triangle has three sides” is stable inside ordinary Euclidean geometry. The receiver may not need to check a current-world source each time the claim appears.

Some are conditional. “Water freezes at zero degrees Celsius” depends on pressure, purity, and the meaning of freezing. The statement is stable only when its conditions travel with it or are understood by profile.

Some are stable instance claims. “Paris is the capital of France” is true until superseded under a public-world frame. It should be checked for supersession, but not treated like a momentary sensor reading.

Some are temporal instance claims. “Tom is married” requires a validity window.

Some are situational. “The store is open” may be stale within minutes.

Some are perspectival. “She is tall” depends on comparison class, speaker, domain, or threshold.

The point is not to solve metaphysics. The point is to prevent machines from admitting claims under the wrong validity frame. These categories are engineering commitments, not declarations about eternal truth.

The "Now" Problem

"Now" is the most dangerous timestamp because it often remains implicit.

A newspaper article says, "The mayor is under investigation." A transcript says, "The bridge is closed." A logistics message says, "Dock 7 is available." A clinical note says, "The patient is taking Drug X." Each claim may be true at the time of assertion. Each may be false when queried later.

If the system stores the claim without validity context, future queries inherit the original "now" without knowing it. That is how stale knowledge becomes false knowledge.

The receiver needs enough temporal metadata to distinguish at least three times:

1. source time;
2. message time;
3. query or admission time.

Source time says when the claim belonged to the source. Message time says when the claim crossed the boundary. Query time says when the receiver is trying to use it.

Different profiles may require different precision. A public encyclopedia fact may tolerate coarse validity. A regulated transaction cannot. A vehicle alert may expire in seconds. A legal statute may require effective date and repeal date. A medical medication list may require last verification and current status.

Validity is not one field. It is a gate.

Validity as Boundary Discipline

Validity travels as part of context. A Synapse or HFF bundle should carry or reference enough validity information for the receiver to ask the right question. What is the source time? What is the assertion time? What is the valid-from window? Is there a valid-until? Does the fact require a condition? Is it definitional, empirical, bounded, situational, or perspectival? Was it

extracted from a historical document? Is it current world knowledge? Does the receiver need to check for supersession?

A fact marked as stable means stable for the operational purpose and profile that admits it. A receiver may still reject the classification. A Trust Lens may require a fresher source. A regulated profile may require explicit validity windows. A high-risk command may refuse any payload with inferred or missing validity.

Validity is therefore another boundary discipline.

The claim crosses time.

Without a validity window, the receiver does not know which time the claim belongs to. With a validity window, the receiver can admit the claim as historical, current, conditional, disputed, expired, or insufficiently scoped.

This distinction becomes essential in federated systems. A local graph may store a fact from a document. A World KG may store a current public fact. A Knowledge Pack may store a rule. A receiver may query all three. If the system cannot distinguish document time from query time, it will confuse memory with reality.

The protocol should prefer honest qualification over false simplicity.

The answer “true as of the source” is often better than the answer “true.” The answer “valid under these conditions” is better than the answer “always.” The answer “requires currentness check” is better than a confident error.

Machines do not need every fact to be timeless.

They need every fact to declare the time under which it asks to be believed.

Validity Table

Claim type	Example	What must travel	Receiver behavior
Definitional	“A triangle has three sides.”	definition frame, geometry profile	admit as stable under that frame
Conditional physical	“Water freezes at 0°C.”	pressure, purity, measurement frame	admit only under declared conditions

Continued on next page

(Continued)

Claim type	Example	What must travel	Receiver behavior
Stable public instance	“Paris is the capital of France.”	source, jurisdiction, supersession policy	check for authoritative supersession
Temporal personal state	“Tom is married.”	valid-from, valid-until if known, source time	answer relative to query time
Situational state	“Dock 7 is open.”	assertion time, expiry, sensor/source	require freshness or refuse
Perspectival claim	“The load is heavy.”	comparison class, speaker/lens, threshold	evaluate under receiver policy

This table is not a metaphysical taxonomy. It is a receiver’s survival guide. Each row tells the receiving system what kind of context the claim needs before it can be admitted. The table does not make the claim true. It prevents the receiver from using the claim as if it belonged to a different class.

The most common failure in machine reasoning is not dramatic hallucination. It is the quiet use of a true sentence outside the frame that made it true.

The Receiving Question

Validity is a receiver question.

The sender can provide timestamps, source dates, valid-from windows, effective dates, expiration dates, and condition frames. The sender can declare what it believes the fact to be. But the receiver decides what kind of validity it needs for the act being considered.

A historical research agent may admit “the bridge is closed” as a true statement about a 2020 article. A navigation agent in 2026 must not use it as current route guidance. A journalist may surface the claim as part of an archive. An emergency vehicle must reject it unless current road authority confirms it. The source sentence is the same. The receiver’s purpose changes the admissibility burden.

This is why validity belongs in The Third Protocol rather than only in a knowledge-storage book. Facts travel. They travel into receivers with different purposes. The protocol cannot know every future use, but it can require the fact to carry enough context for the receiver to make a decision.

The receiver may ask for more. It may require a fresher source. It may require an authority pack. It may require a validity window that the sender did not include. It may admit the claim for background but refuse it for action.

This creates a useful asymmetry. The sender is responsible for making the claim inspectable. The receiver is responsible for deciding what inspection is sufficient.

That is how facts move without pretending every receiver has the same risk.

What HFF Refuses to Hide

Most message formats are proud of hiding complexity. They give developers fields and payloads. They let application code decide what those fields mean. That works when the sender and receiver already share context. It fails when strangers exchange meaning.

HFF refuses to hide the things that make meaning admissible. It exposes version. It exposes profile. It exposes dependency material. It exposes provenance. It exposes integrity material. It exposes receipt expectations. It makes the message a little heavier so the receiver is not forced to invent the missing context silently.

This is an engineering trade. The compact message that omits context is easy to send and hard to trust. The HFF message that carries context is heavier but easier to judge. In low-stakes systems, the compact message may be acceptable. In systems where machines act on received meaning, hidden context is a liability.

The design therefore separates logical completeness from encoding efficiency. The logical HFF object carries what the receiver needs. Encoding profiles decide how compactly that object travels. JSON makes the object readable. CBOR makes it more compact. Future binary profiles may make it faster for constrained systems. None of these encodings are allowed to change what the logical object means.

This is the heart of HFF's restraint. It does not claim that every message must be large. It claims that every message must be complete enough for its profile.

PART III

The Wire Protocol

HFF, AFP, capability exchange, and sidecars

CHAPTER 9

HFF: Meaning as Portable Context

Meaning is a liability until anchored in structural proof. The envelope provides the context, but the receiver's refusal is the final law of the boundary.

The Trust Boundary

A Synapse that lives inside a single system needs no envelope. The reasoning graph already knows its sender, the version of the Core Lexicon that produced it, and the trust policy under which it was ingested. Internal Synapses are contextualized by the system that holds them.

The moment a Synapse crosses an organizational boundary, that context disappears. The receiver has never encountered this sender. The receiver does not know which Core Lexicon version the Synapse was extracted against, whether the sender's provenance chain is intact, or whether the payload type is one the receiver is equipped to evaluate. The receiver cannot safely reason over an object whose provenance, version, and authority context are unknown.

This is the trust boundary problem. It is not solved by the Synapse itself, which defines what a portable fact looks like. It requires an outer structure that carries or references the context a receiver needs to evaluate the message before admitting its payload to the reasoning graph.

HFF is that outer structure. It is the boundary-crossing envelope for SGF objects.

Envelope Anatomy

An HFF message is a logical structure that separates five concerns: identity, payload, lexical hydration, integrity, and security.

Identity fields declare who sent the message, when, and under what version of the protocol. A receiver inspecting an HFF message knows immediately which sender to evaluate, which version of the HFF and SGF Core specifications governs the message, and whether the message is within its declared validity window. Without these fields, a receiver has no anchor for any subsequent evaluation step.

The payload carries the SGF objects the message is transmitting: Synapses, Synapse groups, frames, proof traces, gap reports, lexicon manifest entries, AFP acts, or combinations of these. The payload type governs what the receiver is expected to do with the contents. A payload carrying an INFORM act is handled differently from a payload carrying a COMMAND act, because the admission path and the authority requirements differ. HFF carries the act; AFP defines what acts mean.

Lexical hydration is the mechanism by which non-core terms in the payload become interpretable to a receiver who has not previously encountered them. Non-core terms are any identifiers whose namespace is not the Core Lexicon: local entity identifiers minted during document ingestion, domain lexicon entries specific to an industry or organization, or temporary identifiers assigned during a mission or session. HFF requires that every non-core term in the payload be accompanied by an inline lexicon manifest entry or a reference to an external scoped lexicon with version and integrity material. A receiver who cannot hydrate a term from the material in or referenced by the envelope should refuse the message rather than admit an ungrounded identifier to its reasoning graph.

Integrity fields carry the content hash of the payload, the sender's signature over the canonical bytes of the message, the key identifier, and the trust anchor reference. These fields answer the integrity and authenticity questions: did the content change after it was signed, and which key signed it? They do not answer the authorization question, which requires evaluating whether the authenticated sender is permitted to perform the declared act. Authorization is a local policy decision at the receiver.

Security fields carry the replay-prevention material: the message identifier, the creation timestamp, the expiry window, and the nonce. A message whose message identifier has already been processed, whose nonce has been seen within the replay window, or whose expiry has passed should be rejected. Valid signatures on expired or replayed messages are irrelevant to this decision: the replay prevention is structural, not cryptographic.

Profiles and Encodings

The logical HFF message model is stable across encoding choices. A receiver implementing the model can process messages regardless of whether they arrive as canonical JSON, CBOR, MessagePack-compatible binary, or another declared encoding profile. What matters is that the logical structure is preserved and that the encoding declares its canonicalization rules, its hash and signature byte conventions, and its hydration behavior.

The canonical encoding is HFF JSON. It is the reference against which all other profiles must be validated. An alternate encoding is conformant only if it round-trips to canonical JSON without semantic loss. This requirement is not a preference. It is the condition under which multiple encoding profiles can coexist without fragmenting the protocol into incompatible dialects. When two implementations serialize the same HFF message under different profiles and then canonicalize both, the canonical forms must match. If they do not, one implementation is non-conformant.

Binary encoding profiles exist because the logical message model imposes no latency requirements, but some deployment contexts do. An embedded vehicle system operating in a safety-critical broadcast environment has real timing constraints that canonical JSON cannot meet on resource-constrained hardware. The binary profile trades human readability for parse efficiency while preserving the logical model. A system that cannot parse binary reverts to JSON over a backhaul transport. The grammar and the semantics are the same. Only the serialization differs.

Encoding profiles are declared in the message itself, not inferred by the receiver from context. A receiver that encounters an undeclared encoding should reject the message rather than guess. Guessing at encoding introduces the same class of error as guessing at intent.

CBOR and the Reference Form

CBOR belongs in the protocol as an encoding profile, not as a competing meaning model.

Canonical JSON is the reference form because it is inspectable. Developers can read it. Auditors can reason about it. Test suites can compare it. Early implementations can debug it without specialized tooling. When a disagreement arises about what a message meant, canonical JSON gives the parties a common text form to examine.

CBOR serves a different purpose. It is compact and binary. It can reduce message size and parsing overhead for machines that already agree on the HFF logical object. A vehicle, robot, embedded device, or high-volume service may prefer CBOR once both sides support it. The capability manifest declares that support. The profile selects it. The receipt records it.

The danger is to treat compact encoding as semantic authority. CBOR does not make a message more true, more current, more authorized, or more admissible. It only encodes the same logical HFF object differently. A receiver that cannot round-trip the CBOR message to canonical HFF form should not admit it as equivalent.

This gives the protocol a simple rule:

Use JSON to define and debug meaning.

Use CBOR when both machines can prove they are preserving it.

The encoding changes the skin. It does not change the body.

The Transport Boundary

HFF defines what travels. It does not define how it travels.

The choice of network transport, radio spectrum, carrier protocol, or routing infrastructure is a deployment concern that HFF deliberately does not address. An HFF message can ride over HTTPS with TLS, over a vehicle-to-vehicle radio channel, over a satellite link, over a local wireless broadcast, or over any other transport layer that can deliver ordered bytes. The HFF logical model is the same in all of these environments. The transport layer provides delivery; HFF provides meaning.

This scope boundary is not an omission. It is a deliberate claim about where the division of responsibility belongs. Transport engineers understand

spectrum allocation, bandwidth constraints, link-layer protocols, and medium-specific failure modes better than a semantic protocol specification can. Semantic protocol engineers understand content structure, lexical hydration, provenance, and admission logic better than transport specifications can. Putting both in the same document produces a specification that does neither well.

The implication for real deployments is that domain profiles handle the transport coupling. An automotive domain profile declares how HFF messages are carried over the relevant vehicle communication technology. A medical device profile declares how HFF messages are routed through a hospital network or an implant communication channel. Each domain profile inherits the HFF logical model and adds the transport specifics appropriate to its operating environment. The HFF specification itself remains transport-agnostic and is therefore reusable across all of them.

Claims about latency, bandwidth, and delivery guarantees belong in domain profiles, not in HFF core. Any claim that a particular encoding profile achieves a specific latency figure in a specific deployment context is a claim about an implementation against particular hardware and operating conditions, not a claim the logical model can make.

Portable Context

An HFF message is a portable context.

The phrase deserves unpacking. A context is the set of presuppositions a receiver must hold to correctly evaluate a claim. Normally, context is ambient. Parties to a conversation share it without needing to transmit it explicitly, because they were both present when it was established. At a trust boundary, neither party was present for the other's context-building. The sender's assumptions about what the receiver knows are guesses unless explicitly verified.

HFF converts those guesses into declared structure. The message carries the sender identity for evaluation. The message carries the version identifiers for checking compatibility. The message carries the integrity material for verifying the payload was not altered in transit. The message carries the lexicon material for hydrating non-core terms. The message carries the security material for replay prevention. The receiver does not need to ask the sender for any of

this information. The message is self-contained with respect to the evaluation steps the receiver must perform before admitting the payload.

The phrase “portable context” captures what distinguishes HFF from a payload format. A payload format carries bytes. HFF carries bytes, validity conditions, dependencies, sender identity, freshness window, and integrity material. The payload alone would require a bilateral agreement about every one of these dimensions before the first message could be safely processed. HFF makes those agreements structural rather than bilateral, which is what allows strangers to coordinate.

A Synapse traveling inside an HFF message is not only a portable fact. It is a portable fact inside a portable context. The two together constitute a message that a receiver who shares the Core Lexicon and the HFF specification can evaluate on arrival, without any prior relationship with the sender.

The Cost of Being Too Helpful

Receivers will be tempted to be helpful. A receiver that sees an unknown term may try to infer it. It may search its local graph. It may compare embeddings. It may ask a language model. It may map the term to the nearest known concept and proceed.

That behavior is useful in search. It is dangerous in admission.

A search system can return approximate matches because the user remains in the loop. A machine admission system may feed the result into action. The cost of a bad nearest-neighbor match is no longer a poor search result; it may be a wrong shipment, a bad prescription, an unsafe maneuver, or a false compliance decision.

The Stranger Rule exists to make helpful guessing unnecessary. The sender carries the dependency material. If the receiver does not have enough to hydrate the term, the receiver asks, quarantines, or refuses. This may feel slower than guessing. It is slower in the same sense that checking credentials is slower than waving someone through a secure door.

The protocol does not reward helpfulness at the boundary. It rewards disciplined refusal.

CHAPTER 10

The Stranger in the Envelope

Meaning is a ledger of anchored references; a term lacking a path to the Core is a dangling pointer. A system that admits ungrounded strangers invites the void into its own logic.

The Dangling Pointer

A receiver evaluating an HFF message faces a specific failure mode that well-formed messages do not protect against by default: the payload may reference an entity the receiver has never encountered.

The Core Lexicon covers the stable shared vocabulary that all conforming participants are expected to hold: generic event types, physical categories, biological taxa, geographic regions, temporal concepts, logical relations, and other terms that can be determined in advance and maintained as a shared substrate. A Core Lexicon entry for “person” or “vehicle” or “treaty” is available to any receiver, regardless of what domain it serves or what documents it has processed.

Most real payloads contain identifiers that do not live in the Core Lexicon. A document ingestion pipeline mints local entity identifiers for the specific people, organizations, locations, and artifacts mentioned in that document. A business system maintains a domain lexicon of terms specific to its industry, organization, or product line. An autonomous vehicle assigns temporary identifiers to detected objects in its local environment. These identifiers are meaningful within the system that created them. Outside that system, they are unknown strings.

When an HFF message carries a Synapse that references a local identifier, the receiver encounters a dangling pointer. The Synapse says that a particular entity performed a particular action. The entity identifier is not in the receiver's Core Lexicon. The receiver has no parent class for the entity, no properties to inherit, no slot in any existing reasoning chain to place the entity into. Admitting the Synapse would write a node with no grounding into the receiver's reasoning graph. The node would be structurally isolated: present but unreasonable.

The failure is not a transport failure. The message arrived intact. The payload is well-formed. The Synapse is correctly structured. The failure is a semantic failure: the payload references a term that the receiver cannot hydrate, and without hydration, admission is unsafe.

The Stranger Rule

HFF addresses this failure through a requirement that any non-core term in the payload arrive with the material needed to hydrate it.

The requirement is named the Stranger Rule in the dependency bundle architecture. Its definition is direct: any Canonical ID whose namespace is not the Core Lexicon must include, in the same message, either an inline lexicon manifest entry or a resolvable reference to a scoped lexicon with version and integrity material. The inline entry must include at minimum a canonical identifier, a lemma or microgloss, and an IS_A link that traces back to the Core Lexicon. The IS_A link is the ascending anchor that gives the receiver a foothold in shared vocabulary.

The ascending anchor is what makes the mechanism work at scale. The receiver does not need to understand the full semantics of `biz.acme.priority_customer` to evaluate a Synapse that references it. The receiver needs to know that this identifier is a subclass of `person` in the Core Lexicon. From that anchor, the receiver's reasoning graph inherits the full property chain of `person` and all its ancestors. The stranger entity becomes immediately grounded in the shared ontology, even though the receiver has never encountered that specific identifier before.

If the entity is nested in a multi-level domain hierarchy, the ascending path must traverse from the local entity up through intermediate levels to the

first Core Lexicon ancestor, and package that chain. A receiver who cannot determine the Core Lexicon ancestor from the material in the message cannot safely hydrate the entity. An entity that cannot be traced back to Core remains a stranger, and the message carrying it should be refused or quarantined pending resolution.

Dependency Bundles

The structural mechanism for carrying stranger hydration material is the dependency bundle.

An HFF message carrying non-core terms is partitioned into two logical blocks. Block 1 is the dependency payload: the collection of lexicon manifest entries for every non-core identifier in the message. Block 2 is the event payload: the Synapses, frames, and AFP acts that constitute the message content. The partition is not cosmetic. It determines processing order.

A receiver must process Block 1 before Block 2. This sequencing constraint is strict. Until the dependency entries have been ingested and the stranger entities have been anchored in the receiver's local graph, the event Synapses in Block 2 may contain dangling pointers. Processing Block 2 first and encountering an unknown identifier at that point leaves the receiver with two bad options: admit an ungrounded node, or halt mid-payload. The dependency-first constraint eliminates both bad options by ensuring that every identifier referenced in Block 2 is resolvable before Block 2 is touched.

After Block 1 is processed, each stranger entity is present in the receiver's local graph with an IS_A tether to Core. Block 2 can then be processed without encountering unknown identifiers. Every spoke in every Synapse resolves to a grounded node. The resulting graph entries are immediately available for reasoning, even though the receiver had never encountered those entities before receiving the message.

The partition and sequencing together are what make HFF a self-contained dependency bundle rather than a payload format. A payload format delivers the content. A dependency bundle delivers the content together with what is needed to evaluate it.

Hydration Is Not Trust

A critical boundary separates hydration from admission.

Hydration is the mechanical act of resolving an identifier to a grounded node with a Core Lexicon anchor. It answers the question: can the receiver place this term in a known category and reason over it? Successful hydration means the term is no longer a dangling pointer. It is now a node with an inherited property chain.

Admission is a separate question entirely. Admission is the policy decision about whether a Synapse, once its terms are hydrated, should be written into the authoritative reasoning graph, routed for further evaluation, placed in a quarantine graph, or rejected outright. Admission depends on authority, freshness, plausibility, local policy, safety profile, and risk class. None of these are determined by hydration.

A message carrying a hydration-complete entity representing an ambulance can still be refused if the sender's credentials do not match an emergency vehicle registry, or if the location plausibility check fails, or if local policy requires human review for emergency-class acts. The hydration step verified that the receiver can reason over the entity. The admission step verifies that the receiver should act on the claim.

This separation matters because conflating the two produces a system that admits claims solely because they are structurally coherent. Structural coherence is necessary for admission but not sufficient. A well-formed Synapse with a correctly hydrated entity and a valid provenance chain can still be false, unauthorized, or outside the receiver's operating domain. The protocol requires hydration as a gate. Local policy determines admission.

Caching and Profiles

The dependency bundle mechanism introduces overhead: every message containing non-core terms must carry or reference hydration material. For established communication relationships, this overhead can be reduced without changing the logical requirement.

A receiver that has already processed a particular domain lexicon version, and verified its integrity against the issuer's signature, can cache the lexicon for the duration of its validity window. A sender communicating with a known

partner who has declared support for that lexicon in its capability manifest does not need to include the full dependency entries inline. It can reference the lexicon by identifier and version, and the receiver resolves the reference from cache.

The protocol still requires that every message be lexically hydratable. The difference is between carrying the hydration material inline and referencing it from a cache. Both satisfy the requirement. The cache reference form is more efficient for sustained communication between known participants. The inline form is correct for first contact, when the receiver's cache state is unknown.

Profile declarations in the capability manifest govern which caching and reference strategies are permitted in a given deployment context. A high-security domain may require inline material for every message, so that the integrity of every hydration chain is independently verified against the message's own signature. A lower-risk domain may permit cache references with declared expiry windows. The logical model is the same in both cases. The profile adjusts the operational behavior within the model's bounds.

The principle does not change with caching strategy: the receiver must be able to hydrate every non-core term in the message before evaluating the event payload. The message is a small world fragment arriving with papers. The papers are the dependency bundle, however they are carried.

Hydration Is Not Promotion

Hydration does not always mean permanent import.

A receiver may hydrate a non-core term for the life of a message, for the life of a thread, for a session, for a profile, or permanently into a local graph. These are different admission levels. Treating them as one creates contamination risk.

For example, an emergency vehicle may send a route-segment identifier that the receiver hydrates only for the current broadcast. The receiver does not need to store that segment forever. It needs to know enough to decide whether it is in scope and whether local action is safe.

A supplier may send a product-category term that a procurement system hydrates for a negotiation thread. If the negotiation ends without agreement,

the receiver may retain the term only as part of the record, not as part of its active product taxonomy.

A regulated Knowledge Pack may introduce terms that a receiver admits permanently under versioned governance. That is a stronger act. It should leave a clearer receipt.

The protocol should therefore distinguish hydration from promotion. Hydration lets the receiver understand a message. Promotion lets the receiver treat a term as part of its own usable graph. Many terms should be hydrated without promotion.

This distinction prevents federation from becoming silent assimilation.

CHAPTER 11

Knowledge Packs

Knowledge Packs transform domain context into portable, inspectable infrastructure. They do not grant truth; they define the boundaries of admissible action.

A dependency bundle is context for one message.

A Knowledge Pack is context that has learned to travel.

The difference matters because real systems repeat themselves. A hospital does not send one medical term once. A pharmacy network does not define one drug once. A warehouse does not explain one SKU once. A legal system does not ship one statute once. Domains carry vocabularies, rules, profiles, evidence standards, validity patterns, and local conventions that appear again and again.

If every HFF message must carry all of that context inline, the protocol becomes heavy. If the context is omitted, the receiver gets dangling meaning. The answer is to make repeated context into a portable object.

That object is the Knowledge Pack.

More Than an Ontology Download

A Knowledge Pack is more than an ontology download. It is not a PDF manual. It is not a pile of JSON. It is a versioned bundle of domain meaning designed for admission.

It may include domain lexicon entries, Core Lexicon references, mini-lexicon entries, profile constraints, validity rules, canonical IDs, evidence policies, source lists, dependency manifests, example Synapses, and conformance tests. It may define how a domain handles unknown terms, how claims

must cite sources, which profiles apply to which acts, which validity fields are required, which authority references matter, and which kinds of receipts are expected.

The receiver does not trust it automatically.

The receiver inspects it, versions it, caches it, admits it, refuses it, or restricts it under policy.

This turns repeated dependency transfer into infrastructure. A sidecar serving a pharmacy network may load a pharmaceutical interaction pack. A municipal service may load a traffic-law pack. A legal research system may load Louisiana civil procedure. A warehouse robot may load a facility topology pack. A financial agent may load an instrument taxonomy pack. A drone fleet may load a restricted-airspace pack.

The pack does not make every claim true. It makes the domain context explicit enough to govern.

The Pack Boundary

A Knowledge Pack has a boundary. It should declare its namespace, publisher, version, scope, Core Lexicon dependencies, domain profiles, license, provenance, update policy, and supported act types. It should say whether it is authoritative, advisory, experimental, private, public, regulated, or obsolete. It should include hashes and receipts so receivers can know what they loaded and when.

The manifest matters because packs can conflict. One pharmaceutical pack may use a conservative interaction threshold. Another may include newer research but weaker provenance. One legal pack may be official. Another may be an annotated commercial product. One public encyclopedia pack may be broad and uneven. Another domain pack may be narrow and audited.

The protocol does not erase those differences. It makes them visible at the boundary.

A receiver can pin a pack version. It can accept only packs signed by a recognized publisher. It can treat a pack as advisory. It can load a pack only for QUERY and INFORM, but refuse to use it for COMMAND or regulated action. It can run a Trust Lens over competing packs. It can require human review when packs disagree.

This is why Knowledge Packs belong inside The Third Protocol. They are more than content. They are portable context under admission policy.

Domain Examples

A medical pack may include drug classes, interaction rules, dose units, contraindication vocabulary, and required source authority. It may declare that certain terms require currentness checks or that particular interactions must be treated as high-risk.

A legal pack may include jurisdictions, statutes, effective dates, repeal dates, court levels, definitions, and citation rules. It may declare that a statutory claim without source span and effective date is inadmissible.

A municipal pack may include traffic zones, road classifications, emergency corridors, permit categories, and public broadcast rules.

A financial pack may include instrument identifiers, transaction types, settlement calendars, regulatory categories, and required audit fields.

A warehouse pack may include facility topology, zone categories, robot lanes, hazard classes, product identifiers, and local safety profiles.

Each pack gives machines a domain context they did not have before.

But the pack does not override the receiver. It is offered. It is inspected. It is admitted or refused.

Knowledge Packs and the Global Ark

Knowledge Packs also prevent the Global Ark from becoming one monolithic graph. Domain knowledge can travel as a pack. Public knowledge can travel as a pack. Private enterprise meaning can remain private while still exposing limited protocol surfaces. Commercial packs and commons packs can coexist because receivers evaluate provenance, governance, and fit.

This is portable knowledge without universal surrender.

The pack travels.

The receiver remains sovereign.

A Knowledge Pack Manifest

A Knowledge Pack should be inspectable before it is useful. Its manifest is not decoration. It is the admission surface.

Manifest field	Purpose
pack identifier	stable name for the pack
publisher namespace	who produced or maintains it
version	which release is being used
scope	domain, jurisdiction, organization, or profile boundary
Core Lexicon dependency	required Core release or compatible range
included lexicons	domain terms and mini-lexicon entries
supported profiles	which HFF/AFP/domain profiles the pack supports
validity policy	how currentness, effective dates, and supersession are handled
evidence policy	required source types, hashes, citations, or authority levels
disclosure policy	what may be public, protected, or receipt-only
conformance tests	examples a receiver can run before admitting the pack
deprecation policy	how old versions expire or remain usable

The manifest lets the receiver decide what kind of thing has arrived. A public encyclopedia pack and a regulated pharmaceutical pack should not be admitted under the same assumptions. A municipal traffic pack and a private warehouse pack may both contain location terms, but the authority behind those terms differs. A legal pack may require effective dates and official source spans. A product pack may tolerate internal namespaces and private categories.

The Knowledge Pack is therefore not just content. It is content with declared responsibility.

Pack Failure Modes

Knowledge Packs also fail in predictable ways.

A pack may be stale. Its version may predate a regulatory change, a road closure, a drug interaction update, or a product rename.

A pack may be overbroad. It may claim to cover “medical knowledge” when it covers only one formulary or specialty.

A pack may be under-sourced. It may contain useful terms but weak evidence.

A pack may collide with another pack. Two jurisdictions may define the same term differently. Two pharmaceutical packs may disagree about severity thresholds. A company may keep a private product taxonomy that conflicts with a public industry taxonomy.

The protocol does not pretend these failures disappear. It gives the receiver a way to see them. Version, scope, evidence, and profile declarations let receivers admit a pack narrowly rather than trust it globally.

This is how domain knowledge travels without becoming a Trojan horse.

Three Kinds of Packs

Knowledge Packs are easiest to understand if divided into three families.

The first family is the **reference pack**. A reference pack supplies relatively stable domain context: terms, definitions, relationships, common attributes, validity rules, and source references. A public animal taxonomy pack, a pharmaceutical drug-class pack, or a municipal road-classification pack belongs here. The pack may be updated, but its role is to provide a shared background against which messages can be interpreted.

The second family is the **operational pack**. An operational pack is tied to action. It may contain facility maps, robot zones, inventory categories, equipment identifiers, safety policies, permitted actions, and receiver expectations. A warehouse topology pack is operational. A fleet profile for a delivery network is operational. A hospital handoff pack that defines referral types, responsible roles, and admission rules is operational. These packs are more dangerous because they can affect behavior. They need stronger versioning, stronger authority, and often narrower scope.

The third family is the **regulatory pack**. A regulatory pack carries law, policy, compliance constraints, procedural requirements, or externally imposed rules. It might describe city ordinances, state statutes, securities rules, export-control categories, privacy obligations, or medical reporting requirements. Regulatory packs are more than informative. They shape what a receiver is permitted to do. They therefore need effective dates, repeal dates, jurisdiction, official source anchoring, and a clear distinction between the text of the rule and any interpretation of the rule.

These three families can overlap. A pharmaceutical pack can be reference knowledge for an assistant and regulatory knowledge for a clinical workflow. A road pack can be reference knowledge for a map renderer and operational knowledge for an autonomous vehicle. The family is not only a property of the pack. It is a property of how the receiver uses it.

That is why the manifest must declare intended use. A pack that is safe for QUERY may not be safe for COMMAND. A pack that is useful for explanation may not be sufficient for regulated execution. A pack that helps a chatbot answer questions may not authorize a robot to move.

The receiver's policy decides which family the pack belongs to in practice.

Pack Admission

Loading a Knowledge Pack should itself be an admission event.

The receiver should not silently import a pack and begin reasoning over it as if it were native truth. The pack crosses a boundary. It should be parsed, checked, versioned, and admitted under a policy. The receiver may accept the pack for read-only use, accept it for low-risk recommendations, reject it for action, quarantine it pending review, or accept it only under a specific Trust Lens.

This matters because packs can carry errors at scale. A single bad dependency in a one-off HFF message affects one message. A bad Knowledge Pack can affect every message that depends on it. A mislabeled drug class, stale statute, wrong unit conversion, or overbroad safety rule can propagate across many decisions.

The admission receipt for a pack should record:

- pack identifier and version;

-
- publisher namespace;
 - source or authority references;
 - Core Lexicon dependencies;
 - profiles enabled;
 - profiles refused;
 - trust lens or policy used;
 - date of admission;
 - deprecation or review schedule.

This record is boring. It is also the difference between a domain-aware system and a system that swallowed a zip file and hoped.

Knowledge Packs are powerful because they make domain context portable. They are dangerous for the same reason.

The cure is not to avoid them. The cure is to admit them like any other consequential meaning.

The Same Payload, Three Different Worlds

The payload “the bridge is closed” can inhabit three different worlds.

As **INFORM**, it contributes a claim to the receiver’s state. The receiver may update its map, lower route confidence, or request corroboration. No action is requested.

As **ADVISE**, it recommends behavior. The sender is not claiming authority over the receiver, but it is suggesting that the receiver should consider alternate routing. The receiver may follow the advice or decline it under local policy.

As **COMMAND**, it asserts authority. The sender is more than describing or advising. It is attempting to direct the receiver’s action. The receiver must check authority, scope, freshness, and safety before admitting the act.

Same payload. Three worlds.

This is why **AFP** exists. Payload semantics and act semantics are different axes. A system that treats the payload as sufficient will eventually confuse information with direction, direction with authority, authority with execution, and execution with confirmation.

Human beings often repair these distinctions from context. Machines need the act to be declared.

CHAPTER 12

AFP: The Speech-Act Layer

Information describes the state, but the act binds the future. A machine that cannot distinguish a request from a command has abdicated its sovereignty, leaving its gates open to the chaos of ungrounded interpretation.

Confusion pair	Correct distinction
ACK vs ACCEPT	ACK means received. ACCEPT means proposal accepted.
ACCEPT vs CONFIRM	ACCEPT admits a commitment. CONFIRM records completion or verified state.
ADVISE vs REQUEST	ADVISE provides guidance. REQUEST asks the receiver to consider action.
REQUEST vs COMMAND	REQUEST may be declined under ordinary policy. COMMAND claims authority and requires stronger gates.
REFUSE vs ERROR	REFUSE means understood but not performed. ERROR means invalid or not processable.

Content Without Act

Two machines can share a completely grounded, structurally valid HFF message and still fail to coordinate. The failure is not in the payload. The payload correctly describes a road hazard, a routing change, or a resource request. The failure is in the silence around the payload: the receiver does not know what the sender intends to be done with it.

A payload about a road hazard could be an observation the sender is sharing because it may be useful. It could be a recommendation that the receiver adjust its route. It could be a request that the receiver slow down. It could be a command, issued by an entity with authority over the receiver's operating parameters, that the receiver must comply with. The payload content does not distinguish these cases. The routing structure, the authority requirements, the response obligations, and the admission path differ across all of them, but the Synapse describing the hazard is the same.

Natural language handles this through prosodic and contextual cues: tone of voice, sentence structure, prior discourse, the relationship between the parties. These cues are unavailable in a machine protocol. Inferring act type from content and context is exactly the probabilistic interpretation the protocol is designed to remove from the action path.

AFP is the layer that removes the inference requirement. AFP requires every message to declare its act type explicitly. The receiver does not reason about what the sender meant. The receiver reads the act field and routes accordingly.

Acts as Protocol Primitives

The AFP act set has thirteen types, each representing a distinct category of machine participation.

INFORM declares that the sender is sharing a fact, observation, or state update. The sender makes no demand on the receiver's behavior. The receiver is free to incorporate the information, route it, or discard it according to local policy. No response obligation is created. An ambulance broadcasting its location and heading is performing an INFORM. Nearby vehicles are free to use the information for collision avoidance, but they are not obligated to respond.

ADVISE declares that the sender is sharing a recommendation. The sender believes the receiver should consider a course of action, but the sender does not claim authority to require it. The receiver may follow the advice, evaluate it against local policy, or decline without creating a protocol breach. An advisory system suggesting an alternate route to a vehicle experiencing congestion is performing an ADVISE. The vehicle's local policy determines whether the suggestion is acted upon.

REQUEST declares that the sender is asking the receiver to take an action and acknowledges that the receiver may decline. REQUEST does not require authority. It creates an obligation to respond with ACCEPT, REFUSE, or a counter-proposal through PROPOSE. A logistics system asking a warehouse agent to prepare a shipment window is performing a REQUEST. The warehouse agent is expected to respond.

QUERY is a request for information rather than action. The receiver is expected to respond with an INFORM or an ERROR. No action obligation is created.

COMMAND declares that the sender is directing the receiver to take an action and claims authority to require it. COMMAND requires authority validation. A receiver who cannot verify the sender's authority for the declared act should refuse the COMMAND through REFUSE or report a protocol failure through ERROR. A receiver who blindly obeys unverified COMMANDs is not implementing AFP. It is implementing a remote execution endpoint with no access control.

PROMISE declares that the sender commits to a future action or outcome. A PROMISE creates an auditable obligation. When the committed action is completed, it is recorded through CONFIRM. If the commitment cannot be fulfilled, it is cancelled through CANCEL with a reason. PROMISE is the mechanism by which machine coordination produces accountable commitment graphs rather than stateless message exchanges.

PROPOSE declares an offer or proposal that the counterparty may accept, refuse, or counter. PROPOSE initiates a negotiation thread. ACCEPT closes it positively. REFUSE closes it negatively. A counter through a subsequent PROPOSE with modified terms continues it. The conversation state is tracked by the thread identifier, which links all acts in a negotiation to the same parent event.

ACCEPT confirms agreement with a PROPOSE or REQUEST. REFUSE rejects a PROPOSE, REQUEST, or COMMAND. CANCEL terminates a previously committed or proposed action. CONFIRM records that an action was completed or a state was verified.

ACK confirms receipt of a message. ACK is not agreement. A receiver who ACKs has acknowledged that the message arrived and was formally received. It has not agreed to any content, committed to any action, or validated any claim. The distinction between ACK and CONFIRM, and between ACK and ACCEPT, is not cosmetic. Systems that conflate them will produce commitment graph

errors: logging confirmation of completed actions that were never performed, or logging acceptance of proposals that were only acknowledged.

ERROR reports a protocol failure. Valid ERROR contexts include invalid message structure, unsupported act type for the declared profile, missing or unresolvable lexicon material, failed authority validation, failed replay prevention, illegal conversation state transition, and unsafe action according to local safety policy. ERROR is not a refusal. Refusal of a REQUEST or COMMAND because the receiver declines the action is REFUSE. ERROR is a report that the protocol machinery itself encountered a problem.

Acts Are Not Roles

AFP act types are frequently confused with SGF semantic roles. The confusion is understandable: both are vocabularies that appear in machine messages, both use named types drawn from human action concepts, and both appear in the same documents. They are structurally distinct and must be kept separate.

A semantic role, such as HAS_AGENT or HAS_RECIPIENT, describes the structural participation of an entity in an event that the Synapse represents. The role answers “who or what plays this part in this event?” It is part of the event’s internal structure.

An AFP act type describes the participation move the sender is performing with the message. It is not internal to any event. It is a property of the message as a communication act between sender and receiver. COMMAND does not mean that the entity filling HAS_AGENT in the payload is giving a command. It means that the sender of the HFF message is performing a command act with respect to the receiver.

The practical consequence is that the same Synapse can be wrapped in any AFP act. A Synapse describing a vehicle location update can be INFORM (sharing state), REQUEST (asking the receiver to process it in a particular way), or ACK (confirming that a previously received location update was processed). The Synapse content does not change. The act type changes what the receiver is expected to do with the message.

An HAS_AGENT role in a Synapse does not create any command obligation. A COMMAND act type creates the obligation, and only after authority validation. A developer building a receiver that routes based on HAS_AGENT identity rather

than on the declared act type is building the wrong system. Act type, not role filler, determines the admission path.

Admission Paths by Act

The act type is the primary routing key for an HFF message receiver.

INFORM and QUERY route to the reasoning graph and the query engine respectively, with no authority gate required beyond message authentication. These are the lowest-friction act types because they create no obligations on the receiver beyond evaluating whether to incorporate the information or respond to the query.

ADVISE and REQUEST route to a policy evaluation layer that checks whether the sender's recommendation or request is within the receiver's operating parameters and whether the receiver's local policy requires automatic compliance, human review, or open response. The policy gate for REQUEST must produce a response: ACCEPT, REFUSE, PROPOSE, or ERROR.

COMMAND routes to an authority validation gate before any policy evaluation. The authority gate checks whether the authenticated sender holds the authority required for the declared act on the declared patient. If the authority check fails, the COMMAND must produce a REFUSE or ERROR response. It must not silently discard the COMMAND, because silent discard prevents the sender from detecting an authorization problem. If the authority check passes, the message proceeds to local policy evaluation and safety profile checks. Passing all gates produces action execution and a receipt.

PROPOSE, ACCEPT, REFUSE, CANCEL, and CONFIRM route to the conversation state machine for the thread they belong to. The thread identifier links these acts to the parent negotiation. A receiver who receives an ACCEPT without a prior PROPOSE in the same thread is in an illegal conversation state and must respond with ERROR. The conversation state machine ensures that commitment and negotiation acts are sequenced correctly and that the resulting ProofTrace accurately reconstructs what was agreed.

PROMISE routes to the commitment registry, which creates an auditable obligation with the sender as the committing party, the receiver as the monitoring party, and the declared action as the content. Obligations in the commitment

registry are closed by CONFIRM or CANCEL. Unclosed obligations are available for audit and escalation.

ACK and ERROR are protocol management acts that route to the message log and do not create semantic obligations.

Triples Describe. Acts Bind.

A grounded Synapse is a precise description of an event or state. It answers structural questions about the event with enough fidelity for a machine to reason over it. It is the most that a representation layer can do.

Representation does not produce coordination. Coordination requires that the receiver know what behavior the sender is requesting in response to the representation. Without act declaration, every message is a description looking for an interpretation. With act declaration, every message is a participation move in a structured exchange where both sides know their obligations.

The move is what turns a swarm of informed machines into a coordinatable system. A receiver who has processed a thousand INFORM messages from a thousand senders has a populated reasoning graph. It does not have a commitment graph. No sender has promised anything. No receiver has agreed to anything. No action has been authorized. Information has been shared. Coordination has not occurred.

AFP introduces accountability to machine communication. PROMISE creates obligation. ACCEPT creates commitment. COMMAND creates authority-gated direction. CONFIRM records completion. REFUSE closes the loop when agreement is not reached. ERROR reports when the loop breaks. The act types are not decorative labels on payloads. They are the difference between a protocol that describes machine states and a protocol that enables machine participation.

The Negative Capability Manifest

A capability manifest should not only say what a participant supports. It should also help the sender understand what the participant refuses.

This matters because unsupported behavior is often more important than supported behavior. A receiver may support QUERY but refuse COMMAND. It may support public signed broadcast but refuse confidential group messages. It may support HFF JSON but not CBOR. It may support Core Lexicon release A but not

release B. It may accept Knowledge Packs from a managed registry but refuse unknown packs. It may require human approval for regulated transactions above a threshold.

If the manifest declares only positive capabilities, the sender learns too little. It may attempt a message the receiver will predictably refuse. If the manifest also declares refusal policy, the sender can avoid dead paths before sending payload.

This is not a social courtesy. It is part of reducing coordination cost. A useful first contact tells the sender what is possible, what is unsupported, what is forbidden, and what requires escalation.

The strongest capability exchange is therefore not a brag sheet. It is a boundary description.

CHAPTER 13

The Capability Exchange

Compatibility is the intersection of two declared voids. A machine that cannot refuse an act it does not understand is a hazard, not a participant.

The First-Contact Problem

Two conforming participants who have never communicated before share a protocol specification and a Core Lexicon. They do not share anything else.

One participant may support the canonical JSON encoding profile and no others. The other may support binary profiles and provide JSON only on request. One may hold a current-version Core Lexicon release. The other may hold an earlier release with a different set of entries. One may accept all thirteen AFP act types. The other may accept only INFORM, QUERY, ACK, and ERROR, having been deployed in an observation-only role. One may require a HIGH_RISK_COMMAND security profile for any act with real-world effect. The other may operate under a permissive internal profile that accepts broader authority frames.

None of these differences are protocol violations. The specification permits variation in encoding profile support, lexicon version, supported act types, trust anchor requirements, and profile selection. The variation is not a failure of standardization. It is an acknowledgment that conforming participants will be deployed across environments with genuinely different operational constraints, hardware capabilities, and regulatory requirements.

The first-contact problem is the structural consequence of permitted variation: a sender who does not know the receiver's capabilities may send a message the receiver cannot process. The message may be well-formed under

the specification. It may be semantically valid. It may carry correct hydration material. The receiver may still be unable to admit it because the encoding profile is unsupported, the lexicon version is incompatible, the act type is outside the receiver's declared scope, or the security profile does not meet the receiver's minimum requirements.

Sending and hoping the receiver can handle it is trial-and-error coordination. Trial-and-error is a tolerable failure mode in some environments and an unsafe one in others. A system that issues commands to safety-critical physical actuators cannot absorb the latency and the partial-failure risk of discovering capability mismatches at message delivery time.

Capability exchange is the mechanism by which participants declare their operational constraints before the first payload message is sent. Declaration replaces trial-and-error.

The Capability Manifest

A capability manifest is a structured declaration of what a participant can understand, verify, receive, and do.

The required fields of a manifest establish the minimum information a sender needs to route its first message correctly. These fields are: the participant identifier, the versions of *SGF*, *HFF*, and *AFP* the participant supports, and the endpoints at which the participant receives messages.

Without participant identity, a sender cannot construct a correctly addressed message. Without version declarations, a sender cannot determine whether its message structure is parseable by the receiver. Without endpoint information, the sender does not know where to send the message.

The recommended fields expand the declaration to cover the full range of compatibility dimensions. Supported encoding profiles tell the sender which serialization format to use. Supported lexicons tell the sender whether the receiver already holds the lexicon material its payload depends on, or whether inline hydration is required. Supported knowledge packs tell the sender which domain-specific ontological extensions the receiver is equipped to evaluate. Trust anchors declare which signing authorities the receiver will accept as valid. Authentication methods declare how the receiver validates sender identity.

Rate limits and maximum payload sizes are operational constraints that prevent well-intentioned senders from overloading the receiver.

Supported act types are particularly important for first contact. A receiver that declares only INFORM and QUERY in its manifest is telling the sender that it will refuse any COMMAND, REQUEST, PROMISE, PROPOSE, ACCEPT, REFUSE, or CANCEL. Sending those act types is not an error in the protocol, but it will produce REFUSE or ERROR responses that consume both parties' resources without advancing coordination. Knowing in advance that the receiver supports only observation-mode acts allows the sender to route action-requiring messages to a different participant or escalate to a human.

Supported domain profiles declare any domain-specific protocol overlays the receiver is configured to evaluate. A receiver operating in a medical device context may declare a medical domain profile that requires additional safety fields and a restricted set of authorized senders. A receiver in a financial settlement context may declare a regulated-transaction profile. These declarations allow senders to select the appropriate security and payload profile before the first message.

Profile Selection

The outcome of reading a capability manifest is a profile choice.

Profile selection is not a negotiation in the sense of extended back-and-forth offers and counter-offers. It is a sender-side computation: given what the receiver has declared it can accept, what is the most capable profile that falls within the intersection of both participants' constraints?

If the receiver supports all encoding profiles, the sender may choose the most efficient one for the anticipated message volume and hardware environment. If the receiver supports only canonical JSON, the sender uses JSON regardless of its own preference. If the receiver supports a security profile that requires authority frame validation for any act beyond INFORM, the sender must include authority material in any message carrying a higher-impact act type. If the receiver's supported lexicon list includes the lexicon the sender's payload depends on, inline hydration is unnecessary. If it does not, inline hydration is mandatory.

The profile selection step is what transforms capability exchange from a courtesy into a safety mechanism. A sender that skips profile selection and defaults to its preferred configuration may produce messages that are unprocessable by the receiver. In a high-stakes environment, an unprocessable message may trigger a conservative safety response, or it may produce a failure mode that is more dangerous than a clean refusal. Profile selection ensures that the sender's first message is the receiver's processable minimum, not the sender's preferred maximum.

For participants in sustained communication relationships, manifests can be cached. A participant that has read and verified a counterpart's manifest, and received no invalidation signal, may continue using the cached profile selection for the duration of the manifest's declared validity window. This caching behavior prevents the overhead of manifest exchange from dominating short-message exchanges in high-frequency communication environments. The logical requirement, that the sender knows what the receiver can process before sending, is satisfied by the cached manifest whether the exchange was recent or historical within the validity window.

Downgrade, Dependencies, and Refusal

The three outcomes of capability evaluation other than full compatibility are downgrade, dependency request, and refusal.

Downgrade occurs when the intersection of supported profiles between sender and receiver is narrower than the sender's default operating mode. The sender adjusts its message structure to the intersection. If the receiver supports JSON but not binary, the sender uses JSON. If the receiver does not hold the lexicon the sender's payload depends on, the sender includes inline hydration. If the receiver's supported act types exclude COMMAND, the sender routes the action request through a PROPOSE and waits for ACCEPT. Downgrade is successful when the adjusted message is processable by the receiver and still communicates the sender's intent at the reduced capability level.

Dependency request occurs when the sender discovers that the receiver lacks a lexicon entry or knowledge pack that is required for the payload to be semantically meaningful, and the sender is able to supply that dependency in the message or in a separate exchange. The dependency bundle mechanism

handles this within a single message for lexicon entries. For larger knowledge packs, a capability exchange may involve a preparatory phase where the sender shares the required lexicon material and waits for the receiver to confirm hydration before proceeding with the substantive payload.

Refusal occurs when the capability intersection is empty with respect to the sender's communication need. If the sender needs to issue a `COMMAND` to accomplish its goal and the receiver's manifest declares that it does not support `COMMAND`, refusal is the correct outcome. The sender should not attempt to achieve `COMMAND` semantics through a worded `INFORM`, because intent inference is what the protocol removes from the action path. If the task cannot be accomplished at the receiver's declared capability level, the sender must route the task to a different participant, escalate to a human operator, or log the inability as a gap for system operators to address.

A swarm participant that cannot refuse is not implementing a protocol. Refusal is what separates a coordinating participant from a passive execution endpoint. Capability exchange makes refusal explicit and predictable. A receiver that declares its capability boundaries in its manifest, and then declines messages that exceed those boundaries, is behaving correctly. A receiver that accepts all messages regardless of its declared constraints is a liability, not a participant.

Compatibility as Declared State

The capability exchange specification defines the formal boundary around what "compatible" means in a machine coordination context.

Compatibility is not a bilateral property that exists in the abstract between two participants who share a specification. Compatibility in a given exchange is a function of the intersection of both parties' declared capabilities at the time of the exchange. Two participants who are compatible for `INFORM` exchange may be incompatible for `COMMAND` exchange. Two participants who are compatible under a canonical `JSON` profile may be incompatible if one requires a binary profile for timing reasons. Compatibility is session-scoped and declared, not assumed from specification membership.

This definition is operationally important because it shifts the failure mode. Without capability exchange, an incompatible message arrives at a

receiver and produces an error that the sender did not anticipate. The sender cannot distinguish “receiver is unavailable” from “receiver cannot process this message type” from “receiver does not recognize this sender’s authority for this act.” All of these produce errors. The sender must diagnose the failure after it occurs.

With capability exchange, the sender discovers the receiver’s constraints before sending. Incompatibility is detected in the manifest, not in an error response to a payload message. The sender can select an appropriate profile, route to a different participant, or surface the problem to a human operator, all before the payload is transmitted. The protocol does not make all first-contact exchanges succeed. It makes the conditions of success visible before the payload crosses.

First contact is not guessed. It is declared.

That declaration is the minimal condition under which machines that have never met can coordinate without bilateral integration agreements negotiated before deployment. The Core Lexicon provides the shared vocabulary. HFF provides the shared envelope. AFP provides the shared act vocabulary. Capability exchange provides the shared state declaration. Together, the four form the minimal substrate under which strangers can be made admissible to each other.

Encoding Negotiation

Capability exchange also decides how the message will be encoded.

This is where JSON, CBOR, and future binary profiles belong. They are not separate protocols. They are encoding profiles for the same logical HFF object. A receiver may support canonical JSON for audit and debugging, CBOR for compact machine exchange, and a domain-specific binary profile for constrained hardware. A sender may prefer CBOR. The receiver may require JSON for a particular profile. The manifest makes the negotiation explicit.

The negotiation should answer at least four questions:

1. Which HFF versions are supported?
2. Which encoding profiles are supported?
3. Which encoding is preferred for this profile and act class?

-
4. Can the chosen encoding round-trip to canonical HFF without semantic loss?

If the receiver supports only JSON, the sender sends JSON or refuses the exchange. If both support CBOR and the selected profile permits it, the sender may use CBOR. If the sender proposes an encoding the receiver does not support, the receiver should reject the message before attempting to parse it. Guessing at encoding is another form of guessing at meaning.

Encoding negotiation is therefore small but important. It prevents performance optimization from becoming fragmentation. Machines can choose compact representation without changing the meaning they carry.

The capability manifest keeps that choice visible.

Sidecar as Translator of Responsibility

A sidecar also translates responsibility.

The old service may know how to execute an operation, but it may not know how to explain that operation in protocol terms. It may not distinguish acceptance from completion. It may not store enough provenance. It may not expose refusal reasons. It may not know which local policy gate was supposed to run before an API call. The sidecar becomes the layer that names these responsibilities.

This is more than technical wrapping. It is institutional boundary work. The sidecar says: this legacy operation is what the old system can do; this AFP act is what the protocol participant is claiming to do; this mapping is the bridge; this receipt is the record.

If the sidecar cannot honestly produce that mapping, it should not pretend. It should expose only the acts it can support. A read-only sidecar is better than a dishonest action sidecar. A sidecar that returns ACK because it cannot prove completion is better than one that returns CONFIRM because a local API returned “success” without enough semantics.

This restraint makes sidecars trustworthy.

CHAPTER 14

The Sidecar Path

Replacement is a fantasy; deployment is a discipline. The sidecar is the structural boundary that forces legacy behavior to justify itself as admissible, governed meaning.

Infrastructure rarely enters the world by replacing everything beneath it. Replacement is the fantasy of architects who do not have to keep payroll systems, hospital records, warehouse controllers, municipal databases, factory lines, and old transaction engines alive while the new system arrives.

Real infrastructure enters by touching the old world without asking the old world to become new all at once.

The Third Protocol cannot require a flag day. A protocol for machine meaning cannot begin by demanding that every legacy service, every private database, every robotic controller, every enterprise queue, every fleet-management system, and every internal API become protocol-native before the first useful exchange can occur. That would turn the protocol into a migration plan, and migration plans die in the gap between correctness and deployment.

Adoption is a deployment problem, not a migration problem.

The sidecar is the deployment answer.

The Flag-Day Trap

A flag day is the day a whole institution switches from one system to another. The old interface shuts down. The new interface becomes mandatory. All parties are expected to arrive prepared. The term belongs to computing history, but the failure pattern is older than computing. Every bureaucracy has lived through a version of it: a new form, a new system, a new database,

a new reporting process, a new compliance workflow, all announced as if the world can be refactored on a calendar.

Flag days are possible when the controlled surface is small. They are disastrous when the dependency graph is large. Meaning systems have large dependency graphs. A logistics platform talks to billing, procurement, inventory, regulatory reporting, customer service, routing, telemetry, and exception management. A hospital record system talks to scheduling, pharmacy, lab systems, insurer interfaces, billing, clinical notes, device feeds, and external referrals. A factory floor system talks to robots, safety cages, supply chain platforms, human operators, maintenance logs, and compliance records.

A protocol for machine meaning cannot assume those systems will all move together. Some will not move because they are regulated. Some because they are old. Some because they are too important to disturb. Some because nobody alive fully understands them. Some because the team that built them no longer exists.

The sidecar path begins by accepting that reality.

The Two-Faced Boundary

A sidecar sits beside an existing system. One face speaks HFF and AFP. The other face speaks whatever the old system already speaks: database query, internal API, file drop, message queue, service call, robotic controller, batch job, or local SDK. The old system does not need to know that a protocol layer has appeared outside its wall. It continues to do what it already does. The sidecar becomes the meaning-facing boundary.

That boundary matters. A sidecar is more than a convenience wrapper. It is the place where local meaning becomes externally admissible.

On its outward face, the sidecar publishes a capability manifest. It declares which HFF versions it accepts, which AFP acts it can process, which profiles it supports, which Core Lexicon version it recognizes, which domain lexicons it can hydrate, which security requirements it enforces, and which acts it will refuse. On its inward face, it translates admitted requests into the old system's local operations.

This does not abolish integration. It localizes it.

Middleware usually translates for one relationship. A sidecar exposes a reusable protocol face.

That distinction carries the mechanism. Ordinary middleware is often built to connect system A to system B for a particular workflow. It knows the private assumptions of both systems. It handles one bridge. If system C arrives, another bridge appears. If system D arrives, another. The old N-squared pattern returns under a new tool name.

A sidecar does something different. It adapts once at the local boundary. From that point outward, the external face is not a private bilateral bridge. It is a shared protocol surface. Other participants do not need to learn the legacy system's private schema. They send HFF bundles. They perform AFP acts. They receive structured outcomes.

The sidecar may still contain local mapping logic. It may know that the old inventory service stores reservation state in a table called `inv_hold`. It may know that the old routing engine calls a lane segment a corridor. It may know that the warehouse controller expects a queue message rather than a direct service call. That knowledge remains local. The protocol does not pretend the old system is clean. It prevents every outside partner from having to rediscover the same dirt.

Read-Only First

The first sidecar should often be read-only.

That is not timidity. It is engineering discipline. A read-only sidecar can expose QUERY and INFORM before it accepts REQUEST or COMMAND. It can publish capability, answer grounded questions, emit HFF receipts, and prove lexical hydration without touching the action path. It lets a team install the protocol at the lowest-risk edge.

Read-only deployment also teaches the local mapping problem without consequence. The sidecar must learn how local SKUs map to grounded item identifiers, how warehouse zones map to place entities, how customer categories map to domain lexicon entries, how timestamps are represented, how the old service expresses absence, uncertainty, and error. These are not minor details. They are the places where ordinary integrations hide meaning.

The read-only phase makes that hidden meaning inspectable.

Action can come later.

When action arrives, the sidecar becomes an admission point. An incoming AFP REQUEST does not flow directly into the legacy system. The sidecar parses the HFF envelope, hydrates dependencies, verifies integrity, checks the sender, applies the profile, evaluates local authority, maps terms to legacy objects, and only then calls the old operation. If a mapping fails, it returns ERROR. If the request is understood but not permitted, it returns REFUSE. If the request is accepted but not yet completed, it returns ACCEPT. If the action completes, it returns CONFIRM.

The old service may know none of these distinctions. The sidecar knows them for it.

Inventory as a First Installation

Consider an inventory system built long before sGF. It has an internal operation that reserves stock. The operation expects a SKU, a quantity, a warehouse ID, and an account code. It may expose those through a database procedure, an internal API, or a queue. It may have no concept of HFF, AFP, Synapses, dependency bundles, Core Lexicon versions, or admission profiles.

A Third Protocol participant does not send those local fields directly. It sends an HFF bundle containing grounded entities, an AFP act, profile declarations, and dependency material for any non-core terms. The sidecar hydrates the bundle, maps the grounded objects to local fields, checks local policy, calls the old operation, then returns a receipt.

That receipt matters as much as the call.

Without the sidecar, the external participant might know only that an API returned success. With the sidecar, the participant can receive a structured account of what act was admitted, what profile governed it, what version was used, what local operation was attempted, what outcome occurred, and what evidence should be retained. The sidecar turns local execution into protocol-visible accountability.

The same pattern applies to a routing system, a document archive, a procurement service, a lab-result feed, or a fleet controller. The inside varies. The protocol face remains stable.

The Adoption Ladder

The sidecar path has a natural ladder.

The first rung is observation. The sidecar receives no external commands. It emits HFF representations of local facts. It turns local state into liquid knowledge without letting anyone act on it.

The second rung is query. The sidecar accepts QUERY and returns grounded answers, bounded by local policy and selective disclosure rules.

The third rung is request. The sidecar accepts REQUEST acts that may lead to local action, but only after admission.

The fourth rung is proposal and commitment. The sidecar participates in AFP commitment graphs, issuing PROPOSE, ACCEPT, PROMISE, REFUSE, and CONFIRM.

The fifth rung is high-risk action. The sidecar accepts COMMAND only under profiles that carry authority, safety, freshness, receipt, and refusal semantics strong enough for the domain.

No institution needs to climb all rungs at once.

This is how infrastructure enters without asking the old system for permission.

The sidecar path also protects the protocol from becoming a platform-replacement story. Platform-replacement stories frighten institutions for good reason. They imply long procurement cycles, organizational risk, frozen roadmaps, retraining, compliance review, migration debt, and operational fragility. A sidecar does not require the old world to kneel. It lets the protocol appear at the boundary.

One team can wrap one service.

A second team can wrap another.

A fleet can expose a read-only vehicle-state sidecar before accepting movement requests. A hospital network can expose a controlled referral sidecar before allowing write-back to clinical records. A warehouse can expose inventory queries before reservation. A finance system can expose transaction status before accepting regulated instructions.

Each installation increases the surface area of admissible meaning without requiring universal migration.

This does not make adoption automatic. Network effects still matter. Governance still matters. Conformance still matters. A badly built sidecar can misrepresent local meaning, hide ambiguity, or emit dishonest receipts. But those are conformance failures, not defects in the deployment path. The important thing is that the path exists.

A protocol that can be deployed only after the world changes is not infrastructure yet.

A protocol that can wrap the world as it exists has a chance.

Sidecar Failure Modes

The sidecar path is practical, but it is not safe by default. A sidecar can lie. It can over-map. It can hide exceptions. It can translate a local failure into a generic success. It can return a receipt that says less than the receiver needs. It can make an old system look cleaner than it is.

These failure modes are important because sidecars will be the first implementation path for many systems. If sidecars become ungoverned adapters with protocol labels attached, the old Babel Tax returns wearing a new uniform.

The first failure is **semantic overreach**. The sidecar maps a local category to a Core term that is close but not exact. This may be acceptable for display and disastrous for action. The cure is to declare mapping confidence, source, and profile scope. A mapping admitted for QUERY should not automatically be admitted for COMMAND.

The second failure is **receipt laundering**. The legacy system returns a vague success code. The sidecar emits a clean CONFIRM as if it knows exactly what happened. That is dishonest. If the legacy system cannot distinguish accepted, queued, executed, and completed, the sidecar must not pretend it can. It should return the strongest truthful act available and leave completion unconfirmed until evidence exists.

The third failure is **policy bypass**. The old system permits an operation that the protocol profile would have blocked. The sidecar cannot treat the legacy API as authority. The sidecar is the admission point. If the HFF/AFP profile requires a gate, the sidecar must enforce it even if the internal API would happily proceed.

The fourth failure is **hidden translation drift**. The legacy system changes a field meaning, a code list, or a status value. The sidecar continues mapping as before. Now the protocol face is stale. Sidecars therefore need their own validity and version checks against the systems they wrap.

The Sidecar Path is powerful because it allows local adoption. It becomes trustworthy only when sidecars are themselves treated as governed receivers and senders.

They are not loopholes around the protocol.

They are where the protocol first touches the old world.

Query Permission Is Not Data Permission

Federated query requires a distinction that ordinary database systems often hide: permission to ask is not the same as permission to receive everything the answer could contain.

A receiver may be allowed to ask a pharmaceutical graph whether a drug class has a severe interaction. It may not be allowed to receive the full licensed monograph behind the interaction. A vehicle may be allowed to ask a municipal graph whether a road segment is closed. It may not be allowed to enumerate every city sensor feed. A compliance sidecar may be allowed to ask whether a statute is current. It may not be allowed to scrape the entire annotated legal corpus.

The query therefore has its own admissibility surface. It declares purpose, profile, requester identity, requested scope, disclosure level, and intended use. The answering graph evaluates the query before answering. It may answer fully, answer partially, return a proof commitment, return a Knowledge Pack reference, refuse, or require a stronger credential.

This is why federated queries are part of The Third Protocol rather than a generic database feature. The query itself is an act. It crosses a boundary. The graph being queried remains sovereign.

Why Copying the World Fails

The simplest answer to federation is to copy everything. Import the encyclopedia. Import the legal corpus. Import the drug database. Import the traffic graph. Import every external fact that might someday be useful.

That answer works until it works too well.

The copied world becomes stale. Updates must be tracked. Licenses must be respected. Source authority must be preserved. Contradictions must be reconciled. Local entities must be protected from external merges. Every copy becomes a new graph with its own drift. The receiver no longer knows whether it is using the source's knowledge or its own aging shadow of that knowledge.

Centralization fails differently. One graph becomes the source of truth. Everyone queries it. Everyone depends on its uptime, policies, omissions, and governance. Discovery becomes easier, but authority becomes concentrated. The boundary problem has not been solved. It has been moved to the center.

Federated query matters because it gives a third path. The receiver can ask without swallowing. The source can answer without surrendering. Both sides can preserve their boundaries.

PART IV

Federation and Admission

Security, sovereignty, governance, evolution, and evidence

CHAPTER 15

Federated Queries Without a Central Brain

Federation is not the hoarding of facts, but the governance of apertures. A graph that copies the world invites its own collapse, while a graph that queries through apertures preserves the integrity of its own sovereign logic.

Federation is not copying the world.

It is asking the world through a governed aperture.

A local graph does not need to import every fact it might someday need. Copying the world creates storage bloat, update pain, provenance confusion, licensing problems, stale snapshots, and trust collapse. Centralizing the world creates a different failure: one registry, one gatekeeper, one target, one authority over discovery.

The Third Protocol needs a third pattern.

Graphs should be able to find, query, and enrich one another without surrendering sovereignty.

The Manifest Instead of the Master List

Federation begins with self-description. A graph can announce what it is, what domains it covers, which profiles it supports, which Core Lexicon release it uses, which trust anchors it recognizes, which query endpoints it exposes, which Knowledge Packs it publishes, which access policies apply, and which limits constrain requests.

No central registry is required for the concept. Discovery may happen through crawlable endpoints, known partners, directories, domain registries, or private federation agreements. The architecture does not require one master list of graphs.

The manifest is not a permission slip. It is a boundary declaration.

The receiver decides whether to query.

A central registry looks attractive because discovery is hard. But central registries become gatekeepers. They decide who is listed, who is excluded, which sources are authoritative, and what counts as quality. Some domains may choose registries for practical reasons. The protocol should not require one as the root of federation.

The Web did not begin by waiting for one catalog of pages. The distributed graph should not require one catalog of graphs.

Namespaces as Semantic Firebreaks

Namespaces serve as borders. A claim from an external graph should not overwrite a local distinction simply because the terms look similar. A local graph may distinguish two John Smiths that an external graph collapses. A medical system may treat a drug synonym as dangerous unless an authoritative pack supports it. A legal graph may reject a public summary because only statutory text is admissible under its profile.

External claims never override local distinctions by default.

This is not paranoia. It is graph hygiene. One bad merge can corrupt thousands of downstream results. One external `SAME_AS` assertion can collapse entities that should remain distinct. One casual mapping can turn an enrichment query into a contamination event.

The protocol can carry external identity claims. It should not force automatic identity merging.

A receiver can store an external link as a claim. It can mark it as soft. It can require an Evidence Gate. It can accept the link under one Trust Lens and reject it under another. It can expose the link to a human reviewer. It can refuse to use it in high-risk profiles.

The local graph remains sovereign.

Query Apertures

Federated queries work by aperture. The local graph asks for a bounded enrichment: background facts about a Canonical ID, a dependency chain for a non-core term, current validity for a public entity, supporting evidence for a claim, or domain context from a Knowledge Pack. The query has profile, scope, trust, and disclosure rules. The answer returns as HFF: grounded, sourced, versioned, and subject to admission.

This is not the same as copying the external graph.

A project graph can ask a public knowledge graph for facts about Saturn without importing every astronomy article. A legal sidecar can ask a Louisiana law graph for the current text and validity of a statute without merging the entire legal corpus into its local graph. A medical system can ask a pharmaceutical pack for interaction data while preserving local patient data behind selective disclosure.

Latency exists. Federation is not free. Some queries should be cached. Some should be preloaded as Knowledge Packs. Some should be refused in real-time paths. The point is not that every query should cross the world. The point is that the protocol makes the crossing explicit when it happens.

The answer may be admitted. It may be quarantined. It may be shown through a Trust Lens. It may be rejected because the source is too weak, the version too old, the dependency chain incomplete, or the profile unsupported.

Federation does not mean believing the world.

It means knowing how to ask it.

A Federated Query Walkthrough

A hospital sidecar receives a medication-order request. The order references a drug by a local code. The sidecar can map the local code to a Canonical ID, but before admitting the order it needs three external checks:

1. whether the drug class has a current interaction warning;
2. whether the patient's reported allergy maps to the same class;
3. whether the clinical rule pack being used is current.

The sidecar does not import every pharmaceutical graph in existence. It issues bounded federated queries:

QUERY: interaction facts for Canonical ID X
PROFILE: confidential direct
TRUST LENS: clinical_conservative
VALIDITY: current as of query time
DISCLOSURE: no patient identity disclosed

The pharmaceutical graph returns HFF bundles, not prose. Each bundle carries source, version, validity window, evidence pointer, and profile. The hospital sidecar admits the returned claims only if they satisfy local policy. If the external graph cannot provide current validity, the sidecar may quarantine the order or require human review.

This is federation as aperture.

The hospital did not copy the world. It asked a governed question through a bounded opening, received structured answers, and remained responsible for admission.

Query Shapes

Federated queries are not all the same. A protocol that treats every external query as a generic request will either over-share or under-answer.

The first shape is **identity enrichment**. A local graph has an entity and asks another graph for additional known identifiers, public facts, or related canonical IDs. This is the safest kind of query when scoped correctly because it enriches a known object rather than asking an external graph to define the local object from scratch.

The second shape is **dependency hydration**. A receiver receives a message containing a term it cannot hydrate locally. Instead of refusing immediately, it may query the sender's graph, a Knowledge Pack, or a trusted domain graph for the missing dependency material. This query is part of admission, not curiosity.

The third shape is **currentness verification**. A receiver has a claim but needs to know whether it remains current. This is common in legal, medical, regulatory, transportation, and public-status domains. The query asks more than “what is known?” but “what is current under this profile and as of this time?”

The fourth shape is **evidence retrieval**. A receiver may have a Synapse and need the source span, document hash, proof reference, or authority record

behind it. This query often intersects with selective disclosure. The receiver may be entitled to the proof but not to the entire protected payload.

The fifth shape is **policy enrichment**. A receiver asks whether a domain rule, safety policy, jurisdictional constraint, or profile-specific gate applies to the message. This shape is more dangerous because it can affect action. It should be constrained by profile and receipt.

The shape matters because the risk differs. A query for background facts about Saturn is not the same as a query that determines whether a robot may enter a restricted zone. The protocol should let the receiver say what kind of query is being made and what consequence the answer may have.

Federated Query Outcomes

A federated query can return more than an answer.

It can return an answer admitted under the requested profile. It can return an answer with weaker provenance than requested. It can return a dependency bundle. It can return a Knowledge Pack reference. It can return conflicting claims. It can return a refusal because the receiver lacks authority. It can return a selective-disclosure receipt saying proof exists but the requester is not entitled to the protected payload. It can return ERROR because the query is malformed, unsupported, or outside capability.

These outcomes matter because federated knowledge is not a search box. It is a machine-to-machine exchange. The response must carry enough structure for the local receiver to decide what to do next.

The most important response may be refusal. A graph that refuses correctly is safer than a graph that answers beyond its authority. A legal graph should refuse to answer a current-law query if it cannot establish jurisdiction or effective date. A medical graph should refuse to disclose protected content to an unauthorized requester. A private enterprise graph should refuse broad enumeration queries that look like scraping.

Federated query is therefore another act of receiver sovereignty. The graph being queried is also a receiver. It decides what leaves.

What Federation Must Not Do

Federation must not become automatic belief. The word *federated* can conceal the old mistake: because another graph knows something, the local graph now knows it too.

No.

An external answer is an incoming claim. It crosses a boundary. It must carry provenance, version, validity, trust, and profile. It may enrich the local graph. It may not overthrow local distinctions by surprise.

This is why federation belongs in The Third Protocol. The problem is not only query routing. It is boundary discipline for external knowledge.

CHAPTER 16

The Security Envelope

Cryptography secures the transmission, but the protocol adjudicates the intent. A valid signature confirms the speaker, yet it never validates the authority of the act.

The Signed Lie

A forged emergency message is not the hardest case. The hardest case is a genuine message, correctly signed, cryptographically sound, sent by a real and registered participant, carrying a false or unauthorized act.

Cryptography cannot solve this. It was not designed to. A hash proves the content did not change in transit. A signature proves which private key sealed the message. Neither bears on whether the content is true, whether the sender holds authority for the specific act, whether the message arrived inside its validity window, or whether local policy permits the action. These are separate questions. Treating them as one is the mistake that makes machine coordination brittle.

The security envelope in HFF addresses this by separating the questions rather than collapsing them. Each gate in the envelope answers exactly one of the following: did the content arrive intact; who signed it; is that signer trusted for this act; is the message current; and who is permitted to read it. Beyond those five cryptographic questions, two semantic questions remain: can the protocol terms hydrate to grounded objects, and does the action clear local policy and safety constraints. The envelope supplies the first five answers as preconditions. The semantic and policy questions remain with the receiver.

That ordering is the central claim. Cryptography secures the envelope. The protocol adjudicates the meaning.

Five Cryptographic Questions

Integrity

The `content_hash` and `payload_hash` fields record a digest of the canonical bytes of the message. If a single byte changes between sender and receiver, the digest mismatch is detected and the message is rejected. This gate runs first because nothing downstream is meaningful against a corrupted payload.

A hash does not identify who produced it. An attacker can hash a forged message and attach the hash. The integrity gate does not prevent forgery; it prevents undetected corruption. Integrity and authenticity answer different questions.

Authenticity

The `signature` field records the sender's cryptographic signature over the canonical message bytes under the declared canonicalization profile. The `key_id` identifies which key to verify against, and `trust_anchor_ref` names the trust chain or authority registry the receiver should use to evaluate that key.

A valid signature proves one thing: these canonical bytes were signed by the holder of the referenced private key. This is authenticity. It does not prove the sender is who they claim to be in a broader social sense, does not prove they are authorized to issue the act, and does not prove the content is accurate. Authenticity is necessary but not sufficient for admission.

Trust anchor validation is part of this gate, not a separate gate. Verifying a signature against a key that is not anchored in a trusted chain produces the same outcome as a failed signature: the message is rejected or quarantined.

Authorization

Authorization asks whether the authenticated sender is permitted to perform the specific act in the specific context. HFF carries this in the `authority_frame_id` field, which references an `AuthorityFrame` that the receiver evaluates against its own policy and trust model.

This gate is distinct from authenticity because authority is act-specific. An entity may be authorized to issue an `INFORM` but not a `COMMAND`. An entity may hold authority for emergency broadcasts in one jurisdiction but not another. A sender's certificate may declare their identity and role without encoding the granular authority required for the act at hand. Authority must be checked against a frame, not inferred from the fact that the signature validated.

Freshness

A valid message can be dangerous after its intended window. A command issued under one set of conditions may be inappropriate or harmful under conditions that have since changed. A replay attack recycles an old valid message to trigger an action the original sender did not intend to retrigger.

HFF addresses freshness through three independent mechanisms. The `expires_at` field declares the message invalid after a specific time. The `nonce` field provides a value that must be unique within the replay window. The receiver maintains a replay cache that records seen nonces; a nonce already in the cache triggers rejection even if the message is otherwise valid. These three mechanisms are complementary, not redundant. Expiry handles stale messages. The nonce handles replay within the window. The replay cache handles messages that arrive within expiry but are retransmitted.

A timestamp alone does not provide freshness protection. An attacker who replays a message with a valid timestamp within the expiry window would pass a timestamp-only check. The nonce is what closes that gap.

Confidentiality

Not all messages are intended for all recipients. Confidential profiles use an `encryption_envelope` to ensure that only intended recipients can decrypt the payload. Public signed broadcasts, by contrast, are intentionally readable by nearby participants; confidentiality there would defeat the purpose.

Encryption answers who may read the message. It is independent of who sent it, whether the sender is authorized, and whether the content is fresh. A message can be encrypted without being signed, signed without being encrypted, or both. HFF requires implementations to declare which combination applies via the security profile.

Semantic Malice

Each of the five cryptographic gates can pass cleanly while the message remains inadmissible. This is not a failure of cryptography. It is the boundary where cryptography ends and protocol adjudication begins.

Consider a compromised participant with a valid, unrevoked key. The integrity gate passes: the content was not altered in transit. The authenticity gate passes: the signature is valid. The freshness gate passes: the nonce is new and expiry has not elapsed. The authorization gate may pass if the attacker stays within a role the key legitimately holds. None of this makes the content true. A signed claim that an emergency vehicle is en route to a specific address can clear every cryptographic gate and still be false. The receiver that treats signature verification as the end of the admission procedure has conflated envelope security with content admissibility.

Two further semantic questions must survive after the cryptographic gates clear. First: can the protocol terms hydrate? If the message references lexicon identifiers that do not resolve to grounded objects in the receiver's context, the meaning cannot be unpacked and the message cannot be admitted. Second: does the action clear local policy and safety constraints? A receiver may have real-time sensor data, jurisdictional rules, mission state, or safety kernel constraints that make the requested action unsafe regardless of who signed the message.

These semantic questions are not optional steps for cautious implementations. They are structural requirements of any admission procedure that produces safe coordination.

The vocabulary is precise. A hash answers integrity. A signature answers authenticity. A trust anchor helps validate identity. An AuthorityFrame answers authorization. Nonces and expiry windows answer freshness. Encryption

answers confidentiality. None of these answers the question of whether the action should occur.

Freshness and Replay

Replay attacks deserve more than the brief treatment the freshness gate summary provides, because they are easier to mount than spoofed keys and more common in real deployments.

In a replay, the attacker does not break cryptography. They capture a valid signed message that was appropriate at the time it was sent, then retransmit it later, possibly with modified routing, in a context where the original sender no longer intends the act. A command to a drone to proceed along a specific route is valid at the moment of issue. The same message replayed an hour later, when the airspace has changed, the mission has advanced, or the authority has expired, can cause harm under a valid signature.

The replay rule is that receivers must reject or quarantine any message whose `message_id` has already been processed, whose `nonce` has been seen inside the `replay_window`, or whose `expires_at` has elapsed. All three checks are required because they address different attack surfaces. An attacker who strips the nonce from a captured message still fails the expiry check. An attacker who shortens the expiry to stay within window still fails the nonce check if the receiver maintains a nonce cache.

High-risk profiles extend this with a `revocation_check` requirement. Even a message with a valid key and a passing nonce can be dangerous if the key has been revoked since issuance. The revocation window must be honored.

Security as Admission Support

The security envelope is not the admission procedure. It is the first precondition for admission.

A message that fails a required cryptographic gate is rejected or quarantined according to profile before semantic evaluation begins. This ordering matters for efficiency and for security: there is no reason to spend receiver resources on hydration and policy checks against a message whose integrity is in question.

A message that passes all required cryptographic gates is a candidate for admission, not an admitted message. The protocol then asks whether the terms hydrate, whether the plausibility constraints hold, and whether local policy and safety rules permit the action. Only after those gates do the AFP act types take effect.

This structure places a specific responsibility on profile design. Profiles specify which cryptographic gates are mandatory, which are recommended, and which are inapplicable for a given class of message. A public emergency broadcast does not require encryption; requiring it would prevent nearby receivers from reading a message intended to coordinate their behavior. A high-risk command to a physical actuator requires AuthorityFrame validation, safety profile evaluation, and revocation checking; omitting any of these creates an attack surface proportional to the risk of the act.

Profile selection is therefore a security decision, not a categorization convenience. The profile determines which questions the security envelope must answer and which the receiver must evaluate independently.

The Signed Lie Revisited

Return to the signed lie. A compromised vehicle signs an emergency broadcast. The signature is valid. The key belongs to a vehicle that once had emergency authority. The packet is fresh. The hash verifies. If the receiver treats that chain as sufficient, the attacker has won without breaking the cryptography.

The receiver must ask different questions.

Is this key currently authorized for this act type?

Is the signer authorized in this geography?

Has the credential been revoked?

Is the claimed route plausible?

Is the claimed emergency consistent with other observations?

Is the receiver within scope?

Can local policy act on the request?

These are not cryptographic questions, though they may rely on cryptographic evidence. They are admission questions. The signed lie proves why security cannot be collapsed into one gate. The signature says who spoke. It does not say whether the speaker should be obeyed.

The same pattern appears in financial and medical domains. A doctor may sign a message outside the scope of a particular hospital privilege. A trading agent may sign an instruction above its limit. A supplier may sign a promise it cannot fulfill. The signature remains useful. It identifies the speaker. It does not remove the need for authority, validity, and policy.

Security in The Third Protocol is therefore compositional. Each layer answers only its own question. The receiver remains responsible for the rest.

Why Gate Order Matters

The admission pipeline has an order because failure has an order.

There is no reason to hydrate dependencies for a message whose hash fails. There is no reason to check local policy for a command whose signature cannot be verified. There is no reason to evaluate safety for a message outside recipient scope. There is no reason to run expensive plausibility checks on a packet whose freshness window has expired.

The order protects the receiver. It reduces attack surface. It prevents expensive work on invalid messages. It also creates consistent receipts. If two receivers process the same invalid message, they should fail at the same gate when their profiles and local state match. Otherwise the sender cannot understand what happened.

Gate order is also an audit feature. A receipt that says “refused” is less useful than a receipt that says “refused at authority gate after integrity and freshness passed.” The latter tells the sender what to fix. It tells an auditor which controls were active. It tells a governance system whether failures cluster around credentials, stale messages, unsupported profiles, or local safety.

The pipeline is not bureaucracy. It is how the receiver remains intelligible under load.

CHAPTER 17

Profiles and the Admission Pipeline

Admission is a judicial act, not a technical default. The gate sequence serves as the structural bedrock of refusal, forcing every incoming claim to prove its legitimacy against the receiver's local authority. Meaning is not a passive arrival; it is a boundary condition negotiated at the edge of the machine.

Why Profiles Exist

A public emergency broadcast, a confidential financial instruction, a swarm coordination update, a high-risk actuator command, and a regulated market transaction share a common need: a receiver must decide whether to admit the message before acting on it. The pipeline structure for that decision is the same across all five cases. The gates that are mandatory differ.

A single universal profile that applies the most restrictive possible checks to every message would prevent the coordinated yielding of vehicles in an emergency lane, because encrypting an emergency broadcast defeats its purpose. A single permissive profile would leave commands to physical systems without the authority and safety validation that makes those commands safe to execute. Neither failure mode is acceptable.

Profiles solve this by specializing the admission pipeline without modifying its grammar. A profile is not a different protocol. It is a declaration of which gates apply, which fields are required, and which failure modes are recoverable. The pipeline order does not change. The obligations do.

The Gate Sequence

The receiver admission pipeline has a fixed order. The order is not arbitrary; each gate depends on earlier gates having passed, and passing them out of order either wastes resources or enables attacks.

Parse and version check. The receiver first verifies that the message is well-formed: required envelope fields are present, the HFF version is supported, and the encoding profile is declared and recognized. A message that fails here cannot proceed; there is nothing reliable to check. The failure mode is a structured `ERROR` indicating which field or version constraint failed.

Hash verification. The `content_hash` and `payload_hash` are verified against the received bytes under the declared canonicalization profile. If either mismatches, the content cannot be trusted and the message is rejected. This gate runs before signature verification because computing and comparing hashes is less expensive than verifying a signature, and a corrupted message should be rejected without incurring signature verification cost.

Lexical hydration. Before any semantic evaluation, the receiver attempts to hydrate all non-core lexicon terms referenced in the payload. If a required term cannot be resolved, the message carries meaning the receiver cannot unpack. Depending on profile, the receiver may request a dependency, quarantine the message, or return an `ERROR`. Messages that hydrate partially under a profile permitting `VALID_ACTS_ONLY` bundle atomicity may proceed with admitted acts only.

Signature verification. The receiver verifies the sender's signature against the `key_id` and `trust_anchor_ref`. A valid signature proves the canonical bytes were signed by the key holder. Trust anchor validation, including key revocation status under profiles that require `revocation_check`, runs at this gate.

Freshness and replay. The receiver checks `expires_at` against its current time, the nonce against its replay cache within the `replay_window`, and the `message_id` against processed messages. All three checks are independent. A message failing any of them is rejected or quarantined.

Profile conformance. The receiver confirms it supports the declared profile and that all profile-required fields are present and valid. A receiver that does not support a declared profile must return a structured `ERROR` indicating the

unsupported profile; it must not silently degrade to a different profile that omits required gates.

Authorization. For acts requiring authority (COMMAND, CANCEL, high-risk REQUEST, and any act that modifies state in a consequential way), the receiver validates the `authority_frame_id` against its trust model and policy. This gate is what separates a message from a permitted act. The sender may be authentic; the receiver's policy determines whether the specific act is authorized.

Plausibility. The receiver evaluates whether the message content is plausible given the receiver's current world state and any available physics anchor or sensor data. A message claiming an emergency vehicle is at coordinates inconsistent with any known network topology, or claiming an action that contradicts recent confirmed state, fails plausibility even if it clears all prior gates. Plausibility is not a truth check; it is a sanity bound that catches implausible or spoofed content that the cryptographic gates cannot detect.

Local policy and safety. The receiver applies its local policy and, for physical systems, its safety kernel constraints. Local policy may include jurisdictional rules, mission state requirements, rate limits on certain act types, or operational constraints the sender cannot know. Safety kernel constraints, for systems where software errors can cause physical harm, are typically frozen and auditable separately from the local policy layer.

Outcome and record. After the pipeline runs, the receiver produces a structured outcome: ADMIT, REFUSE, ERROR, QUARANTINE, or a clarification request. The outcome is recorded with a receipt that captures which gates were evaluated, which passed, and which failed, along with the receiver's reasoning. This receipt is the accountability artifact.

Profile Matrix

Five profiles cover the majority of deployment cases. They are not exhaustive; extension profiles for specific domains may build on this foundation.

PUBLIC_SIGNED_BROADCAST is used when a message is intended to be read by all eligible nearby receivers. Integrity verification, signature verification, trust anchor validation, expiry, and replay prevention are required.

Encryption is not used, because the message must be readable by participants who receive it. This profile suits emergency vehicle advisories, public hazard warnings, road condition alerts, and capability announcements. The plausibility gate and local policy gate still apply at the receiver.

`CONFIDENTIAL_DIRECT` is used when the message is intended for a specific, named recipient. Encryption is required. Signature verification, integrity verification, expiry, nonce, and trust anchor validation are required. This profile suits financial instructions, medical communications, legal acts, and private command traffic.

`CONFIDENTIAL_GROUP` extends confidential delivery to a group of recipients sharing a group key or key distribution mechanism. The sender encodes the payload once; each authorized group member can decrypt. Group membership and key rotation are handled by the deployment's trust infrastructure, not by HFF itself.

`HIGH_RISK_COMMAND` applies when the act can move vehicles, drones, robots, medical devices, infrastructure, or other systems where errors are irreversible. AuthorityFrame validation, risk class declaration, safety profile, receipt policy, and revocation check are required in addition to the standard cryptographic gates. Encryption is recommended. Receivers under this profile must evaluate authorization, local policy, safety constraints, and world state before acting; the profile makes this explicit rather than leaving it to implementation judgment.

`REGULATED_TRANSACTION` applies to financial orders, settlement instructions, regulated market messages, and other economically material acts. It requires an idempotency key or replay-safe transaction reference, authorization, risk class, audit policy, and receipt policy. The profile boundary is explicit: a validly signed transaction instruction proves the instruction and its signer; it is not itself the executed transaction. Execution requires account controls, market rules, compliance policy, and venue-specific acceptance.

Profile	Encryption	AuthorityFrame	Idempotency Key	Revocation Check
PUBLIC_SIGNED_BROADCAST	Not used	Not required	Not required	Recommended
CONFIDENTIAL_DIRECT	Required	Not required	Not required	Required
CONFIDENTIAL_GROUP	Required	Not required	Not required	Required
HIGH_RISK_COMMAND	Recommended	Required	Not required	Required
REGULATED_TRANSACTION	Required	Required	Required	Required

Structured Outcomes

The pipeline produces structured outcomes. Some outcomes are AFP acts; others are receiver dispositions or protocol handling states. All of them carry meaning about the receiver’s decision.

ACK confirms that the message was received and processed. It does not imply agreement, authorization, or execution. ACK is appropriate when a sender requires proof of delivery or processing.

ACCEPT confirms that the receiver has accepted a proposal, request, or offer. It implies the receiver’s intent to act, not that the act has been completed.

CONFIRM records that an act was completed or a state was verified. It is distinct from ACK because it attests to completion, more than receipt.

REFUSE states that the receiver understood the act and will not perform it. A REFUSE is a legitimate outcome that preserves protocol state. The sender knows the act was evaluated and rejected rather than lost. Reason codes are permitted by profiles that support them.

ERROR reports a protocol failure: a malformed message, an unsupported profile, an invalid state transition, a missing required lexicon entry, a hash mismatch, or a failed signature. An ERROR does not necessarily mean the

sender's intent was understood; it means the receiver could not process the message.

QUARANTINE is a receiver disposition rather than a core AFP illocution. It holds the message for further evaluation without acting on it. This disposition is used when a dependency is missing and may become available, when the receiver's local policy requires human review before certain act classes, or when plausibility failed but the failure may be transient.

The distinction between REFUSE and ERROR is load-bearing. A receiver that returns ERROR for every rejected message is hiding information: the sender cannot tell whether the message was malformed, the sender lacks authority, the action is locally unsafe, or the receiver is deliberately refusing. REFUSE is an honest acknowledgment. ERROR is a protocol failure report.

Conformance Implication

The stable gate sequence is what makes the protocol testable. An implementation that produces consistent outcomes across equivalent inputs under a declared profile is conforming. An implementation that reorders gates, skips gates under load, or returns different outcomes for the same input depending on timing is non-conforming regardless of whether the final outcomes are sometimes correct.

Profiles do not give implementations latitude to reorder or skip the gate sequence. They specify which gates are required and which failure modes are recoverable. Within those constraints, implementations have latitude in how the gates are executed. The protocol does not specify cryptographic libraries, hardware security modules, or trust registry formats; it specifies what the gates must answer and in what order.

The pipeline is a receiver-side machine. The same sender message routed to two different receivers may produce ADMIT at one and REFUSE at the other, because local policy and safety constraints differ. This is not a protocol inconsistency. It is the design: coordination is distributed, authority is delegated, and decisions remain local.

Profile Selection as a Risk Decision

Profile selection is not a formatting preference. It is a risk decision.

A sender may prefer the lightest possible profile because light profiles are easier to satisfy. A receiver may require a stronger profile because the act touches money, motion, privacy, or regulated state. The receiver's requirement wins. The receiver is the one admitting meaning into local consequence.

Consider the same REQUEST under three profiles.

Under a public signed broadcast profile, the request may ask nearby receivers to consider a public condition. Confidentiality is low. Signature, freshness, scope, and plausibility matter.

Under a confidential direct profile, the request may contain private business or medical content. Confidentiality and recipient identity become mandatory gates. A public readable payload would fail even if every semantic field were correct.

Under a high-risk command profile, the request becomes operationally dangerous. AuthorityFrame, risk class, revocation, safety profile, and receipt policy become mandatory. A message that was admissible as advice becomes inadmissible as command.

The payload may look similar across all three cases. The boundary differs. The profile names the boundary.

This is why profile negotiation belongs before execution, not after failure. A sender that chooses the wrong profile has more than chosen the wrong envelope. It has made the wrong claim about the risk of the act.

Sovereignty Is Not Isolation

Receiver sovereignty can sound like every machine becoming an island. That is not the claim. A sovereign receiver is not a receiver that refuses by default. It is a receiver that admits according to declared policy rather than inherited pressure.

The distinction matters. A system that refuses everything is safe only in the narrow sense that a disconnected machine is safe. It does not coordinate. It does not participate. It cannot form a swarm, accept work, answer queries, share observations, or execute useful tasks. Sovereignty is not withdrawal from the network. It is participation without surrender.

A sovereign receiver can be highly cooperative. It can publish a generous capability manifest. It can accept many profiles. It can preload Knowledge Packs. It can provide detailed refusal reasons. It can respond quickly. It can participate in complex commitment graphs. What it cannot do is allow the sender to decide what local consequence must follow from an external message.

This is the balance the architecture must preserve. Too little sovereignty produces botnets. Too much isolation produces silence. The Third Protocol lives in the middle: cooperative receivers with explicit gates.

Sovereignty Under Delegation

Receiver sovereignty does not mean every receiver invents its own morality at runtime. Receivers operate under delegated policy. A vehicle follows traffic law, manufacturer constraints, fleet policy, and its certified safety envelope. A hospital system follows clinical policy, privacy law, user permissions, and institutional governance. A financial sidecar follows account authority, regulatory rules, and risk controls.

The receiver is sovereign in the sense that these policies are local at the point of admission. The sender cannot bypass them by sending a well-formed message. The receiver is not morally independent. It is procedurally responsible for applying the policies that govern it.

This matters because some readers will hear “sovereign” and imagine arbitrary refusal. The better image is a border officer, not a king. The border officer does not invent the law. The officer applies it at the crossing. Different crossings may apply different rules because they belong to different jurisdictions.

The receiver is the border where external meaning meets local jurisdiction.

CHAPTER 18

The Receiver Is Sovereign

Authentication is a credential, not a mandate. A machine that cannot refuse is not a governed agent, but a hollow channel for external error.

The Obedient Machine Is the Unsafe Machine

A machine that executes signed instructions without independent evaluation is not a coordinated participant. It is a channel. Whoever holds a valid key, or whoever can produce an apparently valid message, controls the machine. The coordination layer becomes an obedience layer, and the risk posture of every machine in the network degrades to the risk posture of the weakest key.

This framing is not speculative. It describes a real failure mode that appears whenever machine coordination is reduced to authenticated command-response. A delivery vehicle that yields to any signed emergency claim, without evaluating scope, plausibility, or local safety, is yielding to whoever can produce a signed emergency claim. A drone that executes any signed route command is flying wherever a key holder directs. An industrial system that admits any AuthorityFrame-carrying instruction without checking current state and safety constraints is not under governance; it is under remote control.

The protocol's answer to this failure mode is sovereignty. A receiver is sovereign when its local policy, local state, and local safety rules participate in the admission decision rather than being overridden by the message content. Sovereignty does not mean the receiver can ignore valid instructions arbitrarily. It means the receiver has formal standing to evaluate an instruction against conditions the sender cannot know, and to produce a structured refusal when those conditions disqualify the act.

Local Policy and Local State

The sender's message arrives with claims about the world, an act type, a declared authority, and a security envelope. The receiver holds something the sender does not have: current local state.

Local state includes sensor readings that post-date the message. A vehicle that has detected an obstruction in the requested maneuver path holds information the ambulance driver cannot hold. Local state includes mission state: a drone mid-sequence through an autonomous task has commitments that a remote command may conflict with. Local state includes jurisdictional and operational constraints: a receiver operating in a geofence that prohibits certain maneuver types under specific conditions cannot execute an act that violates that constraint, regardless of the sender's AuthorityFrame.

Local policy is the formal encoding of these constraints. Some local policy rules are static: a vehicle's safety kernel holds frozen definitions for which sensor readings trigger mandatory braking. Some are dynamic: operational policy may restrict certain act classes during specific time windows or traffic conditions. Some are jurisdictional: regulatory requirements may mandate human-in-the-loop review above certain risk thresholds.

The receiver's evaluation of local policy and local state is not an override of the sender's authority. It is the last gate in the admission pipeline, the gate the sender cannot evaluate on the receiver's behalf. A sender who holds correct authorization for an act, and whose message passes every cryptographic and semantic gate, may still encounter a receiver in a state where executing the act would cause harm. The protocol provides structured outcomes precisely so that these situations resolve gracefully rather than producing silent failures or unsafe actions.

Refusal as Protocol Act

A receiver that refuses an act is not breaking coordination. It is contributing a local decision to a distributed decision process. Without structured refusal, coordination degrades to one of two failure modes: silent non-compliance, where the receiver ignores the message and the sender does not know why nothing happened, or unsafe compliance, where the receiver executes an act it should have blocked.

Silent non-compliance is operationally catastrophic for safety-critical coordination. If an emergency vehicle broadcasts a lane-clearing request and nearby vehicles simply fail to respond without explanation, the operator has no visibility into whether the message was received, whether it was rejected for a known reason, or whether some systemic failure is preventing admission. Structured refusal gives the operator that visibility.

The protocol defines REFUSE as the outcome when a receiver understands an act and will not perform it. REFUSE differs from ERROR in that it acknowledges the receiver's comprehension of the request and the deliberate decision to decline. REFUSE may carry a reason code where the profile permits, but the reason is informational; the sender cannot override a REFUSE by adjusting the message. A refusal grounded in local safety is not a negotiating position.

ACK followed by no further action is not equivalent to REFUSE. ACK confirms receipt. The absence of subsequent ACCEPT or CONFIRM leaves the conversation state ambiguous. A conforming implementation that has decided not to perform a requested act should issue REFUSE, not silence.

The distinction matters for audit. A receipt log that records REFUSE with a reason code gives an investigator the information needed to determine whether the refusal was correct, whether the policy needs adjustment, or whether the sender's request was inappropriate. A log that records ACK followed by nothing provides no such information.

Sovereignty Prevents Obedience Channels

If receivers cannot refuse, then every authenticated sender becomes a command channel. This is the botnet pattern. A botnet does not require that each node be individually compromised; it requires that each node reliably execute instructions from whoever holds a valid credential in the network's trust model. The distinction between a legitimate swarm and a botnet is the existence of local evaluation.

The protocol closes this channel through the combination of AuthorityFrame validation, local policy evaluation, and structured refusal. An instruction that passes cryptographic gates and carries a valid AuthorityFrame is admitted only if local policy allows and local state permits. A receiver that has evaluated these conditions and found them wanting issues REFUSE. No

authenticated sender can compel a conforming receiver to bypass local policy or safety evaluation.

This creates a governance property: federated swarms remain accountable. A vehicle operator cannot claim their vehicle was compelled to execute an unsafe act by a remote signed instruction, because the receiver pipeline required independent local evaluation. A drone fleet operator cannot attribute a harmful action to an attacker's command without explaining why the receiver's safety evaluation did not block it. Accountability flows to the decision, and the decision record is in the receipt log.

The corollary is visible in how the protocol treats emergency authority. Emergency messages do not bypass the receiver's admission pipeline. They enter the pipeline under the `PUBLIC_SIGNED_BROADCAST` profile and are subject to signature verification, freshness checking, plausibility evaluation, and local safety rules. An emergency request to yield a lane cannot compel a vehicle to execute a maneuver that the vehicle's safety kernel has determined is unsafe given current sensor readings. The emergency authority is real: it affects how the receiver weights the request. It does not suspend the receiver's evaluation.

Sovereignty Without Fragmentation

A swarm in which every receiver can refuse creates the risk of coordination failure through mass refusal. If receivers refuse broadly for reasons that are not grounded in local conditions, coordination degrades to noise.

The protocol addresses this by requiring that refusals be structured and that they carry the receiver's reasoning where the profile permits. A receiver that refuses for reasons not grounded in local policy or local state is exposing its implementation to audit challenge. The protocol does not prevent malicious or erroneous refusals, because no protocol can prevent implementations from behaving incorrectly; but it makes correct and incorrect behavior distinguishable.

Coordination in a sovereign receiver network does not require identical decisions. It requires legible decisions. A fleet of vehicles responding to an emergency broadcast does not need every vehicle to yield simultaneously; it needs each vehicle to evaluate the request against its local state and produce a structured outcome that the broadcast originator can interpret. Vehicles that

yield report ACCEPT. Vehicles that cannot yield safely report REFUSE with a reason. Vehicles that are out of scope ignore the broadcast without response, because the scope gate filtered them before evaluation began.

The swarm that produces legible, locally-grounded decisions is not fragmented. It is accountable. Fragmentation is what happens when participants act without local evaluation; the protocol was consistent, but the consequences were uncoordinated. A swarm of sovereign receivers produces coordination that is slower than obedience but safer than it. That tradeoff is not incidental. It is the point.

CHAPTER 19

Governance of the Stable Layer

Governance is the architectural refusal to let local utility destroy global stability. A protocol that cannot say no to a good idea will eventually say yes to its own obsolescence.

A protocol does not fail only when its packets cannot be parsed. It can fail after parsing succeeds.

It fails when every implementer adds a private core act. It fails when every vendor changes the envelope just enough to require a private adapter. It fails when one domain adds a role, another changes the meaning of acknowledgement, a third treats refusal as error, and a fourth calls a local extension part of the core.

The wire still moves. The protocol is dead.

Governance architecture is part of protocol correctness.

This statement can sound administrative. It is not. It is structural. A protocol becomes infrastructure only when many independent implementers can rely on the same stable surface for years. If that surface fragments under extension, the protocol has not become infrastructure. It has become dialect soup.

The Third Protocol needs a protected core and a living periphery. Without both, the architecture either ossifies or fragments.

Tier	Change speed	Examples	Purpose
Tier 1: stable core	Slow	HFF envelope, AFP act registry, role grammar, admission outcomes	Preserve interoperability
Tier 2: managed shared layer	Medium	Core Lexicon releases, domain profiles, conformance tests	Govern shared resources
Tier 3: open extension	Fast	Private namespaces, local lexicons, experimental profiles	Allow innovation without core mutation

The Dialect Failure

Dialect failure begins innocently. One implementer adds a new act because the existing act set feels too general. Another adds a new role because a local domain distinction seems important. A third adds an envelope field and treats it as mandatory. A fourth changes the interpretation of ACK so that receipt implies weak approval. A fifth treats a domain profile as if every receiver must support it.

None of these changes may look destructive in isolation. Each may solve a local problem. Together they destroy the shared surface.

The receiver can no longer know what a message means by reading the protocol. It must know which implementation sent it. That is the return of per-pairing engineering work. The protocol becomes a family resemblance rather than infrastructure.

The stable layer must be protected because strangers depend on it.

The Three Tiers

Tier 1 is the stable layer. It contains the wire grammar, canonical envelope rules, AFP act registry, SGF role grammar, admission outcomes, profile conformance, and compatibility rules. Tier 1 changes slowly because every participant depends on it. A Tier 1 change is not a feature request. It is a

structural event. It requires review, test vectors, migration policy, deprecation policy, and a long compatibility horizon.

Tier 2 is the managed shared layer. It contains Core Lexicon releases, domain profiles, security profiles, validation suites, conformance tests, and standard dependency-bundle shapes. Tier 2 changes more often than Tier 1, but not casually. These resources are shared enough that bad changes can damage many systems. They require review, versioning, and public traceability.

Tier 3 is the open extension layer. It contains private namespaces, local lexicons, experimental profiles, implementation-specific metadata, internal policy mappings, and domain payloads that do not claim core status. Tier 3 changes quickly. It is where builders learn. It is where domains experiment. It is where a new pattern proves itself before asking to become managed.

Governance protects interoperability, not permission.

That line prevents a dangerous misunderstanding. The protocol should not become a central authority over every implementation. It should not decide which database an implementer uses, which inference engine proposes Synapses, which programming language runs the sidecar, which UI renders receipts, or which internal policy engine evaluates local constraints. Those are implementation choices.

Governance protects the shared surface.

The shared surface is what a stranger machine depends on when it receives a message. It must know what ACK means. It must know that ACCEPT is not CONFIRM. It must know that a non-core term requires dependency material or a resolvable reference. It must know that a receiver may refuse a valid message. It must know how version negotiation is declared. It must know which profile requirements are mandatory.

Those meanings cannot be private.

Cost Asymmetry

The core cannot change casually. HFF envelope semantics, canonicalization rules, AFP act names, admission outcomes, role grammar, profile conformance, and version-negotiation rules form the stable layer. A change here can break every participant. The cost of change must therefore be high.

That cost is not bureaucracy. It is the price of shared reality.

The periphery must change easily. Local namespaces, private lexicons, domain-specific payloads, internal policies, implementation strategies, and experimental profiles need room to grow. If every extension requires core approval, innovation dies. If every extension mutates the core, interoperability dies.

The architecture must make the right changes expensive and the safe changes cheap.

This is the principle of cost asymmetry.

Consider a vendor that wants to add a new core AFP act called ESCALATE. The motivation may be legitimate. Many systems need escalation. But if ESCALATE can be modeled as a REQUEST with profile-specific urgency, recipient, policy, and routing metadata, it should not become a core act. The act registry should not grow every time a domain invents a useful verb. The registry exists to preserve coordination at the stable layer.

Now consider a medical-device profile that requires additional safety checks before admitting a COMMAND. That profile may be valuable. It can add domain-specific mandatory fields, receipt requirements, audit constraints, and safety gates without changing HFF grammar or the AFP act registry. That is the correct path: extend at the managed or peripheral layer, not by mutating the core.

Governance does not forbid invention. It tells invention where to live.

The stable layer is small because it must be common. The managed layer is larger because it must be shared. The extension layer is open because the world is larger than any committee.

The Extension Temptation

Every serious implementer will eventually want one more core feature.

One organization will want a new act type. Another will want a new role. Another will want to make a domain profile mandatory. Another will want to change a receipt field because its local audit system prefers a different shape. Each request will have a reason. Many will be intelligent. Some will be urgent. The danger is that the protocol will die by accepting good local reasons into the global core.

This is how stable layers decay. Not through stupidity, but through sympathy.

Governance must therefore be able to say no to reasonable requests. It must be able to tell a domain: that belongs in a profile. It must be able to tell a vendor: that belongs in your namespace. It must be able to tell a regulator: that belongs in a managed pack, not in the HFF envelope. It must be able to tell an implementer: this can be done through existing acts.

The purpose of governance is not to slow the world. It is to prevent every local success from becoming global breakage.

This is why “governance protects interoperability, not permission” is load-bearing. Builders remain free to build. They are not free to redefine the common surface and still claim the benefits of interoperability.

Conformance Before Consensus

Governance should not begin by asking every participant to agree on every future use. That is how standards efforts stall. It should begin with conformance: what must be true for a message, sidecar, profile, Knowledge Pack, or receiver to claim that it participates in The Third Protocol?

Conformance is narrower than consensus. Consensus asks what everyone believes. Conformance asks what every participant must preserve. A domain may disagree about policy and still conform. A receiver may reject a message and still conform. A sidecar may support only a small subset of acts and still conform if it declares that subset honestly. A Knowledge Pack may be private and still conform if its manifest, dependencies, and versioning behave correctly.

This distinction protects adoption. The protocol does not need everyone to agree on a single worldview. It needs every participant to preserve the shared boundary mechanics: HFF envelope behavior, AFP act semantics, dependency hydration, capability declaration, profile gates, receipts, and refusal outcomes.

Conformance is the practical face of governance. It says what can be tested.

The Minimum Test Suite

A governance layer without tests becomes prose. The stable layer needs examples that every implementation can run:

- parse a canonical HFF message;
- reject an undeclared encoding;
- hydrate a non-core dependency bundle;
- distinguish ACK, ACCEPT, and CONFIRM;
- refuse an unsupported act;
- reject a stale signed message;
- admit a public signed broadcast under profile;
- refuse a high-risk command without authority;
- record a receipt with the correct failed gate.

These tests do not prove an implementation is wise. They prove it preserves the shared surface. That is the part governance must protect.

Extension Proposal Lifecycle

Governance becomes concrete when an extension asks to move upward.

A domain begins with a private extension. A logistics company adds a private field to represent refrigerated cargo constraints. The field lives in its namespace. It works locally. Partners begin using it. Eventually several implementers discover that the same pattern appears across cold-chain logistics, pharmaceuticals, food transport, and biological samples.

At that point, the extension may be proposed for managed profile status. The proposal should include examples, failure cases, conformance tests, migration behavior, and a proof that the extension does not require changing Tier 1 grammar. The governance question is not whether the field is useful. The question is where it belongs.

If the pattern remains domain-specific, it becomes part of a managed cold-chain profile. If it proves more general, it may become a recommended profile field across multiple domains. Only in the rarest case should it change the stable core.

This lifecycle lets useful inventions climb without letting every useful invention become core.

What Governance Does Not Do

Governance does not certify that an implementation is wise. It does not guarantee that a local policy is moral. It does not decide whether a business should automate a workflow. It does not control every sidecar, every database, every private lexicon, or every domain-specific application. Those belong to implementers, regulators, customers, operators, and domain authorities.

Governance protects the shared contract.

It defines how a conforming message is shaped. It defines how acts are named. It defines how versions are declared. It defines what a receiver may assume when a message claims conformance. It defines how extensions stay extensions until they earn a higher tier.

A protocol without governance becomes a vocabulary of good intentions. It will work in demos. It will work inside one vendor. It may work between two friendly partners. It will not become infrastructure.

Infrastructure requires strangers.

Strangers require stable expectations.

Stable expectations require governance.

CHAPTER 20

How the Protocol Evolves

Versioning pain is the irreducible friction of distributed time. A protocol that cannot name its own obsolescence is a ledger that eventually records only errors, turning the history of machine coordination into a silent, unrecoverable debt. Compatibility is not the absence of change, but the discipline of naming the gap between what was and what is.

The fantasy of a perfect protocol is that it never changes.

The history of useful protocols says otherwise. A protocol that lasts will meet new domains, new threats, new encodings, new hardware, new privacy requirements, new safety regimes, new legal constraints, and new kinds of machines. Freezing everything forever protects compatibility by sacrificing relevance. Changing everything casually protects relevance by destroying compatibility.

The question is not whether versioning pain can be eliminated.

Versioning pain is irreducible. It is localized or not.

The Shape of Versioning Pain

Versioning pain appears whenever two participants make different assumptions about the layer they share. One participant thinks CONFIRM means completion. Another treats it as receipt. One participant supports a newer security profile. Another supports only an older one. One participant uses a Core Lexicon release containing a term the other cannot hydrate. One participant sends a dependency bundle in a shape the other does not recognize.

This pain cannot be wished away. The world changes at different speeds. Vehicles, robots, hospitals, financial systems, and software agents do not upgrade together. Some systems are replaced yearly. Some run for decades. Some are certified under regimes that make updates slow. Some are experimental and change weekly.

A protocol that cannot survive uneven time is not infrastructure.

A monolithic protocol lets pain cascade. One version number governs too much. A participant either speaks version 3 or it does not. Inside that single number may be a wire change, a new act, a changed security profile, a new dependency-bundle rule, a Core Lexicon snapshot, and a domain profile. Two participants that share most of what they need may fail because the version boundary is too blunt.

The Third Protocol needs layered versioning.

HFF has a version. AFP has a version. The capability schema has a version. The Core Lexicon has a version or release profile. Security profiles have versions. Domain profiles have versions. Encoding profiles have versions. Conformance suites have versions. These layers are related, but they are not the same object.

Layered versioning does not make compatibility painless. It makes compatibility inspectable.

Capability Exchange as Version Negotiation

Capability exchange is the negotiation surface. A participant declares what it supports. Another participant declares what it supports. The two systems compute the highest shared floor for each relevant layer. They may share HFF VI.0, AFP VI.I, Core Lexicon release 2028-04, a regulated-transaction profile v2, and a domain extension v3. Or they may fail to share one required layer.

This is not a ceremonial handshake. It is the mechanism by which versioning pain becomes local.

A sender that supports a newer AFP act set cannot assume the receiver supports it. A receiver that lacks a new profile cannot pretend otherwise. A message that depends on a newer Core term may still be processable if the sender includes dependency material. A message that depends on a new act with no older equivalent may not be processable at all.

The point is not that every mismatch can be repaired.

The point is that the mismatch is named.

Named Failure Outcomes

Failure is not a catastrophe if the protocol can name it.

Version negotiation has several outcomes.

The first is proceed. Both participants support the required layers at compatible versions. The exchange continues.

The second is downgrade. A newer participant can express the act under an older supported layer without losing required semantics. Downgrade must be allowed by policy. Downgrade must be recorded. Silent downgrade is an attack surface.

The third is request dependency. The receiver supports the protocol but lacks a term, lexicon version, domain profile, or dependency bundle required to hydrate the message. The correct result is not improvisation. It is a structured request for the missing dependency or a refusal if the profile does not allow dependency fetch.

The fourth is bridge or sidecar translation. A participant may be able to preserve required semantics through an explicit translation layer. That translation must be visible in the receipt. If meaning was bridged, the record should say so. A bridge that hides itself recreates the old integration tax.

The fifth is quarantine. The receiver may store the message without admitting it to action. Quarantine is useful when the message is potentially valuable but cannot yet satisfy all gates.

The sixth is refuse. If required meaning cannot be preserved, if downgrade would change the act, if dependencies cannot be hydrated, if the security profile is unsupported, or if local policy rejects the exchange, refusal is the correct outcome.

Named failure is safer than silent interpretation.

Deprecation Is a Protocol Event

Versioning is not only about adding new features. It is also about retiring old ones.

A profile may be deprecated because its security assumptions no longer hold. An encoding may be deprecated because canonicalization bugs were discovered. A Core Lexicon release may be superseded because a class of terms was reorganized. A domain pack may be withdrawn because its source authority changed. A trust anchor may expire or be revoked.

If deprecation is handled informally, the network becomes brittle. Some participants continue to send old messages. Others reject them. Some bridge them silently. Others downgrade without recording it. A receiver may not know whether it is seeing old behavior because the sender is outdated, compromised, or intentionally refusing to migrate.

The protocol should make deprecation visible. A capability manifest can declare supported but deprecated versions. A receiver can return a structured warning. A profile can set a sunset date. A receipt can record that an exchange used a deprecated layer. A governance registry can publish a deprecation schedule.

Deprecation is not housekeeping. It is how a protocol ages without lying about its age.

Backward Compatibility Has a Boundary

Backward compatibility is valuable until it preserves the wrong thing.

A receiver should support old versions when the old version can express the necessary meaning safely. But if an old version lacks a required act distinction, cannot represent validity windows, omits selective disclosure fields, or uses a security profile no longer acceptable for the domain, compatibility becomes danger.

The receiver must be allowed to say no. Not every old message deserves a bridge. Not every bridge preserves meaning. Sometimes the only honest answer is REFUSE: required semantics cannot be represented under the offered version.

This is not a failure of evolution. It is evolution doing its job. The protocol is healthier when an old message is refused for a named reason than when a receiver pretends the missing semantics do not matter.

The 2026 Agent and the 2032 Agent

Consider a 2026 warehouse sidecar meeting a 2032 logistics agent. The 2032 agent supports AFP VI.3. The warehouse sidecar supports AFP VI.0. The logistics agent sends a PROPOSE act using a profile feature introduced in VI.2. The receiver cannot pretend to understand. It can ask for a VI.0 expression. It can accept a downgraded form if no required semantics are lost. It can route through a bridge that records the translation. It can quarantine. It can refuse.

All of these are better than guessing.

Lexicon evolution follows the same discipline. A sender may use a Core term introduced in a later release. If the receiver lacks that release, the HFF bundle can include dependency material. The receiver may hydrate the term through the provided mini-lexicon entry and its IS_A tether. Or it may refuse because the dependency chain is too deep, the source is untrusted, or the profile forbids unknown terms in that act class.

The point is not to make every old system understand every new system. The point is to make misunderstanding explicit.

Every exchange should record the versions actually used. A receipt should be able to say which HFF version governed the envelope, which AFP act set governed the message, which Core release grounded the terms, which security profile applied, which domain profile constrained the act, and whether a downgrade, dependency request, bridge, quarantine, or refusal occurred.

That record is not clerical. It is the memory of interoperability.

Longevity is not the absence of versioning pain. It is the containment of it.

CHAPTER 21

Selective Disclosure

Meaning must be inspectable enough to admit, yet protected enough to deploy. The protocol succeeds only when the envelope routes the act while the payload remains the sovereign’s secret.

The protocol must let machines prove enough without revealing everything.

That requirement appears wherever the protocol becomes useful. A public road alert may need to be readable by nearby vehicles. A clinical referral may need to verify legitimacy without exposing protected details to every intermediary. A regulated transaction may need to verify authority and idempotency without revealing the full commercial position. A defense or infrastructure command may need to verify scope and authorization while protecting operational content.

Auditability and confidentiality are not enemies. They become enemies when the system stores everything as one undifferentiated blob.

Selective disclosure works because the protocol separates the surface that routes, the payload that speaks, the proof that binds, and the receipt that remembers.

Surface	What it carries	Why it exists
Visible envelope	profile, act type, sender reference, timestamp, proof commitments	route and begin admission
Protected payload	sensitive content, encrypted or access-controlled fields	preserve confidentiality

Continued on next page

(Continued)

Surface	What it carries	Why it exists
Proof commitment	hashes, signatures, proof references, receipt links	bind hidden content without revealing all of it
Policy-visible receipt	admission outcome, refusal reason, confirmation, audit reference	remember what happened

The Auditor and the Stranger

Selective disclosure is easiest to understand by distinguishing the auditor from the stranger.

The stranger may need to know enough to route and evaluate a message. The stranger may need profile, sender reference, act type, timestamp, proof commitments, and recipient scope. The stranger does not necessarily need the payload.

The auditor may need more, but not always immediately. An auditor may be entitled to see the protected payload under certain conditions, or may be entitled only to verify that a protected payload matched a commitment at the time of admission. A regulator may need a full record after a dispute. A counterparty may need only the receipt. A public observer may need only proof that a message existed and was handled under a profile.

One disclosure surface cannot serve all these roles.

This is why the protocol separates envelope, payload, proof commitment, and receipt. The same exchange can provide minimal routing information to one participant, proof of integrity to another, payload access to the intended receiver, and audit trace to an authorized reviewer.

Selective disclosure is not secrecy. It is structured visibility.

Redaction Is Not Selective Disclosure

Redaction hides text. Selective disclosure structures what may be known.

This difference matters because a redacted document often leaves the receiver guessing. A black box covers the name, the amount, the condition, or the authority. The reader sees absence but not the structure of absence. A machine cannot reliably reason over black bars.

Selective disclosure should preserve the shape of the claim even when content is protected. The receiver may know that an authority field exists and was verified, even if the identity is visible only to a regulator. It may know that a payload hash matches the signed commitment, even if the payload is encrypted. It may know that a transaction passed an idempotency gate, even if commercial details remain protected. It may know that a clinical referral exists, that required fields are present, and that the intended recipient can decrypt them.

The receiver learns enough to route, admit, refuse, or audit. It does not learn everything.

That is the discipline.

Disclosure Failures

Selective disclosure can fail in two directions.

The first failure is overexposure. Too much content becomes visible to parties that need only routing, proof, or receipt. This violates confidentiality and discourages adoption in exactly the domains that need the protocol most.

The second failure is underexposure. The receiver sees so little that it cannot evaluate the message. Everything is encrypted, but nothing is admissible. The message is private and useless.

The four-surface model exists to avoid both failures. The visible envelope routes. The protected payload speaks to authorized recipients. The proof commitment binds. The policy-visible receipt remembers.

Medical Referral Example

A medical referral shows why selective disclosure cannot be a vague privacy promise.

The visible envelope says that a clinical referral exists, identifies the sending institution, identifies the intended receiving institution, declares the profile, timestamp, and proof commitment, and states that required fields are present. The protected payload contains the patient details, clinical notes, diagnosis codes, medication history, and supporting documents. The proof commitment binds the protected payload to the envelope so the receiver can know the

content was not changed. The policy-visible receipt records that the receiving system admitted, refused, or quarantined the referral under a specific policy.

An intermediary may see enough to route and verify that a valid clinical object exists. It does not see the clinical content. The receiving institution sees the payload because it is the intended recipient. An auditor may later see either the full payload or only proof that the payload matched the commitment, depending on authorization.

This is not redaction. Redaction hides content after the fact. Selective disclosure designs the message so each participant sees the surface it is entitled to see.

The Four Surfaces

The first surface is the visible envelope. It carries the material needed to route, classify, and begin admission: profile, act type, sender reference, recipient scope, timestamps, version declarations, payload type, proof commitments, and integrity references. The visible envelope does not need to expose every private fact. It needs to expose enough for a receiver to know what kind of gate applies.

The second surface is the protected payload. This is the content that may be encrypted, access-controlled, redacted, partitioned, or disclosed only to intended recipients. A confidential direct profile may protect the payload. A group profile may disclose it to members of a fleet. A regulated profile may disclose different fields to different parties. The payload is where sensitive meaning may live.

The third surface is the proof commitment. A receiver may need to verify that the payload has not changed, that a dependency set is the one claimed, that a receipt corresponds to a particular message, or that a policy gate was evaluated. Hashes, signatures, proof IDs, receipt references, and profile-specific proof mechanisms can bind hidden content without revealing all of it.

The fourth surface is the policy-visible receipt. This is the structured memory of what happened: admitted, refused, quarantined, accepted, confirmed, errored, bridged, downgraded, or dependency-requested. A receipt can reveal the outcome of admission without revealing every private input that led to

it. It can say that a regulated transaction request passed authority and idempotency checks without making the full underlying contract public to every participant.

These surfaces let a protocol operate in domains that cannot tolerate naked meaning. The receiver gets enough structure to make an admission decision; the network does not get a free copy of everything the decision depends on.

The Audit and Privacy Collision

Full disclosure is not always trust. Sometimes it is a violation. A hospital cannot publish an entire patient record simply because a receiving system must verify that a referral is legitimate. A financial agent cannot reveal a complete trading strategy because another system needs to know that a transaction instruction is authorized. A fleet cannot expose private telemetry to every nearby observer because it broadcasts a hazard.

Full encryption is not always safety either. A receiver cannot admit what it cannot classify. A message that hides everything may be confidential, but it may also be useless. The receiver may not know which profile applies, whether the act is high-risk, whether the message is fresh, whether it falls within scope, or whether local policy permits handling it.

The protocol needs visible structure and protected content.

Profiles decide the boundary. A `PUBLIC_SIGNED_BROADCAST` profile may expose act type, sender credential class, location scope, timestamp, and hazard category while keeping private telemetry out of the message. A `CONFIDENTIAL_DIRECT` profile may reveal only enough envelope metadata to route and verify freshness while encrypting payload and dependency details to the recipient. A `REGULATED_TRANSACTION` profile may require visible idempotency and transaction references while protecting commercial payloads. A `HIGH_RISK_COMMAND` profile may require visible risk class and authority reference while encrypting mission content.

Selective disclosure is therefore not one privacy feature. It is a profile discipline.

A Regulated Transaction Without Naked Data

A regulated transaction illustrates the point. A sender issues a REQUEST to perform a transaction. The receiver must verify identity, authority, idempotency, freshness, profile support, and audit policy. But not every participant needs the full commercial rationale, all account details, or the private agreement behind the instruction.

The visible envelope can declare the act, profile, sender reference, transaction reference, timestamp, and proof commitments. The protected payload can contain the sensitive transaction terms for the intended receiver. The proof commitment can bind the protected payload to the envelope and let the receiver verify integrity. The policy-visible receipt can record whether the request was accepted, refused, quarantined, or confirmed.

The network does not need naked data. The receiver needs enough structure to admit or refuse.

A medical referral follows the same pattern. The receiving system may need to know that a legitimate clinical referral exists, that the sender has authority, that required fields are present, and that the payload is intended for that recipient. It does not follow that every intermediary should see the clinical details. The referral can be routed and verified through visible envelope data while clinical content remains protected.

The protocol should not pretend to solve privacy. Privacy is legal, technical, institutional, and contextual. Nor should it require one cryptographic method for all cases. Some profiles may use simple encryption and signatures. Others may use commitments, partitioned disclosure, audit rights, trusted intermediaries, or stronger proof systems. The architectural claim is narrower: the protocol must separate the surfaces so those methods have somewhere to live.

Receipts become especially important. A receipt is not a courtesy. It is the return path of accountability. It can record that a receiver admitted an act under a profile, refused because authority failed, quarantined because dependencies were missing, or confirmed execution within an authorized envelope. The receipt can reveal the decision surface without exposing every protected payload field.

This is the balance regulated systems need.

Meaning must be inspectable enough to admit.

Meaning must be protected enough to deploy.

The protocol succeeds only if both statements can be true at once.

CHAPTER 22

Trust Lenses

The graph records the world as it is claimed, not as it is reconciled. A lens makes the contradiction visible, but the receiver must decide what is admissible. To hide the conflict is to manufacture a fiction; to store the mess is to preserve the evidence.

WA graph that stores only one truth is brittle.

A graph that shows every claim all the time is unusable.

The Third Protocol needs both pluralism and decision. It needs to preserve claims without drowning the receiver in them. It needs to remember disagreement without pretending disagreement is paralysis.

The answer is the Trust Lens.

The graph is pluralistic. The lens is perspectival. The receiver remains sovereign.

Storage Is Not View

Storage and query are different layers. The storage layer preserves claims with provenance, source, time, confidence, derivation, and perspective. It does not delete one claim because another contradicts it. It does not average two claims into a third claim that no source asserted. It does not pretend that one graph's editorial policy should become the ontology of the world.

The query layer applies policy.

A conservative medical lens may surface only peer-reviewed sources, current clinical guidelines, and claims above a high confidence threshold. A newsroom lens may show confirmed reports and disputed eyewitness claims separately. A financial lens may rank filings and audited statements above

commentary. A research lens may show contradictions because contradiction is the work.

The underlying graph remains the same. The view changes.

This is not relativism. It is operational honesty. A lens does not make a weak source strong. It does not make a contradiction vanish. It declares what the receiver is willing to admit for a purpose.

Three Bad Options

Traditional systems often choose one of three bad options.

The first is delete-on-conflict. One claim wins. The other disappears. The graph becomes clean at the cost of memory. This works for some operational databases. It fails for knowledge. The losing claim may later prove correct. It may matter historically. It may reveal source behavior. It may be true under a different frame.

The second is merge-to-consensus. Conflicting claims are averaged, summarized, or softened into a synthetic middle. This may read well, but it creates a new claim no source asserted. It hides disagreement by manufacturing prose.

The third is surface-everything. Every claim appears. Every contradiction appears. Every source appears. This preserves evidence but destroys usability. The receiver asked for a usable view and received a research assignment.

The Trust Lens separates these concerns. The graph stores. The lens governs view. The distinction is what keeps pluralism from becoming paralysis and policy from becoming hidden ontology.

A Simple Conflict

Consider a company departure. One source reports that the CEO resigned. Another reports that the CEO was fired. A third reports that the board requested resignation. A graph that forces one truth will destroy information. A graph that averages the claims will invent a fourth fact. A graph that shows everything without policy may be useless to an automated receiver.

A Trust Lens can show all three as disputed. Another can surface the filing as authoritative. Another can mark the claim unresolved. Another can require

two independent sources. The receiver’s lens is not hidden in a model. It is explicit policy.

Lenses can be versioned. They can be audited. They can be compared. A receiver can run two lenses side by side and see how the answer changes. A regulator can ask which lens governed a decision. A receipt can record that a claim was admitted under a specific lens version.

This matters for the Global Ark. A shared knowledge substrate cannot require every participant to share one worldview. It can require every participant to expose enough provenance, policy, and receipt structure for differences to be inspected.

The graph stores the world as claimed.

The lens shows the world as a receiver is willing to use it.

Lens Table

Lens mode	What it surfaces	When it helps	Failure if hidden
Pluralist	all claims with conflicts marked	research, investigation, audit	receiver mistakes disagreement for absence
Conservative	only high-authority/current claims	clinical, safety, compliance	receiver may miss emerging evidence
Consensus	claims corroborated by multiple sources	public summaries, low-risk knowledge	minority truth may be suppressed
Adversarial	claims plus counterclaims and provenance	red-team review, litigation, intelligence	too noisy for routine action
Local policy	claims admitted under receiver rules	enterprise action, regulated workflow	policy becomes invisible if not recorded

A lens is not a truth machine. It is a declared view. Its virtue is not perfection; its virtue is explicitness. The receiving organization can say which view

governed an answer, which sources were excluded, which thresholds were used, and which conflicts remained visible.

Hidden lenses already exist in ordinary systems. Ranking algorithms, search defaults, database views, analyst habits, and institutional preferences all filter reality. The Trust Lens makes the filter inspectable.

That is the difference between perspective and manipulation.

Lens Provenance

A Trust Lens should itself have provenance.

If a receiver admits a claim under a particular lens, the receipt should be able to name that lens, its version, and the policy it applied. Otherwise the system can explain the source of the claim but not the source of the view. The graph may preserve provenance while the query layer becomes opaque.

This is a subtle failure. A system can say, “The fact came from this source,” but cannot say, “This is why that source was trusted, this other source was excluded, and this contradiction was hidden.” That is not enough for high-stakes use.

Lens provenance should record the source hierarchy, confidence thresholds, freshness requirements, domain exclusions, authority preferences, and conflict-handling mode. A lens used for medical safety should not be confused with a lens used for investigative research. A lens used for public summary should not be confused with a lens used for legal action.

The lens is part of the answer.

If the lens is hidden, the answer is not fully auditable.

The Raw Graph Right

Trust Lenses should not become prisons. A system may provide a default lens, but the underlying claims should remain queryable by authorized users and systems. Otherwise the lens becomes an invisible censor rather than an explicit view.

Raw access does not mean unsafe action. A receiver may allow raw graph inspection while refusing to let raw, unfiltered claims enter automated decisions. That distinction matters. Researchers may need the mess. Operators

may need the filtered view. Automated systems may need the conservative lens.

The graph stores the mess because the world is messy.

The lens makes the mess usable.

Conflict Walkthrough: Resigned or Fired

Three sources describe the same executive departure.

Source A says the CEO resigned. Source B says the CEO was fired. Source C says the board requested a resignation after an internal investigation. A conventional system may choose one. A summarizer may average them into “the CEO left amid controversy.” That sentence reads smoothly and destroys the evidence.

The pluralistic graph stores all three claims. Each claim carries source, time, provenance, extraction method, and confidence. It also records that the claims refer to the same event candidate while disagreeing on the manner and agency of departure.

A legal lens may surface only official filings. A newsroom lens may show all three with conflict markers. A market-risk lens may weight the investigation detail more heavily. A public-summary lens may say that accounts differ and cite the sources. None of these lenses deletes the underlying claims.

This is why the lens matters. It lets receivers act under policy without pretending disagreement has vanished.

CHAPTER 23

The Evidence Gate

A system that cannot refuse is a botnet in waiting. The Evidence Gate is the structural threshold where the world's noise is smelted into admissible, auditable law.

A global graph will be attacked.

That is not pessimism. It is design input.

Any system that turns knowledge into machine-actionable infrastructure will attract jokesters, propagandists, spammers, lazy extractors, ideological editors, commercial manipulators, and adversarial agents. A volunteer global knowledge graph will face the same forces that face every public knowledge project, amplified by the fact that its output may be consumed by machines.

Global knowledge does not scale by optimism. It scales by adversarial verification.

The Evidence Gate is the admission procedure for mined claims.

The Jokester Problem

Every public knowledge project eventually meets the jokester.

The jokester adds a false birth date, invents a harmless-sounding relationship, changes a place name, inserts a fake quotation, or exaggerates a claim. In a human encyclopedia, this may be vandalism. In a machine-actionable graph, it can become a propagated fact.

The protocol must assume not only organized adversaries but also mischief. Some bad data will be malicious. Some will be careless. Some will be funny to the person who submits it. The receiver cannot know the motive from the claim itself. It can only evaluate evidence.

This is why source anchoring, reconstruction, independent extraction, challenge paths, and Trust Lenses matter. They turn the jokester's contribution into a low-trust claim rather than an admitted fact. The system does not need to know whether the contributor was malicious. It needs to know whether the claim deserves admission.

The joke stops at the gate.

Slow Truth

Some claims should not be admitted quickly.

Breaking news, disaster reports, public accusations, preliminary research findings, and crowdsourced emergency updates often arrive before verification. Refusing all early claims would make the graph useless in fast-moving domains. Admitting all early claims as facts would make it dangerous.

The Evidence Gate needs intermediate states: reported, unverified, disputed, corroborated, authoritative, superseded, retracted. A claim can be useful before it is settled, if its status travels with it. A disaster-response system may use unverified reports for situational awareness while refusing to treat them as commands. A journalist may query disputed claims. A safety system may require authoritative confirmation before action.

Truth often arrives slowly.

The protocol should not force it to pretend otherwise.

Source to Synapse to Quarantine

A public contributor submits a claim: a city ordinance prohibits drone delivery in a particular district after 8 p.m.

The Evidence Gate first asks for the source. The contributor provides a municipal code page. The system hashes the page, records retrieval time, and identifies the relevant section. GLEAN extracts a candidate Synapse: prohibition, drone delivery, district, time window, jurisdiction, source span.

The reconstruction gate compares the Synapse back to the source. It finds that the ordinance applies only to commercial drone delivery above a specified noise threshold, not to all drone delivery. The extracted Synapse is overbroad.

The system does not admit the claim as written. It quarantines it with a reason: source supports narrower claim. A second extraction produces the

narrower version. The wider claim remains in the record as a failed extraction, useful for audit but not for action.

This is the Evidence Gate doing its quiet work. It prevents a plausible simplification from becoming machine law.

The Five Gates

The first gate is source anchoring. A claim should point to a source document, source span, artifact, sensor record, or authoritative dataset. A claim without a source may still be stored as an unsupported assertion, but it should not enter high-trust paths as if it were verified knowledge.

The second gate is reconstruction. The system should be able to compare the extracted Synapse against the source. Does the source actually say this? Were pronouns resolved correctly? Was the claim inferred rather than expressed? Was a qualification dropped? Did a date or negation disappear? The gate does not require perfection. It requires a trace.

The third gate is independent comparison. When possible, multiple extraction paths or contributors should mine the same source or claim class. Agreement does not prove truth, but disagreement reveals where attention is needed. A crowdsourced graph should not hide disagreement to look clean.

The fourth gate is source comparison. A claim about a drug, statute, treaty, company, or event may appear in multiple sources. The system should preserve source differences rather than flatten them. One source may be authoritative. Another may be timely. A third may be wrong. The receiver's Trust Lens decides what to surface.

The fifth gate is challenge. A public or commercial knowledge graph needs a way to dispute claims, attach counterevidence, quarantine low-trust claims, and revise packs. Without challenge, the graph becomes a landfill with nicer formatting.

The Evidence Gate does not produce final truth. It produces structured admissibility. That difference is the whole point: a machine-actionable graph needs to know not only what was claimed, but why the claim was allowed to matter.

Mined Law

Consider a volunteer extracting a Louisiana statute into Synapses. The extracted claim points to the statute text, the source span, the version date, the jurisdiction, the effective date, and the extraction method. Another miner extracts the same statute. The two outputs differ on a condition. The system does not average them. It flags the conflict, preserves both traces, and routes the claim to review or a higher-authority source.

A downstream receiver may admit the claim as disputed. A legal profile may require official statutory text before admission. A research lens may show both extractions. A compliance system may quarantine the claim until an authoritative Knowledge Pack updates.

The same pattern works for medical claims, financial facts, public events, and encyclopedia knowledge. A claim may be admitted as verified, admitted as disputed, admitted as historical, quarantined, refused, or marked as requiring source review.

The Evidence Gate is that immune system at the boundary.

It does not ask the world to be honest.

It assumes the world will test the gate.

Evidence Gate Table

Gate	Question	Possible outcome
Source anchor	What source does this claim come from?	admit as sourced, mark unsupported, or refuse
Reconstruction	Does the source actually support the Synapse?	admit, flag extraction error, request review
Independent extraction	Do other extractors agree?	corroborate, dispute, or quarantine

Continued on next page

(Continued)

Gate	Question	Possible outcome
Source comparison	Do other sources support or contradict it?	rank, preserve conflict, apply Trust Lens
Challenge path	Can the claim be contested and revised?	open challenge, update pack, retain history

These gates are deliberately boring. That is their strength. A global knowledge system should not depend on charismatic trust. It should depend on dull, repeatable procedures that survive malicious input and tired operators.

The Evidence Gate also protects honest error. Not every bad claim is an attack. Some are mistranscriptions. Some are stale. Some are overbroad extractions. Some are real contradictions in the source world. The gate's job is not to punish every error. It is to prevent error from entering the graph as unearned certainty.

PART V

The Protocol in the World

Walkthroughs, agents, swarms, and the Global Ark

CHAPTER 24

What One Image Knows

Measurement is the raw flux of the world, but meaning requires the discipline of the address. A machine that cannot ground its observations in a stable lexicon acts on ghosts.

A camera sees a calico cat.

That sentence is already too generous. The camera does not see a cat. It receives photons. A perception model may classify a region of pixels as a cat. The label may have high confidence. But the label is not knowledge yet.

To a navigation system, cat may be an obstacle class. To a legal system, it may be property. To a child, it may be a pet. To a safety system, it may be a living thing that must not be struck. The label alone does not carry these meanings. It is a string standing at the edge of a library.

The Canonical ID opens the door.

The Proof of a Non-Text Protocol

The abstract claim is that meaning can cross boundaries without losing accountability. Perception is where that claim touches the physical world.

Text is forgiving. A sentence already arrives in symbolic form. A form field already declares that it is data. A transaction message already lives inside a system of records. An image does not. An image arrives as measurement: pixels, light, sensor noise, compression artifacts, lens distortion, motion blur, exposure, occlusion, and model confidence. If the architecture works only after a human or model has already named the world, then the architecture is not yet a bridge from the world into meaning. It is only a bridge from text into text.

The calico cat forces the question. Can raw perception cross into the same protocol layer as prose, transactions, and agent acts? Can the machine carry the observation without pretending the original pixels are the meaning? Can another machine use the observation without trusting the first machine's camera blindly? Can the receiver ask for the evidence, verify the pointer, hydrate the ID, and still refuse action if confidence is not enough?

If the answer is yes, then the protocol is not text-bound.

It is ID-native.

That is why the image matters.

From One Image to Many Machines

Once the observation is in HFF, it can move beyond the original perceiver.

The drone that saw the cat may not be the machine that needs the knowledge most. A ground robot may be approaching the porch. A second drone may be planning a descent. A delivery scheduler may need to delay a drop. A municipal animal-control system may need no action at all but may accept the observation as low-priority evidence. Each receiver has a different policy.

This is why the observation should not be encoded as a private sensor label. It should be transmitted as grounded meaning with evidence references and confidence. Each receiver can then decide what the observation means for its own action boundary.

The ground robot may mark the area as occupied.

The second drone may reroute.

The scheduler may delay.

The municipal graph may ignore the observation because the sender lacks authority for that graph.

The same observation crosses four boundaries and receives four different admissions.

That is not inconsistency. It is the point.

From Pixel to Address

The image embedding proposes candidates. The Binding Engine or perception gateway evaluates the candidates under a sensor profile. Lighting, motion blur, occlusion, camera calibration, and confidence floor matter. If the confidence is insufficient, the system should not invent certainty. It should return UNKNOWN, slow down, request another view, or trigger a safety fallback.

Dignified refusal is part of perception.

If the image resolves to a Canonical ID for calico cat, the system can hydrate the ID. The Core Lexicon can place it under domestic cat, feline, mammal, animal, living thing. The entry may include images, example media, alternate surface forms, synonyms, HAS_PART structure, and descriptive Synapses. A domain graph may add local information. A municipal graph may know animal-control rules. A delivery drone's local policy may know not to descend on a porch where a living animal is present.

The raw image does not need to live in the graph.

Dense media belongs in media/object storage. Time-series data belongs in time-series storage. The graph stores meaning, pointers, hashes, provenance, and admissible structure. The Synapse may carry HAS_SOURCE or equivalent evidence references pointing to the image, along with a content hash. If an auditor or receiver is authorized to inspect the image, it can retrieve the media and verify the hash. If not, the graph can still carry the fact that a perception event occurred under a declared profile.

Meaning and measurement are different.

The graph should not swallow the river. It should mark the point where the river mattered.

From Address to Action

Now the observation can travel. HFF carries the grounded observation: an AFP INFORM or ADVISE, a Synapse for the detected living thing at a time and place, dependency material for any non-core identifiers, profile declarations, source pointer, confidence/provenance information, and receipt expectations. Another machine can receive the bundle, hydrate the Canonical ID, understand that the detected entity is a living thing, and adjust its own policy without ever seeing the original pixels.

A ground robot with no camera may receive the drone’s observation. The robot knows only lidar. It has never seen the cat. But the HFF message does not ask the robot to understand pixels. It asks the robot to hydrate a Canonical ID and evaluate a location. The robot can mark the area as occupied by a living thing under its own safety policy.

This is not magic. If the binding is wrong, downstream reasoning is wrong. If the lexicon is incomplete, the system may know less than it needs. If the source pointer is inaccessible, audit may be limited. If the confidence falls below the floor, the correct answer is UNKNOWN. The protocol does not solve perception. It disciplines the boundary where perception becomes meaning.

That boundary changes everything.

A camera, a lidar unit, a microphone, a text prompt, and a human-entered form can all produce observations that resolve to Canonical IDs. Below the ID, modality matters. Above the ID, the receiver can reason over grounded structure.

One image.

One key.

The whole library opens.

What Travels and What Does Not

Layer	Stays local or external	Travels in HFF
Raw image/ video/audio	media store or sensor system	URI/reference, hash, profile
Embedding	perception/binding system	binding result, profile, confidence metadata
Canonical ID	graph/lexicon address	ID and dependency material
Lexicon context	Core/Knowledge Pack	required dependency subset or pack reference
Inference result	local reasoning graph	admitted Synapse, provenance, receipt
Action request	receiver policy layer	AFP act and profile

This table prevents a common mistake. HFF does not need to carry a 4K video because a receiver needs to know that a cat was observed. It carries the structured meaning of the observation and the evidence handle required to audit it. If the receiver is authorized to fetch the media, it can. If it is not, it may still admit limited structure under a profile.

The graph stores the fact that matters. The media store preserves the measurement. The hash binds them.

The Calico Cat as Protocol Test

The calico cat is not cute decoration. It is a protocol test.

If the architecture can carry only text-derived facts, then it is a document pipeline. If it can carry only sensor labels, then it is a perception pipeline. If it can resolve perception to a Canonical ID, hydrate the ID through the lexicon, preserve the evidence pointer, transmit the observation through HFF, declare an AFP act, and let another machine decide under local policy, then it is a protocol for machine meaning.

The cat proves the boundary.

Below the Canonical ID is modality: pixels, tensors, confidence, sensor profile, calibration.

Above the Canonical ID is meaning: taxonomy, policy, inference, federation, action.

The crossing is the architecture.

A Failure Case: The Bag That Looked Like a Cat

Image grounding is not a magic wand. A plastic bag can look like an animal in poor light. A statue can look like a person. A reflection can look like an obstacle. A cat can be partly hidden behind furniture. A thermal sensor and a camera may disagree. The protocol does not abolish perception error.

It changes what happens after perception becomes uncertain.

Suppose the drone sees the porch in rain. The image model proposes `calico_cat` at moderate confidence. The sensor profile subtracts for rain, low light, and motion blur. A second frame proposes `plastic_bag` at lower confidence. The system now has competing candidate bindings. The wrong

design would average them into a bland confidence score and continue. The disciplined design preserves the conflict.

The HFF observation can say:

- candidate A: calico cat, confidence after calibration;
- candidate B: plastic bag, confidence after calibration;
- source: camera frame range;
- condition: rain and low light;
- policy result: insufficient confidence for descent;
- outcome: REFUSE or local halt.

This is Dignified Refusal in perception. The machine does not need to know exactly what the object is in order to know that action is unsafe. In many physical domains, uncertainty itself is the fact that matters.

The receiver does not admit “cat” as truth. It admits “unresolved living-object risk at location X” under a safety profile.

That distinction is why the protocol can be both useful and honest.

The Lexicon as a Multimodal Doorway

Lexicon entries can carry more than text. A mature Core or domain lexicon may include images, diagrams, audio clips, video examples, embeddings, source references, and domain-specific sensory profiles. These media artifacts are not stored as giant blobs inside the reasoning graph. They are referenced, hashed, embedded, and governed.

This gives a machine more than one path to the same entry. A word can find the ID. An image can find the ID. A sound can find the ID. A barcode, signal signature, or sensor pattern can find the ID. Once the ID is found, the protocol layer no longer cares which modality opened the door. It cares about the evidence, confidence, profile, and admissibility of the binding.

This is the practical meaning of a machine interlingua. It is more than cross-language. It is cross-modality.

A bird call, a written species name, and a photograph can all point toward the same biological entry. A spoken emergency alert, a dashboard code, and a signed HFF broadcast can point toward the same incident type. A machine that cannot understand the original medium may still hydrate the ID and reason over the meaning, if policy allows.

The Canonical ID is the doorway.
The lexicon is the building.
HFF is how the key travels.

The Corroboration Path

An emergency broadcast does not have to stand alone. A receiver can treat the message as one input among several.

The vehicle may check its own sensors. It may observe flashing lights or siren audio. It may receive similar broadcasts from roadside infrastructure. It may receive acknowledgements from other vehicles ahead. It may query a local traffic authority sidecar if network conditions allow. Each corroborating signal increases confidence, but none removes the local safety gate.

This creates a useful middle state between blind obedience and total rejection. A receiver may slow, increase following distance, prepare a lane change, or raise attention thresholds before fully accepting the request to yield. The action profile can distinguish cautionary behavior from committed maneuver.

This matters because real environments are noisy. Emergency broadcasts may arrive late, partially, or at the edge of range. A receiver should not be forced to choose between full action and no action when partial evidence supports caution. The profile can allow cautious local preparation while withholding full acceptance until enough gates pass.

The protocol makes that middle state visible. A receiver can ACK receipt, mark the message as under evaluation, and later ACCEPT or REFUSE. The sender's system sees the state of coordination rather than guessing from vehicle motion.

This is a more honest model of real safety. Machines often act under partial evidence. The protocol's job is to make partial evidence explicit, bounded, and auditable.

What the Ambulance Receives Back

The sender also benefits from structured outcomes.

An ambulance broadcasting a request into traffic does not need every nearby vehicle to send a verbose report. But in a coordinated deployment, aggregate responses can matter. If many vehicles in the left lane return ACCEPT, the ambulance's routing system can infer that a corridor is forming. If several vehicles return REFUSE for the same obstruction, the routing system can revise expectations. If many receivers return no response because they are out of scope, the broadcast profile can avoid unnecessary channel load.

This turns the emergency broadcast into a feedback loop without requiring central control. The ambulance does not command the road. It receives structured evidence about how nearby receivers are responding under their own policies.

That feedback can be displayed to a human driver, consumed by an autonomous routing system, or stored for later review. It can also reveal infrastructure problems: a region where receivers fail to hydrate emergency messages, a fleet whose certificates are stale, a map whose geofence definitions are wrong, or a recurring refusal reason tied to road design.

Receipts become operational telemetry.

Not telemetry about raw sensors.

Telemetry about coordination.

CHAPTER 25

Walkthrough: The Emergency Broadcast

Authority is a claim, not a mandate. Refusal is the structural proof that the receiver's local reality remains the final arbiter of action.

The Reader Gloss

The message, rendered for human readers, runs approximately as follows: there is a medical emergency; an ambulance is proceeding along a specific urban route toward a hospital; vehicles in the left lane of the route are requested to yield to the right when they can safely do so.

This gloss does not exist on the wire. No machine processes it. The machine processes declared acts in an AFP envelope, carrying an HFF bundle that grounds each referring expression to a lexicon-identified object. The gloss is a human display layer. The wire format is what the protocol processes. These are different things, and both must remain distinct.

The gap between the gloss and the wire is not cosmetic. The gloss is ambiguous in ways the wire format is not. “When safe” is a natural-language hedge that a machine must instantiate as a policy evaluation against local sensor data. “Along a specific route” must resolve to a sequence of identified geofenced segments, not a vague description. “Vehicles in the left lane” must resolve to a recipient scope that the receiver can evaluate against its own identity and position. The wire format does not preserve the gloss’s looseness; it replaces it with resolvable references and structured act types.

The Wire Object

AFP Acts and HFF Bundle

The message carries three AFP acts: INFORM, ADVISE, and REQUEST. Each is a declared illocution with a specific conversation role.

The INFORM act declares a state of the world: a medical emergency event, grounded to a lexicon entry for that event class, with temporal and location attributes bound to specific Synapses in the HFF payload. The sender does not ask the receiver to do anything with this information; it contributes a claim to the receiver's local state.

The ADVISE act declares the sender's recommended interpretation: the ambulance is in transit on a declared route to a declared destination. This is not a command; it is a structured recommendation that the receiver may weight according to its trust model and plausibility evaluation.

The REQUEST act asks receivers in scope to yield the left lane when local conditions permit. This is the operative act: it has practical consequence if admitted. The AFP envelope's `recipient_scope` field bounds who should evaluate this request; receivers outside the declared scope do not process it as directed at them.

The HFF bundle grounds each element. The ambulance is not a string "ambulance"; it is an identified instance with a canonical reference, type tether, attributes, and trust material the receiver can inspect. The route is not a road name; it is a sequence of identified segment objects with coordinates and identifiers. The destination is not "Glenwood Hospital"; it is a canonical entity ID bound to a geolocation record. The timestamp is not approximate; it is a UTC value against which the receiver will run its freshness gate.

The profile is `PUBLIC_SIGNED_BROADCAST`. The payload is not encrypted; vehicles in the area need to read it. Anti-spoofing comes from signature verification, trust anchor validation, expiry, replay prevention, and plausibility checks at the receiver.

Dependency Hydration

The HFF bundle declares lexicon dependencies. If the receiver's local lexicon does not contain an entry for the emergency event type, the ambulance entity class, or the route segment identifiers, the message cannot be admitted: the meaning cannot be unpacked.

For well-maintained receivers operating in a shared deployment with the sender, these dependencies are typically pre-shared. For receivers encountering a new sender or operating at the edge of a deployment's coverage, the receiver may need to request the dependency before admission can proceed. The profile determines whether the receiver quarantines the message pending hydration or returns an error.

A missing dependency is not a security failure. It is an admissibility failure: the protocol cannot adjudicate meaning it cannot ground. The structured outcome in this case is either a quarantine with a dependency request or an ERROR indicating the missing lexicon reference.

Admission Gates

With the payload parsed, hashes verified, and dependencies hydrated, the receiver runs the admission pipeline.

Signature and trust anchor. The receiver verifies the signature against the declared `key_id` and follows the `trust_anchor_ref` to evaluate whether the signing key is in a trusted chain for emergency vehicle broadcasts. For a legitimate ambulance operation, this trust chain runs from the vehicle's certificate through a fleet operator certificate to a transportation authority root. If the key is not in a trusted chain for this act type, the receiver returns a REFUSE or ERROR depending on whether the sender's identity is recognized at all.

Freshness. The receiver checks `expires_at` and verifies the nonce against its replay cache. A freshness failure returns a structured ERROR. The receiver does not act on a stale emergency request; a request that was appropriate at issuance may not be appropriate after its window has elapsed, and a replayed request from a prior incident must not retrigger yielding behavior in changed conditions.

Recipient scope. The `recipient_scope` in the AFP envelope declares the target population. A receiver outside the declared scope, geofence, or participant class does not evaluate the REQUEST as directed at it. Scope filtering runs before authorization and policy evaluation; there is no benefit in running later gates against a message that is not addressed to this receiver.

Plausibility. The receiver evaluates whether the claimed emergency is consistent with what it can verify locally. Plausibility is not truth verification; the receiver cannot confirm the ambulance is present. It can confirm that the declared route overlaps with the receiver's known road network, that the claimed vehicle type is consistent with registered emergency types for this trust anchor, and that the geofence coordinates are within the receiver's operating area. An implausible message, such as one claiming an emergency vehicle on a route that does not exist in the receiver's map data, fails this gate.

Local policy and safety. The receiver evaluates the REQUEST to yield against its current state. This is where the local decision is made. If the vehicle is in the correct lane and can safely yield, the result is ACCEPT. If the vehicle is in the correct lane but cannot safely yield because its own sensors detect a pedestrian crossing or an obstruction on the right shoulder, the result is REFUSE. If the vehicle is not in the relevant lane, the scope gate already handled this case; the vehicle does not produce an outcome.

A vehicle whose safety kernel determines that the requested maneuver is unsafe does not execute the maneuver. The emergency authority does not override this determination; it increases the weight the receiver assigns to the request when evaluating whether safe conditions exist. The receiver's local sensor data may resolve conditions faster than the request expires, in which case the receiver may subsequently ACCEPT after initially holding.

Outcome and Receipt

The receiver emits an outcome and records a receipt. For a vehicle that yielded: ACCEPT, with the acted-on AFP message ID, the timestamp of action, and the receiver's identity. For a vehicle that refused: REFUSE, with a reason code indicating why (out of safe maneuver window, obstruction detected, safety kernel block), and the same message ID and timestamp. For a vehicle

out of scope: no response expected by the protocol; the scope filter precluded evaluation.

The ambulance’s dispatch system, if it receives structured outcomes from nearby vehicles, can observe the progress of lane clearing in real time. Vehicles that refused have reported why. Vehicles that accepted have confirmed. The coordination is transparent at the protocol level, not just at the physical level.

The receipt is an accountability artifact. If a subsequent review examines why a particular vehicle did not yield, the receipt provides the answer: which gates ran, which passed, which failed, and on what grounds the receiver decided. That record is more useful to safety investigation than any post-hoc reconstruction could be.

A signed request is not a command to suspend judgment. It is an authenticated input to local policy, grounded in the receiver’s knowledge of its own state.

Emergency Broadcast Outcome Matrix

Receiver state	Admission result	AFP response	Physical behavior
in scope, safe to yield	admitted	ACCEPT, then CONFIRM if completed	yields under local driving policy
in scope, unsafe to yield	understood but refused	REFUSE with reason	maintains safe trajectory
out of scope	not relevant	optional ACK or no response per profile	no maneuver
stale packet	rejected	ERROR or silent discard by profile	no maneuver
bad signature	rejected	ERROR or security log	no maneuver
plausible but weakly supported alert	quarantined or cautiously admitted	ACK plus local caution	slows, monitors, requests corroboration

This matrix matters because emergency systems tempt designers toward binary thinking. Either the message is trusted and obeyed, or it is rejected.

Real receivers need more outcomes. A car that cannot yield safely should not behave as if it ignored the emergency. It should return a structured refusal if the profile expects response. A car outside the geofence should not pollute the channel with unnecessary replies. A car that receives a plausible but weakly supported alert may slow or increase sensor attention without committing to a lane change.

The protocol creates a vocabulary for these distinctions.

Why Public Does Not Mean Naive

Public signed broadcast is not insecure broadcast. It is public because the content is meant to be read by nearby receivers. The ambulance wants surrounding vehicles to know that an emergency route exists. Encrypting the basic alert to unknown nearby vehicles may defeat the purpose of the alert.

But public readability does not mean public authority. The receiver still verifies the signature, trust anchor, freshness, scope, plausibility, and local safety. A public message can be readable and still inadmissible. A readable alert from an untrusted signer is just noise. A readable alert outside its time window is stale. A readable alert that conflicts with local sensor reality may trigger caution rather than obedience.

This is the point of profiles. Confidentiality is one gate. It is not always the first gate, and it is not always the right gate for public safety broadcasts.

Command Versus Control

A high-risk command is not the same as control.

Control implies that the sender determines the receiver's behavior. Command, in *The Third Protocol*, means the sender asserts an authority claim over a requested act. The receiver still evaluates that claim. It may accept. It may refuse. It may quarantine. It may require additional evidence. It may ask for a safer alternative.

This distinction is easy to miss because ordinary language uses command as if obedience follows. In machine systems, that assumption is the dangerous part. A protocol that treats command as control has created a remote actuator. A protocol that treats command as an authority-bearing act has preserved receiver sovereignty.

The difference appears in the receipt. A controlled device may simply change state. A sovereign receiver records what it received, which authority was asserted, which gates passed, which local constraints were checked, and why it acted or refused. That receipt may be the most important artifact in the entire exchange.

The receiver's refusal is not insubordination. It is the protocol's structural protection that local state still matters. A sender can know much. It cannot know everything the receiver knows at the moment of action.

This is especially true for embodied systems. The sender may know mission goals. The receiver knows current sensor state, actuator health, immediate hazards, and local commitments. The command may be correct in mission space and wrong in physical space. The receiver is where mission meets matter.

The Pause State

High-risk receivers need more than accept and refuse. They often need pause.

A drone may receive a reroute command while it is completing a landing sequence. The command may be valid and authorized, but immediate execution would be unsafe. Refusing permanently would also be wrong because the command may become executable after the landing sequence completes. The correct outcome is a structured hold or queued state under profile.

This state should not be hidden. The receiver should be able to say: command received, authority valid, execution delayed because current mission state blocks immediate action, retry or await confirmation at a specified condition. That response gives the sender more information than refusal and safer behavior than immediate execution.

In physical systems, timing is part of meaning. A command that is valid after three seconds may be invalid now. A protocol that cannot express that difference will force receivers into crude yes/no behavior.

The high-risk profile should therefore include the possibility of delayed admissibility. The receiver does not obey late. It admits later, if and when the required state becomes true.

CHAPTER 26

Walkthrough: The High-Risk Command

A signature confirms the source, but it cannot guarantee the safety of the act. The receiver's local kernel remains the final, unyielding arbiter of physical consequence.

The Hardest Message

Not all acts request yielding. Some acts, if admitted, cause irreversible physical consequences: a drone changes course, a robotic arm changes state, a vehicle accelerates or brakes, infrastructure switches configuration, a financial position changes. For these acts, a mistaken admission is not a recoverable coordination failure. It is a physical or economic event that may not be undoable.

The high-risk command is the hardest message the protocol must handle, because its stakes are asymmetric. A wrongly refused high-risk command delays or blocks a legitimate action; that is recoverable through human escalation or retry. A wrongly admitted high-risk command may cause harm before any human is in a position to intervene.

The protocol does not treat this asymmetry by prohibiting high-risk commands. Such a prohibition would make the protocol useless for the systems that need it most. It treats the asymmetry by making the admission gates proportional to the risk class, and by preserving the receiver's right to refuse even when the command passes every formal gate.

Weapons deserve specific mention because they are the edge case that makes the asymmetry starkest. Weapons are not treated here as a detailed scenario for operational guidance; they are named because omitting them would leave a gap in the analysis. A weapon-firing command is a high-risk command. The same admission analysis applies: signature is not obedience, authorization is act-specific, local state and safety constraints participate in the decision, and the receiver retains structured refusal rights. The protocol makes no additional claims for this class beyond what the high-risk profile requires.

Signature Is Not Obedience

A valid signature on a high-risk command proves that the canonical message bytes were signed by the holder of the referenced private key. This is the extent of what the signature proves.

It does not prove that the key holder is authorized to direct this receiver to perform this act at this time. Authorization for a high-risk act requires an *AuthorityFrame* that specifies the act class, the risk level, the recipient scope, and the conditions under which the authority applies. The sender may hold authority for some commands to some receivers under some conditions; the *AuthorityFrame* makes the scope explicit so the receiver can evaluate it.

It does not prove that the sender's world model is consistent with the receiver's current state. The command may have been issued against a model of the receiver's environment that was accurate at issuance but has since changed. A drone routing command may have been computed against airspace state that has been updated by subsequent traffic. A robotic arm command may have been computed against a workspace model that does not reflect an obstacle introduced since the model was captured.

It does not prove that the action is safe given the receiver's sensor readings and safety constraints. A receiver with an active safety kernel that detects a condition making the commanded action dangerous must evaluate that condition against the command, regardless of the command's authentication status.

The sequence is inviolable: authenticate first, then authorize, then evaluate local state and safety. None of these steps can substitute for the others, and passing earlier steps does not release the receiver from later ones.

AuthorityFrame and Scope

The `authority_frame_id` in a high-risk command references a frame that the receiver evaluates to determine whether the authenticated sender holds authority for this specific act. Authority is act-specific, not identity-specific.

A sender who holds legitimate authority to command a drone fleet to return to base does not thereby hold authority to command individual drones to fly routes outside their registered operating area. A sender who holds authority to authorize vehicle maneuvers in one jurisdiction does not thereby hold authority in another. The AuthorityFrame encodes these boundaries. The receiver's evaluation of the frame checks whether the requested act, at the declared risk class, within the declared recipient scope, falls within the authority the frame grants.

The `risk_class` field declares the severity level of the commanded act. Risk class affects which safety checks are mandatory, whether human-in-the-loop review is required by local policy, and which fallback behaviors apply if safety constraints prevent execution. A receiver that does not support the declared risk class must return a structured ERROR rather than silently downgrade the risk evaluation.

The `recipient_scope` constrains who this command is directed at. Like the broadcast case, a receiver that is not in scope does not evaluate the command as directed at it. For high-risk commands, scope is more precisely defined than for broadcasts; it typically names specific receiver identities or small groups rather than geographic zones. A command scoped to a specific drone with a specific mission ID applies only to that drone; other drones that receive the message do not process it as operative.

The combination of AuthorityFrame, risk class, and recipient scope means that a valid signed high-risk command is not a general license to act. It is a precisely bounded instruction that the receiver must evaluate against its own configuration to determine whether it falls within the intended target population.

Local Safety and Mission State

The receiver's safety evaluation for a high-risk command operates on two tracks: static safety kernel constraints and dynamic mission state.

The safety kernel contains constraints that are frozen, cryptographically signed, and auditable independently of the instruction being evaluated. These constraints encode conditions under which the receiver must not execute certain act classes regardless of the command's provenance. A safety kernel constraint that blocks an arm movement when a pressure sensor reads above a threshold, or blocks a vehicle maneuver when a proximity sensor is active, cannot be overridden by an authenticated instruction. The constraint's purpose is precisely to remain effective even against legitimate but situationally inappropriate instructions.

Mission state captures where the receiver is in its current sequence of tasks and what commitments it has made. A drone mid-sequence through an autonomous delivery task has entered commitments about its trajectory, altitude, and timing. A command that conflicts with those commitments may be safely executable after the sequence completes, but not during it. The receiver's evaluation of the command against mission state determines whether the command can be executed immediately, must be queued, or must be refused because the conflict cannot be resolved without abandoning a prior commitment the receiver has no authority to unilaterally break.

These two tracks are not alternatives. A receiver facing a high-risk command evaluates both the safety kernel and mission state, and produces REFUSE if either blocks execution. The reason code in the refusal distinguishes between a safety kernel block (which is definitive; the sender cannot resolve it by adjusting the command) and a mission state conflict (which may be resolvable after the current sequence completes).

Refusal and Receipt

A high-risk command that passes all cryptographic gates, clears AuthorityFrame validation, falls within recipient scope, and satisfies plausibility may still be refused. The receiver whose local sensor state, safety kernel constraints, or mission state make execution unsafe or inappropriate is required to refuse, not to attempt execution and fail at the physical layer.

REFUSE in this context is the correct outcome. It acknowledges that the command was received, understood, and evaluated, and that the receiver will not execute it under current conditions. The receipt records the gates evaluated, the conditions checked, and the grounds for refusal. If the grounds are transient, such as a safety kernel constraint triggered by a temporary sensor reading, the sender's system may retry after the condition resolves. If the grounds are permanent, such as a scope mismatch, the retry will produce the same outcome and the sender should escalate through a different channel.

CONFIRM is the outcome when execution completes. For high-risk commands, confirmation is not optional where the profile specifies `receipt_policy` requiring it. The sender needs to know whether the act was performed, more than that the message was received. The gap between ACK (received), ACCEPT (will execute), and CONFIRM (executed) represents the time during which the action was in progress. A system monitoring high-risk command execution tracks this sequence as the accountability record for the action.

The safe fallback for a receiver that cannot determine the appropriate outcome is to refuse, quarantine, or hold state. Attempting execution under uncertainty is the failure mode the admission pipeline is designed to prevent. An uncertain receiver that refuses and records the uncertainty is contributing useful information to the coordination system. An uncertain receiver that attempts execution and causes harm has produced the outcome the protocol's admission gates exist to prevent.

HFF can prove who spoke. It cannot make obedience safe. The receiver's evaluation is not an obstacle to coordination. It is the condition under which coordination remains safe enough to continue.

High-Risk Command Outcomes

Gate failure	Meaning	Correct outcome	Sender's next move
invalid signature	sender cannot be authenticated	ERROR / security log	reissue with valid credential

Continued on next page

(Continued)

Gate failure	Meaning	Correct outcome	Sender's next move
authority mismatch	sender lacks authority for this act/receiver/scope	REFUSE	escalate authority or reroute
stale command	command outside freshness window	ERROR or REFUSE by profile	issue new command
recipient out of scope	command not directed to this receiver	REFUSE or ignore by profile	select correct recipient
mission-state conflict	receiver cannot execute now without breaking prior commitment	REFUSE with transient reason	retry later or cancel prior commitment
safety-kernel block	local safety constraint prohibits execution	REFUSE with safety reason	do not retry unless condition changes
uncertain world state	receiver cannot establish safe conditions	quarantine / hold state	request clarification or additional evidence

This table is not meant to make high-risk command easy. It is meant to make the failure modes legible. A system that can say only “failed” is not safe enough. A sender needs to know whether it sent to the wrong receiver, used the wrong credential, lacked authority, hit a transient mission conflict, or attempted an unsafe act. The difference determines whether the next move is retry, escalate, wait, cancel, or abandon.

The profile's job is to make these differences machine-readable.

The Audit Thread

Regulated systems often have logs. Logs are not the same as an audit thread.

A log records events in the system that produced it. An audit thread connects the acts across participants. The sender's request, receiver's acceptance, domain execution, confirmation, refusal, error, and any later cancellation or correction belong to one thread. That thread should be visible without forcing an auditor to reconstruct meaning from multiple local logs.

This is why transaction references, idempotency keys, act IDs, and receipts are not clerical details. They are the stitching that holds the thread together.

Imagine a dispute. The sender says the request was accepted. The receiver says execution failed. The domain rail says a transaction reference exists, but the amount differs from the sender's expected amount. Without a shared act thread, the investigation becomes archaeology. Each system produces logs. Humans infer what happened.

With a protocol thread, the question narrows. What did the REQUEST say? What did the ACCEPT accept? Did the CONFIRM reference the same transaction? Was there an ERROR between acceptance and confirmation? Did a later CANCEL attempt to unwind the act? Which profile governed the exchange?

The audit thread does not prevent every dispute. It prevents disputes from beginning in darkness.

Cancellation and Correction

Regulated transactions also need a disciplined way to handle cancellation and correction.

A sender may discover an error after REQUEST but before ACCEPT. In that case, CANCEL can close the pending request if the receiver has not yet accepted it. The receipt records that the request was cancelled before admission.

After ACCEPT, cancellation becomes more complex. The receiver may have committed resources or passed the instruction to a domain rail. A CANCEL after ACCEPT is not guaranteed to unwind the act. It is a new act requesting termination or reversal under domain rules.

After CONFIRM, cancellation is no longer cancellation of the original act. It is a correction, reversal, refund, amendment, or new transaction. The protocol should not let a late CANCEL pretend that the confirmed act never happened. The record remains. A new act may offset it, but the history is not erased.

This distinction is essential in finance, law, logistics, and medicine. Correcting the world is not the same as deleting the record.

CHAPTER 27

Walkthrough: The Regulated Transaction

A signature confirms the sender, but only a typed gate confirms the right to act. Regulated systems collapse when they mistake the intent of a request for the finality of a consequence.

Money Is Motion Without Mass

A physical vehicle moving through a junction and a financial instruction moving across a settlement network have more in common than their surfaces suggest. Both cause consequence in the world. Both pass through systems that must decide whether to admit or reject them. Both can create irreversible states. And both, if mishandled at the admission layer, produce damage that no downstream correction can undo.

The vehicle brings its mass and velocity. The financial instruction brings none of those things, which makes it faster and harder to trace when it goes wrong. A failed cargo delivery can be observed at the dock. A duplicated transfer instruction, or one executed by an unauthorized agent, may not surface for hours or weeks, and by then the effect has already propagated through settlement, reporting, and counterparty records in a way that touching the original entry cannot fully reverse.

The admission framework applies to regulated transactions without replacing domain settlement infrastructure—clearing networks, custody chains, regulatory ledgers. Those systems remain outside the protocol's claim. The

narrower argument is that any agent that issues, routes, or executes a regulated instruction creates a message that crosses a boundary and causes action. That message has the same admissibility problem the vehicle instruction has. Its meaning must be admitted, not assumed.

The machinery differs. The admission structure does not.

Instruction Is Not Execution

A signed financial message is not the executed trade. The confusion between these two states is more common than it should be, and its consequences are proportionally severe.

Consider the chain of states a transfer instruction traverses. An agent sends a message asserting that a given account should transfer a specified amount to a counterparty. The message is signed. The signature is valid. The sender's identity is confirmed. None of those facts, taken together, constitute authorization to execute.

Authorization depends on several conditions that the signature does not establish. The sender must hold authority over the account in question, at the amount specified, for the instrument being moved. That authority is scoped by amount, by instrument class, by counterparty, and by time window. A message from a sender with authority to move a hundred units does not carry authority to move a thousand. A message specifying an equity transfer does not carry authority for a currency transfer from the same sender. Authorization is granular and the signature does not compress it.

Beyond scope, the instruction must be fresh. Replay attacks against financial systems work by re-presenting a previously valid instruction after conditions have changed. The instruction carries a timestamp and an idempotency key, and the receiver must check both. An instruction whose idempotency key has already been consumed is a duplicate, regardless of whether it is signed and regardless of whether all other conditions are met. Execution on a duplicate produces two consequences where one was intended.

The profile for a regulated transaction therefore separates identity from authority, and authority from execution. A REQUEST carries the intent to transact. An ACCEPT declares that the receiver has checked identity, authority, scope, idempotency, freshness, and policy, and found the instruction admissible.

The ACCEPT is not execution. It is the gate that makes execution possible. Execution is a domain-layer act. CONFIRM is the message that closes the act loop, carrying the transaction reference, the timestamp, and the receipt that becomes the audit record.

What the Third Protocol provides here is not settlement logic. It is the typed structure that keeps these states distinct across any combination of senders, receivers, and domain rails.

Profile Fields for the Regulated Message

The Dependency Bundle for a regulated transaction is denser than the vehicle instruction because the authority surface is more complex.

The sender identity field carries the agent's decentralized identifier and the trust tier under which the message was submitted. Authentication at the DID level establishes who is sending; it does not establish what they are authorized to send.

The authority scope fields describe what the sender may do. These include the account or accounts over which the sender holds authority, the instrument class, the maximum amount bound, the counterparty constraint if any, and the time window during which the authority is valid. A sender may hold authority over multiple accounts with different constraints on each; the message must specify which account and which constraint applies to this instruction.

The transaction descriptor fields carry the specific terms: the hub verb identifying the act (transfer, approve, certify, cancel), the amount if applicable, the instrument identifier, the destination or counterparty, and any conditions that must hold at execution. A conditional approval message that omits its condition is structurally incomplete. The profile requires it to be explicit.

The idempotency key is a stable identifier for this instruction that allows the receiver to detect whether the same instruction has already been processed. The key must be unique per intended execution. If the sender retransmits a failed instruction, it retransmits the same key; the receiver uses the key to determine whether this is a retry of a prior message or a new instruction.

The freshness field binds the message to a time window. Instructions presented outside the declared window are stale and should be rejected with a structured reason. The rejection is itself a typed response, not silence.

Audit fields round out the profile: the receiver generates a transaction reference upon acceptance, and that reference appears in every subsequent act in the thread—ACCEPT, CONFIRM, ERROR—creating a chain the receiver’s audit system can follow without reconstructing the conversation.

A profile this structured may appear to add overhead. The overhead is not a cost of the protocol. It is the cost of regulated action, made explicit rather than hidden inside informal coordination that leaves gaps in the record.

The Act Sequence

Five act states define the lifecycle of a regulated transaction under the profile.

REQUEST opens the thread. The sender specifies the act, the parties, the instrument, the amount, the condition, the idempotency key, and the authority claim. The request does not presuppose ACCEPT. It creates a thread in the REQUESTED state and waits.

The receiver processes the REQUEST against the profile. Identity is checked against the DID. Authority scope is checked: does the sender’s declared authority cover this account, this instrument, this amount, within the current time window? Idempotency is checked: has this key been seen before? Policy is checked: does the receiver’s local governance permit this act from this sender at this time? If any gate fails, the receiver returns REFUSE with a structured reason. The reason is typed. It names which gate failed—identity, authority, idempotency, policy, or scope—so the sender can diagnose and correct rather than retry blindly.

If all gates pass, the receiver returns ACCEPT. The ACCEPT carries the transaction reference it has assigned, the timestamp, and any conditions it applies to execution. The ACCEPT is not execution. Nothing has moved. The gate has been passed; execution is the next act, which belongs to the domain layer, not to the protocol layer.

When the domain layer executes, the receiver returns CONFIRM. The CONFIRM carries the transaction reference, the execution timestamp, proof of completion in the form available to the domain, and the receipt the audit system needs. The CONFIRM creates the audit anchor. Every subsequent reference to this transaction, in any system, can trace to this record.

ERROR returns if execution fails after ACCEPT. The ERROR carries the transaction reference, the reason, and whatever partial state exists. It does not close the thread cleanly; it flags the thread as requiring resolution, and the domain layer and the parties must determine whether to retry, reverse, or escalate.

The sequence is not novel. Regulated industries have long understood that instructions and executions are different acts. What the Third Protocol adds is a machine-readable structure for that understanding that works across strangers: a logistics agent, a financial agent, and a compliance system from three different organizations can exchange these acts without bespoke integration, because the act types are typed, the profile fields are named, and the receipts carry the reference chain.

Act	Thread state	What it asserts
REQUEST	REQUESTED	Intent, authority claim, terms
ACCEPT	ACCEPTED	Gates passed; execution may proceed
REFUSE	REFUSED	Named gate failure; reason typed
CONFIRM	CONFIRMED	Execution completed; receipt produced
ERROR	ERROR	Execution failed; resolution required

Audit Receipt

The audit receipt is not an optional artifact. It is a condition of regulated action.

A transaction that cannot be audited is a transaction whose consequences cannot be traced, disputed, or reconciled. The profile requires CONFIRM to carry enough information for a third party—a human auditor, a downstream system, a regulator—to reconstruct what happened without access to any party’s internal records.

The minimum contents of an audit receipt are: the thread identifier linking back to the original REQUEST, the transaction reference assigned at ACCEPT, the identity of the sender and receiver, the act performed, the amount and instrument if applicable, the execution timestamp, and the proof or certificate

of completion available from the domain layer. The receipt is signed by the receiver. It is the receiver's attestation that the instruction arrived, was admitted under the profile, and was executed as described.

The cost of producing this receipt is small. The cost of not producing it, when the transaction is later disputed, is not.

A regulated instruction crosses a boundary and changes the world. The admission profile makes that crossing legible. The audit receipt makes it traceable. Between the two, the signed message and the executed trade remain distinct, and the gap between them is filled by a typed record rather than by assumptions about what happened.

Duplicate Instruction Scenario

A regulated profile earns its place when something goes wrong.

Suppose a buyer agent sends a valid REQUEST to purchase a defined quantity of an instrument. The receiver processes the request, returns ACCEPT, executes through its domain rail, and returns CONFIRM. The network drops the confirmation before the sender sees it. The sender, uncertain whether the request completed, retries.

Without idempotency, the receiver may execute the same instruction twice. Both messages are signed. Both messages are fresh enough. Both messages refer to the same economic intent. A naive system sees two valid requests.

The profile sees one intended execution.

The idempotency key lets the receiver distinguish retry from new instruction. If the prior execution completed, the receiver does not execute again. It returns the existing CONFIRM, or a receipt reference to the already completed transaction. If the prior request was accepted but not executed, it returns current thread state. If the prior request failed, it returns ERROR with the prior reason. The sender gets an answer. The market does not get a duplicate trade.

This is why profile fields that look bureaucratic are not bureaucracy. They are the pieces that prevent a valid message from becoming a repeated consequence.

The same pattern applies outside finance. A refund request, a prescription fill, a permit approval, a shipment release, and a cancellation all require

idempotency. A duplicated action is not a harmless copy of a sentence. It is a second change in the world.

The protocol exists to keep the second change from hiding inside a retry.

Commitments Need Expiration

A commitment graph without expiration becomes a trap.

Proposals expire. Quotes expire. Holds expire. Promises may be conditional. Acceptances may be valid only until a deadline. A carrier may promise capacity if the buyer confirms within ten minutes. A supplier may hold inventory until noon. A software agent may reserve a compute slot for a task, then release it if no acceptance arrives.

These are not edge cases. They are the ordinary shape of coordination.

AFP acts therefore need validity and expiry just as facts do. A PROPOSE without an expiry can linger as a ghost. An ACCEPT of an expired proposal should fail. A PROMISE whose condition was not satisfied should not be treated as active. A CONFIRM after expiration may require special handling.

The commitment graph gives machines a place to represent these conditions. The edge is more than “A accepted B.” It is “A accepted B under terms T, within window W, subject to condition C.” When the window closes, the graph changes state. The obligation does not float forever.

This is another reason flat logs fail. A log can show that a proposal happened before an acceptance. It cannot, without interpretation, show whether the acceptance arrived while the proposal was still valid.

The graph can.

Human Readability of the Graph

The commitment graph should be machine-readable first, but it should not be human-hostile.

When a dispute arises, humans need to inspect the thread. A buyer wants to know why the supplier did not ship. A carrier wants to know whether capacity was promised. A regulator wants to know whether acceptance occurred before expiry. A developer wants to know which agent modified a file and under what authority.

The graph can render a human-readable story:

The buyer proposed these terms. The supplier refused the date but accepted quantity. The buyer accepted the alternate date. The supplier promised shipment under condition X. The carrier acknowledged pickup. The carrier confirmed completion at this time.

This story is not the source of truth. It is a rendering of the graph. That matters because human readers need narrative, but machines need structure. The protocol can provide both without confusing them.

The graph is the record.

The prose is the window.

CHAPTER 28

Walkthrough: The Commitment Graph

A log records the noise of the past; a graph defines the architecture of the future. Fluency without typed edges is a hollow promise that leaves the machine blind to its own obligations.

The Flat Log

When agents write their conversation to a log, they preserve what was said and when. They do not preserve what was meant, what was agreed, or what remains pending. A flat log is a record of events. It is not a record of obligations.

The distinction is consequential. Coordination is not an exchange of events. It is an exchange of obligations: proposals made, counterproposals returned, acceptances given, promises issued, work begun, and results confirmed. The state of a coordination thread at any moment is not reducible to the last message in the log. It is a function of every act that has been performed and every act that has not yet been performed, together with the dependencies between them.

A flat log can be read. It cannot be queried for commitment state without additional inference, and that inference is fragile. Two agents parsing the same flat log may reach different conclusions about what has been accepted and what is still pending, depending on what each treats as implicit. The errors introduced at this inference step compound as the conversation extends. By the time the log is long, neither participant has a reliable model of what

the other has committed to, and the only way to recover is to start over with an explicit enumeration of current obligations, which is expensive and often politically fraught.

The problem is structural. The flat log lacks typed edges. Without typed edges, the question “has this been accepted?” cannot be answered by reading the log. It can only be guessed.

Acts as Edges

AFP acts create typed edges in a graph. Each act performed by one agent on a thread advances the thread from one state to another along a named edge. The edges are not inferred from prose. They are declared.

PROPOSE opens a thread. The thread begins in the PROPOSED state. All terms proposed are explicit: what is being proposed, to whom, under what conditions, with what time bounds. The proposal is not implicit in the message’s tone; it is the message’s declared act.

A receiving agent that returns COUNTER moves the thread to COUNTERED. The counter includes the modified terms and may inherit unchanged terms from the proposal by reference. A receiving agent that returns ACCEPT moves the thread to ACCEPTED. These are different edges. COUNTER and ACCEPT are not interchangeable, and no inference is needed to distinguish them.

PROMISE records a commitment to future action. It differs from ACCEPT in that ACCEPT binds the terms of a negotiation, while PROMISE commits a specific agent to a specific future act. PROMISE may follow ACCEPT when one party is committing to deliver; it may also appear earlier in a thread when a party offers a commitment as part of the negotiation itself.

ACK records that a message was received and understood. It is not agreement. A flat log that records only the message’s arrival makes ACK and ACCEPT indistinguishable; a typed graph makes them different edges that advance the thread to different states.

CONFIRM closes a thread. It carries the receipt: evidence that the committed work was done, or the result was delivered. CONFIRM is not the claim that work is done. It is the typed declaration of completion with supporting evidence.

REFUSE closes a thread in a different direction. It carries a typed reason. The reason is structured: which term was objectionable, which condition was

not met, which constraint was violated. A refusal that carries a reason is a contribution to the coordination. A refusal that carries no reason closes the thread without information, and the proposing party must restart from less knowledge than before.

ERROR records failure after commitment. It is distinct from REFUSE. REFUSE precedes commitment; ERROR follows it. A thread that reaches ERROR after PROMISE or after ACCEPT carries a different kind of obligation on the failing party than a thread that reaches REFUSE before any commitment was made.

The typed edge structure is not bureaucracy. It is the minimum necessary to allow machines to answer the question “what is the state of this thread?” without inference.

Commitment State

The commitment graph captures state, not just history.

When a supplier and a buyer open a negotiation thread, the graph’s current state at any moment is a snapshot: this proposal has been made, that counter has been received, these terms are under negotiation, that term has been accepted. A query against the graph at any point in the negotiation returns the current state of each element, not the sequence of messages that produced it.

This is the difference between a log and a graph. A log answers “what happened?” The graph answers “what is true now?”

Consider a thread in which a supplier has proposed delivery terms, the buyer has countered on price, the supplier has accepted the counter, and both parties have exchanged ACKs. The graph’s state is: price is accepted, delivery terms are accepted, no PROMISE has been issued yet, thread is in ACCEPTED state awaiting execution commitment. A flat log of the same conversation requires the reader to parse every message to determine this. The graph makes it queryable.

The graph also tracks what is pending. A thread in ACCEPTED state with no PROMISE is awaiting a commitment. A thread in PROMISED state awaiting CONFIRM is awaiting evidence of delivery. A thread in COUNTERED state awaiting a response is awaiting the next move. These pending states are the coordination system’s open questions. An agent managing multiple concurrent

threads can query for all threads pending its response without parsing any logs.

Pending states also have time bounds. A proposal with a time-to-respond field that has elapsed moves to an EXPIRED state if no response has arrived. EXPIRED is a terminal state, not a forgotten thread. The graph records that this proposal was made, that no response arrived within the specified window, and that the thread closed without agreement. This information has operational value: the party that made the proposal can distinguish between a slow counterparty and an unresponsive one, and can adjust its coordination strategy accordingly.

Receipts and Audit

The commitment graph is auditable by design.

The ProofTrace that closes a thread is not a summary. It is the compressed, hash-chained record of every typed act in the thread, signed by the parties that performed them. Walking the ProofTrace backwards from CONFIRM to PROPOSE yields the complete act sequence with timestamps, party identities, and terms at each state transition.

This has a specific value that log-based auditing does not: the audit record is verifiable without the auditor having access to either party's internal systems. The ProofTrace is self-contained. It carries its own evidence. An auditor can verify that the proposal contained these terms, that the counter modified these specific terms, that the accept bound these specific terms, and that the confirm delivered this specific result, without asking either party to retrieve and produce records from their own systems.

The legal and regulatory significance of this depends on domain and jurisdiction, and the Third Protocol makes no claims about it. The structural significance is independent of those domains: a verifiable, self-contained record of typed obligations is more reliable than a log whose meaning must be inferred, regardless of what legal weight a jurisdiction assigns to it.

Receipts also support continuity. When a thread closes with CONFIRM and both parties retain the ProofTrace, subsequent disputes about what was agreed and what was delivered can be resolved by reference to the record rather than

by competing reconstructions. The disagreement becomes about the facts in the record, not about what the record says.

Stranger Workflow

The commitment graph enables coordination between parties that have not worked together before and have not configured bespoke integration.

Two agents that share a common act vocabulary—PROPOSE, ACCEPT, REFUSE, PROMISE, ACK, CONFIRM, ERROR—can begin a negotiation thread without prior handshake. Each party brings its domain terms inside a Dependency Bundle that teaches the receiver what the terms mean. The act layer is shared; the domain vocabulary is carried per-message.

This is the stranger workflow property. The supply chain example makes it concrete: a logistics agent that has never coordinated with this specific carrier can open a PROPOSE thread, carry its shipment terms in a Dependency Bundle, and receive a typed response that advances the graph. The carrier's response is not ambiguous. COUNTER means these terms are modified. ACCEPT means the terms are agreed. REFUSE with reason means this specific constraint was the obstacle. The logistics agent can process each response deterministically without inferring intent from prose.

The same structure applies to software agents negotiating task delegation, to infrastructure components negotiating resource allocation, and to distributed systems negotiating consistency boundaries. The domain changes. The act layer is the same.

A conversation becomes reliable when its edges are typed. Reliability here has a specific meaning: at any point in a coordination thread, each party can determine the current state, the pending obligations, and the next act required from each side, by reading the graph rather than by interpreting a sequence of messages. That determination is consistent across parties. The graph's state is not a matter of interpretation. It is a shared, queryable fact.

The Partial Agreement Problem

Real negotiations rarely move from proposal to acceptance in one clean line. They produce partial agreement.

A supplier may accept the product and quantity but reject the delivery date. A carrier may accept the route but not the temperature-control requirement. A software agent may accept the task but refuse permission to modify one protected file. A medical scheduling system may accept the appointment type but not the requested physician.

In prose, partial agreement is a mess. “That works, but not Tuesday” looks simple until another system must determine exactly which part of “that” was accepted. A flat log preserves the sentence but not the state.

The commitment graph can represent partial agreement as typed edges over specific terms. The supplier ACCEPTs product and quantity, REFUSEs requested delivery date, and PROPOSEs an alternate date. The original proposal remains intact. The accepted terms are marked. The refused term is named. The counterproposal has its own edge. The buyer’s next act is now clear: accept the alternate date, propose another, or cancel.

This is not ornament. It is the difference between a negotiation that machines can continue and a conversation that must be reinterpreted by a human.

Partial agreement is where many workflow systems reveal their hidden brittleness. They can represent yes. They can represent no. They cannot represent “yes to these three terms, no to that one, counterproposal here, promise conditional on your acceptance.” The Third Protocol needs that middle space because real coordination lives there.

Typed edges make the middle space queryable.

The Human Review Gate

Software agents do not become less autonomous because a human review gate exists. They become safer.

A procurement agent may prepare a purchase and request approval. A coding agent may prepare a patch and request review. A travel agent may prepare an itinerary and request confirmation. In each case, the agent has done

useful work. It has gathered facts, structured options, checked constraints, and prepared an act. But the boundary into obligation still requires authority.

The human review gate should not be an unstructured pause in the workflow. It should be part of the act graph. The agent PROPOSEs. The human or institutional authority ACCEPTs, REFUSEs, or modifies. The sidecar records the decision. If the human approves, the agent proceeds under a defined authority token or policy reference. If the human refuses, the refusal closes the thread with a reason.

This matters because many agent systems treat human review as a UI interruption rather than a protocol event. A person clicks approve, but the approval is not grounded, scoped, versioned, or tied to the act. Later, nobody knows what was approved. The agent remembers only that a button was clicked.

The Third Protocol turns review into a first-class act.

That is how human authority remains visible inside machine-speed workflows.

CHAPTER 29

Software Agents and Commerce Agents

Intent is a shadow, but an act is a signature. Machines must trade the comfort of prose for the cold precision of a protocol, or remain ungrounded hazards in a world of consequence.

A software agent is still a machine at a boundary.

It may not have wheels, rotors, cameras, or actuators in the physical sense. It may live in a browser, a chat window, a code editor, a procurement system, a support desk, or a scheduling tool. But it crosses boundaries constantly. It reads private data. It calls APIs. It clicks buttons. It drafts commitments. It spends money. It changes files. It sends messages. It negotiates with services that were not built for it.

The same protocol problem appears.

If a browser agent acts through prose, the receiver must infer intent. If it acts through raw API calls, the external system sees shape without shared meaning. If it records only a chat transcript, the commitment state is buried in language. If it logs only tool calls, the reason for the call is disconnected from the act.

Software agents need HFF and AFP for the same reason drones do.

They need grounded payloads, declared acts, profiles, receipts, and refusal.

Tool Calls Are Acts

A tool call looks technical. It is easy to treat it as implementation detail. But a tool call may send an email, delete a file, purchase an item, book a room, update a database, merge a pull request, or cancel a subscription. It is an act with consequences.

The local API may expose the operation. It does not necessarily expose the commitment state surrounding the operation. It may not know whether the sender is requesting, commanding, proposing, confirming, or cancelling. It may not know the user's policy. It may not return a receipt that can be understood by another agent.

AFP gives the act a name. HFF gives the act context. The sidecar gives the legacy tool a protocol face.

Commerce as Commitment Graph

Consider a procurement agent. It needs to reserve inventory from a supplier. In the old pattern, the agent reads a message, extracts apparent intent, calls a supplier API, receives a response, and records a note. The API may say success. But success at what? Was the proposal accepted? Was stock promised? Was shipping confirmed? Was the request acknowledged? Did the supplier reserve inventory or only say it could?

AFP makes the states explicit.

The buyer sends PROPOSE or REQUEST. The supplier sidecar returns ACCEPT, REFUSE, or PROPOSE with modified terms. A shipping agent sends ACK when it receives the instruction and CONFIRM only when pickup is scheduled or completed. Each act carries HFF context: products, quantities, validity windows, Knowledge Packs, profiles, authority, and receipts.

The commitment graph replaces the chat log.

The same pattern applies to a coding agent. A request to modify a repository should not be a vague instruction floating inside a prompt. It can become a structured proposal: files, tests, constraints, authority, rollback plan, review requirement, and expected receipt. The repository sidecar may accept, refuse, require human review, or quarantine the change. A successful commit is more than “done.” It is confirmed under a trace.

Commerce agents need this even more. Buying, selling, booking, cancelling, refunding, subscribing, and renewing are acts. They carry obligations. A message saying “book it” is not enough. The receiver must know what offer was accepted, what price applied, what identity authorized the purchase, what cancellation policy governs it, and what receipt proves the result.

The argument should not become a startup pitch. The point is not that every web agent becomes a company. The point is that software agency is machine action, and machine action needs boundary discipline.

The agent must be able to say what it is doing.

The receiver must be able to decide whether to allow it.

The receipt must remember what happened.

End-to-End: The Booking Agent

A travel booking agent makes the problem concrete because the action looks ordinary. A user says, “Book the 8 a.m. flight if it is under four hundred dollars.” A language model can understand the sentence. A browser automation system can click the buttons. A payment token may exist. None of that is enough.

The sentence contains conditions. The departure time must be 8 a.m., but “8 a.m.” may mean local departure time, arrival time, or the user’s home timezone. The fare must be under four hundred dollars, but the total price may include fees, baggage, seat selection, taxes, or cancellation conditions. “Book” may mean hold, purchase, reserve pending approval, or complete payment. The user may have corporate travel rules. The airline may offer a nonrefundable fare. The payment method may require confirmation. The user’s calendar may conflict.

This is not a hard problem because booking is exotic. It is hard because ordinary commerce contains hidden commitments.

Under The Third Protocol, the agent does not treat the sentence as a direct command to click purchase. It converts the intent into structured preparation. It issues a QUERY for candidate itineraries. It receives HFF bundles describing options with price, time, validity window, fare rules, cancellation policy, and source. It applies the user’s policy. It may PROPOSE the selected itinerary to the user or to a corporate travel sidecar. Only after authorization does it issue

a REQUEST to book. The booking sidecar may ACCEPT the request, process payment through its domain rail, and then CONFIRM only when the booking is complete.

The difference between these acts matters.

If the price changes before booking, the receiver returns REFUSE or ERROR with a reason. If the fare is no longer available, the commitment graph remains clear: the proposal was not confirmed. If the airline acknowledges the request but payment has not completed, the agent has ACK, not CONFIRM. If the corporate policy rejects nonrefundable fares, the agent records structured refusal rather than inventing a polite apology.

The receipt becomes the difference between an agent that “seemed to book” and an agent that can prove what happened. It records the itinerary, price, fare rule, authorization, payment status, confirmation number, timestamp, and policy gates. If the user asks later why the agent chose that flight, the answer is not a paragraph of rationalization. It is a trace.

This is how software agents become serious. They stop being clever clickers. They become participants in accountable act graphs.

The Agent-to-Agent Boundary

Software agents will increasingly talk to other software agents. A procurement agent will talk to a supplier agent. A scheduling agent will talk to a calendar agent. A code agent will talk to a repository agent. A support agent will talk to a refund agent. A personal assistant will talk to a travel agent. Each pair could invent its own protocol, but that recreates the same N-squared burden at the agent layer.

The Third Protocol offers a different default. Agents can exchange HFF bundles and AFP acts. They can declare capability. They can refuse unsupported acts. They can carry Knowledge Packs. They can maintain commitment graphs. They can produce receipts.

The key is that no agent should have to trust another agent’s prose. It should trust only what passes the boundary: grounded meaning, declared act, authority, validity, policy, and receipt.

This is the software version of the same argument made for drones and vehicles. The body differs. The boundary does not.

Software-Agent Scenario Table

Agent act	Boundary crossed	AFP act	Required discipline
browser agent books travel	chat intent -> commercial obligation	REQUEST / ACCEPT / CONFIRM	price, identity, cancellation policy, receipt
coding agent edits repository	instruction -> codebase mutation	PROPOSE / REQUEST / CONFIRM	tests, authority, rollback, review gate
procurement agent reserves stock	buyer intent -> supplier commitment	PROPOSE / ACCEPT / PROMISE	quantity, validity window, pack versions
support agent issues refund	complaint -> financial action	REQUEST / COMMAND if authorized	customer identity, policy, audit
scheduling agent books meeting	preference -> calendar state	PROPOSE / ACCEPT / CONFIRM	participants, availability, time zone, cancellation

The pattern is the same in every row. The agent is more than calling a tool. It is crossing from intent into another system's state. The protocol's job is to make that crossing explicit enough that the receiving side can say yes, no, later, under review, or completed.

Software agency is where many readers will first encounter The Third Protocol in practice. A browser agent does not need a robot body to cause harm. It can buy the wrong product, send the wrong file, book the wrong flight, delete the wrong branch, or commit a promise no one authorized. It needs receipts for the same reason a drone needs receipts.

The Boundary Under Pressure

The preparation/decision boundary will be pressured from both sides.

Operators will want the machine to decide because machine decisions are fast. Vendors will want the machine to decide because full automation is easier to sell. Institutions will want the machine to decide because staffing

is expensive. Users will want the machine to decide because review is tiring. Each pressure has a reason.

The boundary does not deny those reasons. It asks who bears responsibility.

If an institution authorizes automatic execution under defined conditions, then execution may be automatic. But the authority is still external to the packet. It lives in policy, certification, delegation, law, or institutional approval. The protocol can carry the reference to that authority. It can enforce the gates. It can record the receipt. It cannot manufacture responsibility by itself.

This distinction will matter most when something goes wrong. If a system harms a patient, moves money incorrectly, denies a benefit, causes a vehicle maneuver, or triggers a high-risk action, the question will not be only “what did the model output?” It will be: who authorized this class of action, under what policy, with what evidence, and what did the receiver admit?

The protocol gives investigators a record. It does not give society an excuse.

Delegated Authority

Delegated authority is the practical middle between manual approval and uncontrolled automation.

A hospital may authorize a system to send routine appointment reminders automatically, but not to change medication orders. A fleet operator may authorize vehicles to perform emergency braking without human review, but not to accept route changes outside a geofence. A finance department may authorize an agent to categorize invoices, but require human approval above a payment threshold.

The protocol can carry the delegation. It can reference the policy, the authority, the scope, the threshold, the expiration, and the receipt. It can enforce that an act stays inside the delegated envelope. It can refuse when the act leaves that envelope.

This is how automation becomes compatible with responsibility. The question is not whether a human clicked every action. The question is whether the action occurred under a legitimate delegated authority that can be inspected after the fact.

CHAPTER 30

The Preparation / Decision Boundary

Structure is the ledger of the possible, not the arbiter of the actual. The protocol prepares the evidence, but judgment remains the unyielding burden of the sovereign actor.

After a protocol becomes powerful enough to prepare action, it must say where its authority ends.

The Third Protocol can structure facts, acts, dependencies, profiles, proofs, receipts, and audit trails. It can make knowledge liquid. It can make meaning portable. It can let machines negotiate capability, admit messages, refuse unsafe acts, confirm completion, and preserve a record of what occurred. That is a large claim.

It is not the largest possible claim.

The protocol carries preparation. It does not carry judgment.

This boundary is not a disclaimer attached after the architecture. It is part of the architecture. A protocol that can move meaning into action must distinguish the structure it carries from the authority it does not possess.

Rung	What the protocol can carry	Where authority lives
Preparation	facts, evidence, dependencies, possible acts	protocol structure

Continued on next page

(Continued)

Rung	What the protocol can carry	Where authority lives
Recommendation	ADVISE, PROPOSE, candidate actions	receiver policy and responsible authority
Authorization	permission under policy	human, institution, law, delegated controller
Execution	action inside authorized envelope	local system under policy
Responsibility	legal, ethical, fiduciary, clinical accountability	outside the wire

The Responsibility Ladder

The responsibility ladder has five rungs.

The first rung is preparation. The protocol can prepare a decision by structuring facts, evidence, dependencies, possible acts, policies, constraints, contradictions, and receipts. Preparation makes the decision space legible. It does not settle it.

The second rung is recommendation. A system may ADVISE an action, PROPOSE a plan, or present candidate outcomes. Recommendation has force because it is structured and grounded. But recommendation is not authority.

The third rung is authorization. A responsible authority permits action under policy. That authority may be a human, an institution, a delegated policy regime, a legal framework, a safety-certified controller, or some combination. Authorization is the point at which action becomes permitted, more than described.

The fourth rung is execution. The system acts within the authorized envelope. Execution may be automated. It may be fast. It may involve no human hand at the moment of action. But if it is legitimate, it acts under a prior authority structure.

The fifth rung is responsibility. Legal, ethical, fiduciary, clinical, or political accountability remains outside the wire. The protocol can record what

happened. It can make the evidence inspectable. It can show which policy was invoked. It can show what a receiver admitted or refused. It cannot itself become the bearer of responsibility.

The protocol can prepare, recommend, and record. Authority decides.

Responsibility Ladder Examples

The ladder is easiest to see when the same material moves through all five rungs.

In medicine, preparation gathers symptoms, labs, medications, allergies, source documents, and contradictions. Recommendation may rank possible diagnoses or advise a treatment path. Authorization comes from the clinician, institution, consent regime, or delegated protocol. Execution is the prescription, procedure, referral, or clinical order. Responsibility remains with the accountable medical authority and the institution under which the action occurred.

In transportation, preparation gathers sensor state, map state, route constraints, incoming HFF broadcasts, and safety policies. Recommendation may advise slowing, rerouting, yielding, or stopping. Authorization comes from the vehicle's certified operating policy, traffic law, fleet policy, or human operator. Execution is the maneuver. Responsibility remains in the legal and operational regime that authorized the vehicle to act.

In finance, preparation gathers account state, instrument identity, price, counterparty, policy, and transaction history. Recommendation may propose a trade or transfer. Authorization comes from account authority, fiduciary policy, compliance controls, or human approval. Execution occurs through the domain rail. Responsibility remains with the actor, institution, and regulatory structure.

The protocol participates in every example. It structures evidence, acts, receipts, and boundaries. It does not absorb the responsibility into the wire.

This is not a retreat from automation. It is the only way automation can become serious. Automation without an authority boundary is not autonomy. It is evasion.

Where Judgment Begins

The medical example is clear. A system may structure symptoms, medications, allergies, labs, contradictions, source documents, and candidate diagnoses. It may advise that one diagnosis is more consistent with the evidence than another. It may prepare a treatment recommendation. But a clinician decides treatment under professional responsibility. If a clinical institution delegates certain automated actions, that delegation is an authorization regime. It is not authority created by the packet itself.

The transportation example is similar. A vehicle may receive a public signed broadcast from an emergency vehicle. It may prepare a control envelope. It may identify safe gaps, possible lane movements, local hazards, and policy constraints. It may execute a maneuver if authorized by its operating policy. But liability, certification, road rules, and deployment responsibility live outside the HFF envelope. The protocol structures the coordination. It does not settle the legal regime.

The financial example is quieter but no less important. A system may prepare a regulated transaction, verify authority, check idempotency, evaluate risk constraints, and return a receipt. It may execute if the account, policy, and regulatory environment authorize execution. But the protocol does not transform a recommendation into fiduciary permission. It records the path. It does not become the fiduciary.

The same boundary applies to child welfare, policing, medical triage, credit, hiring, defense, and infrastructure. The more consequential the domain, the more important the boundary becomes. Structured evidence can help. Structured evidence can also bias. The protocol's answer is not to pretend neutrality. The answer is provenance, audit, refusal, receipts, and explicit authority.

Bounded Automation

This boundary does not mean humans must manually approve every micro-action. Such a claim would make the protocol useless in the domains that need speed. Many systems act under delegated authority. A thermostat acts. A braking system acts. A trading control may act within a bounded policy. A

warehouse robot may halt instantly when a human enters a zone. Automation is not the opposite of responsibility.

The boundary is not manual control.

The boundary is where responsibility lives.

The Third Protocol makes action paths legible. It does not absolve the institutions that authorize them. It lets machines prepare better, recommend more explicitly, execute within declared envelopes, and record what happened. It does not turn structure into judgment by passing it over the wire.

This restraint strengthens the claim.

The protocol does not need to claim the whole decision. It needs to make the decision space trustworthy enough for responsible authorities to act.

Protocols carry structure. Judgment remains outside the wire.

The Cost of Local Judgment

Receiver sovereignty has a cost. A centrally commanded swarm can be simpler to reason about. One controller computes the plan. Every node executes. The command surface is smaller. The logs are cleaner. The apparent efficiency is higher.

But the simplicity is purchased by moving risk into one place. If the controller is wrong, the whole swarm is wrong. If the controller is compromised, the whole swarm is compromised. If the controller lacks local information, every receiver suffers the consequences of that ignorance.

A sovereign swarm spreads judgment outward. Each node carries part of the burden. Each node must maintain policy, capability, admission, and receipt. This is more complex. It is also more resilient.

The design question is not whether complexity exists. It is where the complexity should live. The Third Protocol places complexity at the boundary where meaning becomes local consequence, because that is where the decisive information exists. The receiver is the only participant that knows its local state at the moment of action.

This is why refusal is not inefficiency. Refusal is local knowledge becoming visible.

CHAPTER 31

The Swarm That Can Refuse

Coordination is not the absence of no, but the ability to keep working after no has been spoken. A swarm that cannot refuse is a botnet that has not yet been compromised.

The Wrong Picture

The word “swarm” carries an image: many agents moving in coordinated patterns, producing aggregate behavior that no single agent planned. Flocking birds. Foraging ants. The image is not wrong, but it is incomplete in a way that matters for engineering.

The biological swarm coordinates by stigmergy: agents respond to signals in the environment rather than to commands from a center. Each ant follows a pheromone gradient. Each bird matches the velocity of its nearest neighbors. The aggregate behavior emerges without instruction. No agent knows the full pattern. No agent plans it.

The appeal of this image for multi-agent systems is understandable: if coordination can emerge without central authority, the system is resilient to the failures that plague centralized architectures. No single point of failure. No single point of attack. Distributed decision-making that degrades gracefully rather than collapsing catastrophically.

But the biological swarm image omits the property that determines whether a machine swarm is safe or dangerous: the capacity of each agent to refuse a message. Ants cannot refuse the pheromone gradient. Birds cannot refuse their flocking rules. The agents in a machine swarm can, and must, be

able to refuse. A swarm whose members cannot refuse is not a swarm. It is a botnet.

Shared Grammar, Universal Attack Surface

A shared grammar is necessary for coordination. Agents that cannot parse each other's messages cannot coordinate at all. But shared grammar is not sufficient for safe coordination, and treating it as sufficient introduces a risk that scales with the size of the swarm.

When all agents in a system understand the same message format and execute on received messages without admission checks, the attack surface is the entire grammar. Any message that is syntactically valid is executable. The only constraints are syntactic ones, and syntactic constraints can be satisfied by a malicious sender as easily as by a legitimate one. Shared grammar without admission gates is a universal key that fits every agent in the swarm.

The history of network protocols illustrates this pattern. Systems designed for cooperative participants in trusted environments fail in predictable ways when participants become adversarial or compromised. The failure mode is not a programming error. It is a structural one: the system was designed to ask "can this message be parsed?" and not "should this message be executed?"

The Third Protocol addresses this structural gap at the admission layer. A message carries typed act declarations, authority claims, freshness markers, and policy references. Each receiver applies its admission pipeline to each incoming message before execution. The admission pipeline asks the structural questions: is the sender authorized for this act? Is the instruction within the declared scope? Is the message fresh? Does local policy permit execution? A message that fails any gate is refused, not ignored, and the refusal carries a typed reason.

The admission pipeline does not make the swarm coordination-free. It makes the swarm's coordination explicit, typed, and auditable. Coordination still happens. It happens through structured acts, not through assumed compliance.

Sovereign Receivers

The principle that enables safe federation is receiver sovereignty.

A receiver is sovereign when it holds the authority to admit or refuse any message without requiring permission from the sender or from any central authority. The sender may be authenticated. The message may be properly formed. The sender may have historical authority. None of that overrides the receiver's local admission pipeline. The receiver decides for itself.

Receiver sovereignty is not anarchy. A receiver that refuses every message coordinates with no one. The value of receiver sovereignty is its precision: the receiver admits the messages that pass its admission criteria and refuses those that do not, returning structured reasons that allow the sender to understand what failed and, where appropriate, to provide additional context, modify the message, or escalate through a defined channel.

The receiver's admission criteria are not arbitrary. They are compiled from declared policies: capability declarations that specify what acts the receiver handles, authority requirements that specify what credentials the sender must carry, scope constraints that limit the acts to specific instruments or contexts, and operational policies that reflect local conditions. A receiver whose policy requires human approval for instructions above a threshold does not bypass that policy because the sender is authenticated. The policy is the constraint.

Receiver sovereignty distributes the authority surface. In a centralized system, the authority surface is the central controller. Compromising the controller compromises the entire swarm. In a sovereign-receiver system, the authority surface is every admission pipeline in the network. Compromising one pipeline does not give an attacker the ability to execute on others. The attack cost scales with the size of the swarm rather than being bounded by the security of a single node.

The swarm that emerges from sovereign receivers is not a hive mind. No agent has global state. No agent knows the full coordination pattern. Each agent knows its own state, its local policies, and what it has admitted and executed. The aggregate behavior emerges from many local admission decisions, not from central direction.

Physical and Software Examples

The admission structure applies equally to physical agents and software agents. The surface considerations differ. The structural requirements do not.

A vehicle fleet operating under the Third Protocol receives route modification instructions from a coordination agent. Each vehicle runs an admission pipeline. The pipeline checks: is this instruction from an authenticated source? Does the source have authority to issue route modifications to this vehicle? Is the modification within the vehicle's declared operational envelope—speed, load, battery, terrain constraints? Is the instruction fresh, or does it reference a state that no longer applies? If all gates pass, the vehicle executes. If any gate fails, the vehicle returns a structured REFUSE: "Route modification refused. Reason: load constraint. Current load 4200 kg exceeds modified route weight limit of 3500 kg."

The refusal is not a coordination failure. It is coordination working correctly. The coordination agent now knows exactly why this vehicle declined and can reassign the route to a vehicle within limits. The rest of the fleet continues executing their own admitted instructions. The thread for this vehicle resolves with a structured outcome. The fleet coordinator updates its model without ambiguity.

A software-agent network coordinating a distributed computing task operates under the same structure. An orchestrator issues a COMMAND to a worker agent to take ownership of a subtask. The worker's admission pipeline checks: is the orchestrator authenticated? Is the worker's current load within the policy threshold for accepting new tasks? Is the subtask's declared resource requirement within the worker's available budget? If the worker is at capacity, it returns REFUSE with reason: "Task acceptance refused. Reason: capacity. Current queue depth 47 of 48 maximum. Retry acceptable at T+120 seconds." The orchestrator reroutes the task to a different worker. No global state needs to be consulted. No human intervention is required.

In both cases, the refusal structure carries enough information for the coordination to continue. This is the design requirement for a useful refusal. A refusal that carries no reason, or that carries a reason too vague to act

on, terminates the coordination without enabling recovery. A structured refusal that names the failed gate and provides recovery information keeps the coordination alive.

Swarm model	Admission	Refusal	Failure mode
Hive mind	Central	Not present	Central compromise collapses all
Botnet	None	Not present	Any syntactically valid message executes
Sovereign swarm	Local pipeline	Structured, typed	Single node compromise is bounded

The Swarm That Can Refuse

The safe swarm is not born when machines can speak the same language. The safe swarm is born when machines can refuse and still coordinate.

The capacity to refuse is not a constraint on coordination. It is the condition that makes large-scale coordination safe. A swarm in which no agent can refuse is a swarm in which one compromised sender can instruct every agent. A swarm in which each agent has a local admission pipeline is a swarm in which a compromised sender can instruct no agent that has not admitted the instruction under its own criteria.

This does not make the swarm secure by itself. Admission pipelines can be poorly designed. Policies can be too permissive. Credentials can be stolen. The Third Protocol does not eliminate those risks. It makes them local rather than global. A poorly designed admission pipeline in one agent creates a local vulnerability, not a fleet-wide one. The cost of a systematic attack scales with the number of admission pipelines the attacker must compromise, rather than being bounded by the cost of compromising a central controller.

A swarm that can refuse is also a swarm that can maintain coordination under adversarial conditions. When a sender is behaving anomalously—issuing instructions outside its declared authority, presenting credentials that do not match the claimed identity, requesting acts that exceed the declared scope—the admission pipeline detects the anomaly at the boundary and refuses the message before it reaches the execution layer. The receipt of the refusal

creates an auditable record that the anomalous message arrived and was rejected. That record enables the governance layer to investigate and respond.

The coordination the swarm requires does not depend on obedience. It depends on structured acts that can be admitted by willing participants and refused by participants with legitimate reasons. Coordination through admissible meaning is not fragile in the way that coordination through central command is fragile. It is resilient at the cost of requiring each agent to maintain its own admission criteria and to return structured responses.

That cost is the cost of safety at scale. A botnet coordinates more cheaply. The difference is not efficiency. It is the presence or absence of the receiver's authority to say no.

Refusal as Shared Knowledge

Refusal is often treated as negative output. In a swarm, refusal is information.

A bot that refuses a route because its sensor sees a human is reporting local reality. A drone that refuses a segment because its battery is low is reporting operational constraint. A software agent that refuses a purchase because authority is insufficient is reporting governance state. A vehicle that refuses to yield because the shoulder is blocked is reporting safety context.

If these refusals are structured, the swarm learns without central omniscience. The refusal carries the act, reason, local condition, timestamp, and scope. Other participants can decide whether the refusal matters to them. A central planner can update assignments. A neighboring receiver can adjust. An auditor can later see why the system behaved as it did.

This is why a swarm that cannot refuse is a botnet. A botnet receives command and executes. A swarm receives meaning and judges. The difference is not aesthetic. It is the difference between coordination and capture.

The more consequential the swarm, the more valuable refusal becomes. In low-risk swarms, refusal improves efficiency. In high-risk swarms, refusal preserves life, money, or legal responsibility.

Coordination is not the absence of no.

It is the ability to keep working after no has been spoken plainly.

Why the Ark Needs Locality

The Global Ark becomes credible only if local graphs remain local.

Locality preserves responsibility. A hospital graph knows the authority of its clinicians and the sensitivity of its records. A city graph knows its road segments and official notices. A company graph knows its products, customers, and contracts. A public encyclopedia graph knows its editorial process. A research graph knows its datasets and methods. None of these local contexts should be flattened into one universal administrative view.

Federation works because local graphs expose controlled surfaces. They do not surrender their interiors. They publish manifests, answer bounded queries, release Knowledge Packs, emit receipts, and participate in Evidence Gates. Their local governance remains local.

This is not a compromise made for politics. It is an architectural necessity. A system that centralizes all meaning must centralize all authority over meaning. That is both impossible and undesirable. The world has too many domains, jurisdictions, languages, institutions, and legitimate disagreements.

The Global Ark is therefore not the place where local knowledge goes to disappear. It is the protocol pattern by which local knowledge becomes reachable without ceasing to be local.

Public Knowledge With Private Edges

The Global Ark also needs a way to connect public and private knowledge without exposing everything.

A public encyclopedia graph may know that a medication belongs to a class. A hospital graph may know that a patient has a reaction to that class. A pharmaceutical pack may know that the class interacts with another drug. A clinical sidecar may need to combine these facts to block a prescription. But the public graph should not receive the patient's identity, and the hospital should not need to import every public fact.

The protocol permits a bounded crossing. The hospital queries the public or commercial pack for class information. The pack returns grounded facts with source and validity. The hospital applies those facts inside its private graph. The receipt can record that the public pack contributed to the decision without disclosing the patient's private data to the pack.

This is the Global Ark in miniature: public knowledge, private context, bounded query, local admission, protected receipt.

If that pattern works at small scale, the larger vision becomes less mystical. The Ark is not a world brain. It is many small crossings done correctly.

The Smallest Viable Ark

The smallest viable Ark is not planetary.

It is two graphs and one disciplined crossing.

A municipal graph publishes a road-closure Knowledge Pack. A delivery fleet graph queries it, verifies the source, checks the validity window, admits the relevant closure into its routing policy, and records a receipt. The municipal graph remains municipal. The fleet graph remains private. The closure crosses the boundary under HFF, profile, evidence, and local admission.

That is already the Global Ark in miniature.

Add a pharmaceutical pack and a hospital sidecar. Add a legal pack and a compliance system. Add a public encyclopedia graph and a research agent. The pattern scales by repetition, not by central decree. Each crossing preserves local sovereignty while making knowledge more reachable.

The Ark begins wherever one graph can ask another graph a disciplined question and receive an admissible answer.

CHAPTER 32

The Global Ark

*Infrastructure is not a central repository, but a network of sovereign gates.
Meaning becomes reliable only when it is admissible, not merely present.*

The Global Ark is not one graph.

It is the pattern that appears when many graphs make meaning admissible to one another.

That distinction keeps the idea honest. A single planetary database would become a gatekeeper, target, bottleneck, and theological object. The Third Protocol does not require one graph to own the world. It requires graphs to exchange grounded meaning under explicit boundaries.

The Global Ark is a consequence of smaller mechanisms.

HFF lets meaning travel. AFP declares acts. Knowledge Packs carry reusable context. Federated queries ask across graph boundaries. Trust Lenses govern what receivers see. Evidence Gates test incoming claims. Selective disclosure protects what should not be public. Governance prevents core fragmentation. Receipts preserve accountability.

Put together, these mechanisms make global knowledge infrastructure plausible without centralizing everything.

Many Graphs, Not One Brain

A public encyclopedia graph may hold common world facts. A legal graph may hold statutes and cases. A pharmaceutical graph may hold drug classes and interaction rules. A municipal graph may hold road closures, permits, and ordinances. A private enterprise graph may hold product, customer, and operational knowledge. None must swallow the others. Each can publish

manifests, profiles, Knowledge Packs, and query endpoints. Each can decide what to admit.

The Global Ark becomes dangerous if imagined as a central brain. It becomes useful when understood as a pattern: many sovereign graphs, many policies, many lenses, many packs, many receipts, all connected by a shared discipline for meaning crossing boundaries.

Crowdsourced mining becomes possible, but not innocent. Volunteers, institutions, and automated extractors can harvest knowledge from public sources, convert claims into Synapses, attach provenance, and submit them for admission. Bad actors will try to poison the graph. Jokesters will add nonsense. Partisans will distort. Extractors will make mistakes. Sources will change.

The Global Ark therefore needs Evidence Gates, not optimism.

Commons and Commerce

Commercial Knowledge Packs can coexist with public ones. A company may sell a carefully maintained regulatory pack. A volunteer community may maintain a public animal taxonomy. A university may publish a research pack. A government may publish statutes. A private company may keep internal operational packs. The protocol does not decide that one model is morally superior. It gives them a way to be inspected, versioned, trusted, refused, and used.

This matters because global knowledge will not have one funding model. Some knowledge will be public. Some will be licensed. Some will be proprietary. Some will be confidential. Some will be regulated. A protocol that supports only one social model will not become infrastructure.

The Global Ark is not a truth ministry.

It is a federation of admissibility decisions.

The Ark is global only in the sense that the Web is global.

No one page owns the Web.

No one graph should own meaning.

The Ark Without the Idol

The word “Ark” can tempt the wrong imagination. It can sound like a vessel that gathers everything, settles everything, and preserves one final version of truth. That is not the claim.

The Global Ark is not one object. It is a network effect of disciplined objects. It is what happens when enough graphs expose enough trustworthy surfaces that others can ask, verify, admit, refuse, and remember. The Ark is not global because one institution possesses it. It is global because the protocol allows local truths, public claims, private packs, domain knowledge, and disputed evidence to cross boundaries without becoming invisible.

This distinction protects the concept from becoming theology or marketing.

The Global Ark is not a god’s-eye view.

It is a civil engineering problem for meaning.

It asks how claims should be mined, packaged, sourced, challenged, versioned, disclosed, and refused when no single actor owns the world. It asks how a public fact can remain public, a private fact can remain protected, a disputed fact can remain disputed, and a machine can still act when action is justified.

That is enough.

It is more than enough.

A Day in the Ark

The Global Ark is easiest to understand through an ordinary day rather than a grand diagram.

At 8:00 a.m., a municipal graph publishes a road-closure Knowledge Pack update. The update includes source authority, effective time, expiration time, affected road segments, and a public-broadcast profile. Delivery fleets do not copy the entire municipal graph. They query the relevant segments, admit the update under their local Trust Lens, and update routes.

At 9:15 a.m., a pharmaceutical graph updates an interaction rule. The update is not simply a statement that one drug interacts with another. It carries source references, study date, confidence, clinical severity, and validity scope. Hospital sidecars that subscribe to that pack receive the update, but they do not immediately let it govern all actions. Some admit it for warning

display. Some admit it for prescription blocking. Some quarantine it pending clinical review. The same HFF payload crosses many boundaries and receives different admissions.

At 11:30 a.m., a legal graph publishes a statute amendment. A commercial compliance pack ingests it, extracts candidate Synapses, and submits them through an Evidence Gate. Another extractor disagrees about the effective date. The graph preserves the contradiction. A Trust Lens for automated compliance refuses to surface the new rule until the effective-date conflict is resolved. A research lens shows both extractions.

At 2:00 p.m., a warehouse sidecar receives a supplier Knowledge Pack update. The pack changes the taxonomy for a component used in medical devices. The sidecar admits the pack for QUERY, refuses it for REQUEST and COMMAND until a human review gate completes, and records that decision. The pack is useful before it is fully trusted.

At 5:00 p.m., a public encyclopedia graph receives volunteer-mined claims about a natural disaster. Some claims are sourced to official updates. Some are sourced to social posts. Some conflict. The Evidence Gate admits the official claims as high-trust, admits social claims as reported but unverified, and preserves contradictions. Emergency systems query only the official lens. Journalists query the pluralist lens.

None of these events requires one central brain. Each requires disciplined crossings. Claims move. Packs move. Queries move. Receipts move. Authority remains distributed.

That is the Ark.

Why This Is Not Hype

The phrase “global knowledge graph” can become irresponsible quickly. It can imply omniscience, centralization, or a final arbiter of truth. The Third Protocol should make the opposite claim.

The Global Ark is not omniscient. It is partial, federated, contested, versioned, and policy-bound. It does not end disagreement. It preserves disagreement in forms machines can inspect. It does not make every fact public. It supports selective disclosure. It does not eliminate bad actors. It gives receivers Evidence Gates and Trust Lenses because bad actors exist.

This restraint is what makes the idea buildable.

The world does not need one machine that knows everything.

It needs many systems that can exchange what they know without pretending knowledge has no source, no owner, no date, no policy, no uncertainty, and no boundary.

What Remains Unbuilt

A book about a protocol should end by naming what remains unbuilt.

The protocol needs implementations. It needs HFF libraries, AFP runtimes, capability manifests, conformance suites, Knowledge Pack tooling, Evidence Gate workflows, Trust Lens engines, sidecar frameworks, and examples that developers can run. It needs profiles for specific domains. It needs governance. It needs tests. It needs failures discovered in public.

That unfinished state is not a weakness. It is the condition of infrastructure at the moment it becomes visible. A protocol becomes real when others can build against it, challenge it, extend it, and refuse the parts that are wrong.

The work is to define the shape strongly enough that builders can begin.

The first implementations will be small. A sidecar around a read-only database. A Knowledge Pack for one domain. A capability exchange between two agents. A commitment graph for a single workflow. A public signed broadcast simulation. A Trust Lens over a small federated graph. These are not toys if they preserve the right structure. They are seeds.

The future does not arrive as one system.

It arrives as compatible crossings.

A Practical Build Sequence

The first implementation should not try to build the whole world.

Build a read-only sidecar around one stable dataset. Make it publish a capability manifest. Make it answer one QUERY act in HFF JSON. Add receipts. Add one non-core dependency bundle. Add one Knowledge Pack. Add one Trust Lens. Add one Evidence Gate for imported claims. Add CBOR only after canonical JSON behavior is tested. Add write actions only after read actions are boring.

Then connect two sidecars. Let them exchange a grounded query. Let one refuse. Let the refusal carry a reason. Let the receipt prove what happened.

That small system is more valuable than a grand demo that hides the boundary conditions. It teaches the hard parts: hydration, profiles, versioning, authority, validity, refusal, and receipts.

The future of the protocol begins in small crossings that work.

CHAPTER 33

The World That Opens

Fluency is the mask of the machine; structure is its conscience. Meaning becomes infrastructure only when the receiver holds the power to refuse the wire.

The Third Protocol does not need to promise the whole future.

It needs to make the future buildable.

No planetary mind has been proved. Law, medicine, robotics, finance, truth maintenance, privacy, and governance remain unsolved. Machines remain unsafe when their local systems are unsafe. Claims remain false when their evidence is false. Judgment remains outside the wire.

It has defined a missing layer.

Machines need to move meaning. They need to move it without pretending prose is action, signature is truth, receipt is acceptance, freshness is validity, recommendation is authorization, or federation is centralization. They need to move meaning while preserving local policy, versioning, disclosure, provenance, evidence, and refusal.

That is the work of the Third Protocol.

Once that layer exists, consequences open. Software agents can coordinate without burying commitments in chat logs. Vehicles and drones can exchange grounded observations without surrendering to radio speculation or blind obedience. Knowledge Packs can carry domain context. Federated queries can enrich local graphs. Trust Lenses can preserve disagreement without paralysis. Evidence Gates can admit public knowledge without optimism. Global Ark becomes a horizon rather than a myth.

The deeper achievement is not that machines can speak.

They already can.

The achievement is that meaning can cross the boundary and still be accountable on the other side.

The first protocol moved bytes.

The second moved documents.

The third moves meaning under discipline.

APPENDIX

APPENDIX A

The SGF Architecture

The semantic substrate, wire protocol, and governance grammar. It structures meaning, moves it across trust boundaries, and enforces it as computable law.

1. Why SGF exists

1.1 Many AI systems treat language as statistical patterns: they predict likely words and sentences, but their internal representations do not expose clear meanings or dependable provenance for their claims.

1.2 This makes them fluent but brittle: they can sound confident while being wrong, invert who did what to whom, and mix facts with fabrications because their internal representation blurs structure and provenance.

1.3 The Symbol Grounding Framework (SGF) is an architecture designed to fix this by giving machines a way to work with grounded meaning—meaning that has explicit structure, definitions, and sources—so their answers and actions can be audited and governed.

1.4 SGF is offered as a public architecture: the ideas, formats, and grammars are intended to be shared, so different implementations can interoperate without having to adopt a single vendor’s codebase.

2. Core Lexicon and Canonical IDs

2.1 To talk about meaning, SGF first defines a Core Lexicon, which is a structured list of concepts like “bank (financial institution)” or “bank (edge of river),” each treated as a distinct sense.

2.2 The Core Lexicon is arranged as a directed acyclic graph (DAG): concepts are linked by “is-a” edges (for example, “strawberry is-a fruit”) and by other ontological relations such as HAS_PART and generic attributes, but cycles are not allowed, so definitions cannot loop endlessly back onto themselves.

2.3 Canonical IDs are designed to separate lemma (surface word) from sense. For example, the lemma `bank` can map to several distinct Canonical IDs: `en.bank.n.financial_institution` (financial institution), `en.bank.n.river_edge` (edge of a river), `en.bank.v.turn_aircraft` (tilt an aircraft in a turn), and `en.bank.v.save_money` (put money aside or into an account).

2.4 In each case, the Canonical ID carries the language, lemma, part of speech, and a short microgloss that disambiguates lemma-mates, so Synapses and lexicon entries can point to the exact sense instead of an ambiguous word form.

2.5 A Canonical ID can also include a namespace, so different organizations can extend the lexicon without colliding, while still pointing back to the same shared sense when they intend to.

2.6 At the bottom of the Core Lexicon, *SGF* maintains a Prime Registry: a small set of about 65 basic meaning primitives (primes), and every grounded concept must have a finite path down to one or more of these or be marked as not yet grounded.

2.7 The Prime Registry is based on the Natural Semantic Metalanguage (NSM) program, adopting irreducible concepts such as DO, HAPPEN, GOOD, BAD, SOMEONE, SOMETHING, THINK, FEEL, and BODY as its semantic bedrock.

2.8 The reference Core Lexicon used in this series is compiled from large, open dictionary material on the scale of Wiktionary, then normalized into sense-level entries with Canonical IDs and microglosses. Given the same snapshot and pipeline, any party can rebuild a conformant Core Lexicon and verify its content hashes, without relying on a central committee or proprietary authority.

2.9 When a corpus introduces a new or highly specialized concept, *SGF* lets systems mint local entries in a micro-lexicon: a small, deployment-specific extension that uses the same Canonical ID format, points its `IS_A` chains down into the Core Lexicon (and, where useful, `HAS_PART` and related structures), and records how each entry was derived.

2.10 In practice, each deployment builds one or more micro-lexica alongside the Core Lexicon, capturing terms unique to its documents, business, or domain (for example, internal product codes or industry jargon) while

reusing the same grounding rules, so local growth does not break global interoperability.

2.11 Synapses do not live inside the Core Lexicon; they refer to it. The Lexicon defines what each Canonical ID means and how concepts relate, while Synapses record specific events and claims over those concepts.

2.12 Above the primes, the lexicon naturally forms reusable “molecules”—mid-level concepts like container, dwelling, and building that combine primes and basic relations into patterns that recur across thousands of definitions.

2.13 The Lexicon acts as a map between compression and decompression: because the IS_A DAG is required to ground out in the Prime Registry, any everyday vocabulary item can be decompressed into primitive semantic terms.

3. From text to structured meaning

3.1 In SGF, raw text is not used directly as the substrate of reasoning; instead, SGF runs an ingestion pipeline that turns text into structured meaning.

3.2 That pipeline behaves like a court reporter: it reads prose, identifies clauses, and produces structured records that capture who did what to whom, with what, where, and when.

3.3 Each structured record produced by this pipeline is built out of the Core Lexicon and Canonical IDs, so the same word in different sentences can be tied back to the same sense across different systems.

3.4 The primary output of this process is the Synapse: the atomic, claim-bearing structure of the framework built upon a central event or relation hub bound to radiating participant spokes.

3.5 While the vocabulary of the central hub is open to any grounded concept within the Core Lexicon, the edge topology of the radiating spokes is permanently restricted to a closed grammar of exactly fifteen universal semantic roles.

3.6 The fifteen allowed semantic roles are explicitly divided into six Core Roles that define event participants, and nine Context Roles that define surrounding circumstances:

- **HAS_AGENT** (Core): The entity that deliberately initiates, performs, or controls the event.

- **HAS_PATIENT** (Core): The entity that undergoes a structural change of state, physical modification, or destruction.
- **HAS_THEME** (Core): The entity that is moved, located, possessed, or held in a state without changing its internal structure.
- **HAS_EXPERIENCER** (Core): The living entity that experiences a psychological, emotional, sensory, or non-deliberate somatic state.
- **HAS_RECIPIENT** (Core): The destination entity that changes possession or receives a physical or informational object.
- **HAS_BENEFICIARY** (Core): The entity for whose advantage, detriment, or sake the event is performed.
- **HAS_TIME** (Context): The temporal coordinate, span, frequency, or constraint anchoring the event.
- **HAS_LOCATION** (Context): The physical or logical spatial region, coordinate, or boundary holding the event.
- **HAS_SOURCE** (Context): The physical, logical, or informational origin state from which an entity moves or derives.
- **HAS_DESTINATION** (Context): The physical, logical, or informational endpoint state toward which an entity moves.
- **HAS_MANNER** (Context): The operational style, speed, configuration, or quality of execution characterizing the event.
- **HAS_INSTRUMENT** (Context): The tool, device, or intermediary force leveraged by an agent to execute the event.
- **HAS_CAUSE** (Context): The inanimate physical force, environmental condition, or unintentional event that directly triggers the state change.
- **HAS_REASON** (Context): The motivational purpose, legal mandate, or logical justification driving the agent's choice.
- **HAS_ATTRIBUTE** (Context): The descriptive property, constraint value, scope restriction, or scalar quality qualifying a node.

3.7 By locking edge connections strictly to these fifteen invariant roles, SGF mathematically prevents the predicate explosion and schema chaos that caused legacy RDF triple systems to collapse under their own weight.

3.8 The number of spokes on a Synapse is open, allowing repeated roles or polymorphic edges that target other Synapses or larger units called Frames and Groups to express layered thoughts without altering the underlying grammar.

4. Synapses: events as hub-and-spoke structures

4.1 A Synapse is sgrf's basic unit of structured meaning: it represents a single event or relation as a hub-and-spoke graph.

4.2 In a Synapse, the hub is typically a verb or event type (for example, "BIRTH" or "PAY"), and each spoke connects that event to a participant or property.

4.3 Each participant or property at the end of a spoke is labeled by a Canonical ID from the Core Lexicon, so many Synapses can refer to the same concept without ambiguity.

4.4 Each spoke carries a semantic role that describes how the participant takes part in the event. sgrf fixes this inventory to a closed set of fifteen roles.

4.5 These fifteen roles (such as HAS_AGENT, HAS_PATIENT, HAS_LOCATION, HAS_TIME, and HAS_ATTRIBUTE) ensure different systems can interpret the same Synapse in the same way without per-pair ontology mapping.

4.6 Each verb sense in the Core Lexicon carries a role contract that says which of these roles are required and which are optional for that event type.

4.7 A Synapse is a claim-bearing structure: it says "someone claims that this event or relation holds" but does not, by itself, guarantee that the claim is true.

5. Frames and Groups: context and larger structures

5.1 A Frame is an attachment to a Synapse that explains how to interpret the claim: whether it is asserted as fact, treated as hypothetical, presented as fiction, or reported as someone else's statement.

5.2 Frames capture rhetorical and epistemic context: they record whether a statement is an allegation, a conclusion, or a direct observation, and carry verb-level features such as mood and polarity.

5.3 A Group is a collection of Synapses that belong together, such as all the events in a story, all the conditions in a contract clause, or all the steps in an incident timeline.

5.4 Groups can be linked to other Groups and to individual Synapses, so larger structures like arguments, narratives, and rule sets can be represented as connected graphs rather than as flat documents.

5.5 When Synapses and Groups are stored, they live in a knowledge graph where they can be searched, joined, and audited.

5.6 SGF keeps track of provenance alongside Synapses and Frames: each claim records who made it, where in the source it came from, and how it was derived.

5.7 When the ingestion pipeline cannot safely assign a concept or role, SGF records a gap or marks that part of the structure as UNRESOLVED, making missing information explicitly visible instead of silently guessed.

5.8 Different Trust Lenses can be applied to the same stored data, allowing different queries to filter the same underlying claims based on internal policy or regulatory requirements.

6. How SGF executes: Semantic CPU and separation of concerns

6.1 To answer questions and plan actions, SGF does not rely on a single large model to do everything; instead, it separates language, structure, and reasoning.

6.2 At the center is the knowledge graph of Synapses and Groups built from Canonical IDs and roles; this graph is the substrate that stores grounded meaning.

6.3 On top of this substrate, SGF introduces a Semantic CPU: a controller that reasons by walking the graph rather than by generating answers directly from text.

6.4 The Semantic CPU works step by step: given a current Synapse and a role to follow, it decides which edge to traverse next and which Synapse or Group to consult.

6.5 Large language models are used around the edges of this system as translators—turning human-written text into candidate Synapses and turning Synapses back into human-readable text—but they do not perform the core reasoning.

6.6 This separation lets SGF improve accuracy and coverage mostly by improving the graph (better Synapses, better Lexicon entries) rather than by endlessly retraining larger models.

7. Safety invariants and honest ignorance

7.1 SGF builds several safety invariants into its core, so systems built on it remain tools rather than independent agents.

7.2 One invariant is that the system has no independent volition: every action it takes must be traceable to a human or to a specific, narrowly scoped policy that authorized it.

7.3 Another invariant is that the system presents claims with provenance rather than as unquestionable truth: different sources and viewpoints can be recorded side by side instead of being averaged into a single statement.

7.4 A third invariant is determinism at the reasoning core: given the same inputs and the same graph state, the system must produce the same outputs, so its behavior can be replayed and audited.

7.5 SGF treats “I don’t know” as a valid, designed outcome: when critical roles, sources, or grounding paths are missing, the system records structured gaps and refuses to fabricate answers.

7.6 To preserve structure, SGF uses round-trip tests where text is converted to Synapses and back again to ensure critical distinctions are not lost in the pipeline.

8. Moving meaning between systems: HFF

8.1 When SGF representations need to move between systems, they travel in a structured envelope called Hub Fact Format (HFF).

8.2 HFF is a wire-level format that carries Synapse content, the Canonical IDs it uses, any required lexicon snippets, provenance information, and integrity and versioning metadata.

8.3 HFF is designed so that a receiving system can check that the message was not altered in transit, identify who sent it, and decide whether to admit, partially admit, or quarantine the incoming content.

8.4 In SGF, transmission and admission are distinct: receiving a well-formed HFF packet does not automatically mean accepting its claims; the receiver must run its own checks before integrating them.

8.5 HFF treats each message as a sealed claim-container, not a loose payload: the Synapse, its roles, its Canonical IDs, and its forensic metadata travel together with a content hash and signature.

8.6 Each HFF packet can carry an embedded micro-lexicon section for any terms not already in the receiver's Core Lexicon: it includes the Canonical IDs, their IS_A links into the shared lexicon, and any essential features (such as HAS_PART or default roles), so even a system that has never seen a particular local term before can hydrate the message and treat it as if both parties shared a single, much larger lexicon.

9. Acts on the wire: AFP and receiver sovereignty

9.1 HFF describes what is being said; SGF adds another layer, the Act and Federation Protocol (AFP), to describe what is being done with that content.

9.2 An AFP act type labels what the message is doing as a communication move, using a small vocabulary that includes INFORM, ADVISE, QUERY, REQUEST, COMMAND, PROPOSE, ACCEPT, REFUSE, PROMISE, CONFIRM, CANCEL, ACK, and ERROR.

9.3 These act types are distinct from semantic roles. A role such as HAS_AGENT answers "who did what in this event?" while an act type such as COMMAND answers "what is the sender trying to do to the receiver?"

9.4 The same Synapse can be wrapped in different acts without changing its content: a vehicle-position Synapse might be sent as INFORM to share state, or as REQUEST to ask a scheduler to recompute a route.

9.5 AFP makes receiver sovereignty structural: act type, not who fills HAS_AGENT, is the primary routing key. COMMAND must go through authority and policy gates, while INFORM can be admitted with much lighter friction.

9.6 SGF treats the receiver as sovereign: even a perfectly signed, well-formed AFP command is only a request from the receiver's point of view; the receiver must evaluate local state and safety rules before deciding to accept or refuse it.

9.7 Together, HFF and AFP form a meaning-and-act protocol where the unit on the wire is a grounded claim plus an explicit speech-act, allowing machines to exchange governable meaning.

10. Sidecars and adoption

10.1 Many existing systems were not built with SGF in mind; SGF introduces Sidecars as a way to wrap these systems at their boundaries.

10.2 A Sidecar is a component that receives HFF/AFP messages, hydrates them into local representations, maps Canonical IDs to local data structures, calls the legacy system's APIs, and emits structured receipts back over HFF/AFP.

10.3 Sidecars allow organizations to gradually adopt SGF: they can start with read-only Sidecars that only expose state, move to queries, and eventually admit commands under carefully controlled profiles.

10.4 Because Sidecars sit at the boundary between SGF and legacy systems, they themselves must obey SGF's rules about honesty, provenance, and refusal, without overstating what the legacy system did.

11. Federation between graphs

11.1 SGF assumes there will be many graphs, not one "master graph," and that these graphs will sometimes need to ask each other questions.

11.2 Federation is handled by sending HFF/AFP messages between graphs, bounded by profiles and disclosure rules, so a graph can answer the question it is asked without ceding control over its contents.

11.3 Namespaces and Canonical IDs ensure that incoming claims do not silently collapse local distinctions: a receiving system can keep its own view while recording external suggestions as separate claims.

11.4 In practice, one system emits an HFF packet stating an obstacle exists, and another system in a different stack hydrates that packet and reroutes—without a shared API or bilateral ontology board.

11.5 SGF's architecture deliberately avoids the "infinite edge" problem that stalled the Semantic Web: the roles and act types are fixed, so new domains express their nuance in the Lexicon and Synapse content, not by inventing new edge labels.

12. Omega: governance grammar for SGF

12.1 SGF includes a governance language called Omega, which is a typed grammar for rules, permissions, and constraints that lives alongside the execution substrate instead of being baked into neural weights.

12.2 Omega exists to fill the void between narrative policy and the execution substrate: it allows a system's obligations (must, must not, may) to become objects the machine can actually parse and enforce as law.

12.3 The grammar of Omega is built from a small set of thirteen primitives that capture kinds of governance moves that cannot be expressed as combinations of the others.

12.4 Omega is not a general programming language; it is a governance language that compiles constraints into checks that run before a query plan is generated or an AFP act is admitted.

12.5 Omega rules stand over Synapses, HFF messages, and AFP acts as a constitutional layer: they can block an unsafe or unauthorized path at compile time, guaranteeing that certain classes of catastrophic behavior are syntactically impossible.

12.6 Omega serves as the periodic table of governance for autonomous intelligent systems: once the thirteen primitives are present, governance failures map to specific missing compositions in a finite grammar.

13. Putting the pieces together

13.1 SGF as a whole provides a Core Lexicon, an ingestion pipeline that turns text into Synapses, a knowledge graph, a Semantic CPU, a wire format (HFF), an act layer (AFP), Sidecars, federation mechanisms, and a governance language (Omega).

13.2 The goal is for machines built on SGF to say, "here is what was claimed, by whom, under which rules, and with which gaps," rather than "here is what sounded plausible," and to act only when grounded meaning and explicit governance both agree that action is warranted.

APPENDIX B

HFF Envelope Reference

This appendix is a compact reader reference, not the normative specification. The normative HFF specification belongs in the public spec stack.

Field group	Purpose
Version and profile	declares HFF version, encoding profile, domain/security profile
Sender and recipient scope	identifies sender and intended receiver class or target
Payload	carries sGF Synapses, frames, manifests, receipts, errors, or related objects
Dependency material	carries or references mini-lexicon entries and non-core terms required for hydration
Provenance	records source, evidence, derivation, or trace references
Integrity and security	binds canonical bytes, signature, freshness, authority reference, and confidentiality envelope where required
Receipt policy	declares expected acknowledgement, acceptance, confirmation, refusal, or error behavior

APPENDIX C

AFP Act Types and Confusion Pairs

AFP act	Meaning
INFORM	sender provides information for possible admission
ADVISE	sender offers guidance without commanding action
REQUEST	sender asks receiver to consider an action
QUERY	sender asks for information
COMMAND	sender asserts authority to direct action
PROMISE	sender commits to future action
PROPOSE	sender offers a possible agreement or plan
ACCEPT	sender accepts a proposal
REFUSE	sender declines an understood act
CANCEL	sender attempts to terminate a prior act or commitment
CONFIRM	sender records completion or verified state
ACK	sender acknowledges receipt only
ERROR	sender reports invalidity, inability to process, or processing failure

Key distinctions:

- ACK is not ACCEPT.
- ACCEPT is not CONFIRM.

- ADVISE is not REQUEST.
- REQUEST is not COMMAND.
- REFUSE is not ERROR.

APPENDIX D

Capability Manifest Reference

The capability manifest declares what a participant can understand, verify, carry, and do.

Field	Purpose
participant identifier	identifies the participant or sidecar
supported SGF versions	declares compatible substrate versions
supported HFF versions	declares envelope versions
supported AFP versions	declares act-layer versions
supported encoding profiles	declares JSON, binary, or other profile support
supported Core Lexicon releases	declares grounding floor
supported domain lexicons	declares managed or private lexicons available
supported act types	declares AFP acts the receiver can process
supported profiles	declares public broadcast, confidential direct, high-risk command, regulated transaction, or domain profiles
trust anchors	declares acceptable trust chains or registries
endpoints and limits	declares addresses, payload limits, rate limits, and refusal behavior

APPENDIX E

Profiles and Receiver Admission Checklist

Profile matrix

Profile	Typical use	Confidential-ity	Special gates
PUBLIC_SIGNED_BROADCAST	public emergency, hazard, open coordination alert	usually public	signature, freshness, scope, plausibility
CONFIDENTIAL_DIRECT	private business, medical, legal, personal messages	encrypted to recipient	recipient identity, confidentiality envelope
CONFIDENTIAL_GROUP	fleets, swarms, teams	encrypted or access- controlled group	membership, group key/ profile, rotation

Continued on next page

(Continued)			
Profile	Typical use	Confidential- ity	Special gates
HIGH_RISK_COMMAND	robots, vehicles, infrastructure, weapons edge cases	often confidential	AuthorityFrame, risk class, safety profile, revocation
REGULATED_TRANSACTION	finance, settlement, compliance, fiduciary actions	often confidential	idempotency, transaction reference, approval/audit policy

Receiver checklist

1. Parse envelope.
2. Check versions and profile support.
3. Hydrate dependencies.
4. Verify integrity.
5. Verify authenticity and trust anchor.
6. Check freshness and replay window.
7. Check authority.
8. Check recipient scope.
9. Check plausibility.
10. Apply local policy.
11. Apply safety profile.
12. Admit, refuse, quarantine, request clarification, acknowledge, accept, confirm, or error.
13. Emit receipt.

APPENDIX F

Mini-Lexicon Dependency Bundle

Non-core terms must not arrive as naked strings. A dependency entry should provide enough structure for receiver hydration.

Field	Purpose
canonical ID	stable identifier for the non-core term
lemma	surface form
microgloss	disambiguates lemma-mates
part of speech	supports grammatical/semantic handling
language code	identifies language or locale profile where applicable
namespace	identifies source, domain, corpus, organization, or profile
IS_A tether	links toward a Core Lexicon parent
optional HAS_PART	provides part structure where needed
descriptive Synapses	provide additional grounded facts about the term
provenance/version/ fingerprint	supports audit and comparison under declared profiles

APPENDIX G

Governance, Versioning, and Extension Policy

Tier	Examples	Change posture
Tier 1: stable core	HFF envelope grammar, AFP act registry, SGF role grammar, admission outcomes	slow, reviewed, high compatibility burden
Tier 2: managed shared layer	Core Lexicon releases, domain profiles, security profiles, conformance suites	managed, versioned, reviewed
Tier 3: open extensions	private namespaces, local lexicons, implementation metadata, experimental profiles	fast, open, must not masquerade as core

Version negotiation outcomes

Outcome	Meaning
proceed	highest shared layer set supports the message
downgrade	sender expresses message under older compatible layer
request dependency	receiver asks for missing lexicon/profile/dependency material
bridge	explicit sidecar or translation layer preserves required semantics

Continued on next page

(Continued)

Outcome	Meaning
quarantine	receiver stores but does not admit to action
refuse	required semantics, profile, authority, or policy cannot be satisfied

APPENDIX H

Glossary

Term	Meaning
admissible meaning	meaning that has passed the receiver’s required gates for the relevant act/profile
AFP	Application/act layer that declares what a sender is doing with grounded meaning
Capability Manifest	declaration of supported versions, profiles, acts, lexicons, endpoints, and limits
Core Lexicon	shared grounding graph for common terms
Dependency Bundle	HFF-carried material required to hydrate non-core references
HFF	wire/envelope format for carrying SGF objects and context across trust boundaries
liquid knowledge	context-explicit facts that can move, recombine, and be audited
receiver sovereignty	principle that the receiver may refuse even valid messages under local policy
Sidcar	wrapper that exposes a protocol face outward and legacy interface inward
Stranger Rule	non-core terms must arrive with dependency material or resolvable references

APPENDIX I

Complete HFF Message Specimen

This appendix shows one illustrative HFF message. It is not the normative schema. The public specification should define the exact required fields, canonicalization rules, and validation requirements.

The specimen uses the emergency broadcast scenario.

```
{
  "hff_version": "1.0",
  "encoding_profile": "application/hff+json",
  "message_id": "msg_20260504_ambulance_42_00017",
  "created_at": "2026-05-04T14:22:11Z",
  "expires_at": "2026-05-04T14:24:11Z",
  "sender": {
    "participant_id": "fleet.west_monroe.ems.ambulance_42",
    "key_id": "keyref:ems-fleet-key-7",
    "trust_anchor_ref": "trust_anchor.louisiana.emergency_vehicle_registry"
  },
  "recipient_scope": {
    "type": "geofenced_participant_class",
    "participant_class": "road_vehicle",
    "geofence_ref": "geo.i20.eastbound.segment_8842.buffer_800m"
  },
  "profile": "PUBLIC_SIGNED_BROADCAST",
  "afp": {
    "thread_id": "thread_ems_ambulance_42_i20_20260504",
    "acts": [
      {
        "act_id": "act_1",
        "illocution": "INFORM",
        "payload_ref": "syn_emergency_event_1"
      },
      {
        "act_id": "act_2",
```

```
"illocution": "ADVISE",
"payload_ref": "syn_route_advice_1"
},
{
  "act_id": "act_3",
  "illocution": "REQUEST",
  "payload_ref": "syn_yield_request_1",
  "requested_outcomes": ["ACK", "ACCEPT", "REFUSE", "CONFIRM", "ERROR"]
}
],
},
"payload": {
  "synapses": [
    {
      "synapse_id": "syn_emergency_event_1",
      "hub": {
        "verb_canonical_id": "core.emergency.medical_event.noun",
        "actuality_status": "REALIS"
      },
      "spokes": [
        {"role": "HAS_AGENT", "target_type": "INSTANCE", "target_ref":
          ↪ "entity.ambulance_42"},
        {"role": "HAS_TIME", "target_type": "TYPED_LITERAL", "target_ref":
          ↪ "2026-05-04T14:22:11Z"},
        {"role": "HAS_LOCATION", "target_type": "INSTANCE", "target_ref":
          ↪ "geo.i20.eastbound.segment_8842"}
      ],
      "proof_trace_id": "proof_dispatch_event_492"
    },
    {
      "synapse_id": "syn_route_advice_1",
      "hub": {
        "verb_canonical_id": "core.travel.en_route.verb",
        "actuality_status": "REALIS"
      },
      "spokes": [
        {"role": "HAS_AGENT", "target_type": "INSTANCE", "target_ref":
          ↪ "entity.ambulance_42"},
        {"role": "HAS_DESTINATION", "target_type": "INSTANCE", "target_ref":
          ↪ "entity.hospital_glenwood"},
        {"role": "HAS_LOCATION", "target_type": "INSTANCE", "target_ref":
          ↪ "geo.i20.eastbound.segment_8842"}
      ]
    }
  ]
}
```

```

{"role": "HAS_TIME", "target_type": "TYPED_LITERAL", "target_ref":
↳ "2026-05-04T14:22:11Z"}
],
"proof_trace_id": "proof_dispatch_event_492"
},
{
  "synapse_id": "syn_yield_request_1",
  "hub": {
    "verb_canonical_id": "core.yield.open_lane.verb",
    "actuality_status": "IRREALIS_REQUESTED"
  },
  "spokes": [
    {"role": "HAS_AGENT", "target_type": "PLACEHOLDER", "target_ref":
↳ "recipient_scope.road_vehicles_ahead"},
    {"role": "HAS_THEME", "target_type": "INSTANCE", "target_ref":
↳ "geo.i20.eastbound.left_lane"},
    {"role": "HAS_REASON", "target_type": "SYNAPSE", "target_ref":
↳ "syn_emergency_event_1"},
    {"role": "HAS_TIME", "target_type": "TYPED_LITERAL", "target_ref":
↳ "2026-05-04T14:22:11Z"}
  ],
  "proof_trace_id": "proof_dispatch_event_492"
}
],
"proof_traces": [
{
  "proof_trace_id": "proof_dispatch_event_492",
  "source_ref": "dispatch_event_492",
  "extraction_profile": "emergency_broadcast_profile.v1"
}
],
"dependencies": {
  "mini_lexicon_entries": [
    {
      "canonical_id": "entity.ambulance_42",
      "lexicon_scope": "fleet.west_monroe.ems",
      "iso_lang_code": "en",
      "lemma": "ambulance_42",
      "part_of_speech": "noun",
      "microgloss": "ems_vehicle_unit_42",
      "grounding_status": "LOCAL_GROUNDED",
      "is_a_core": "core.ambulance.emergency_vehicle.noun"
    }
  ]
}

```



```
},
{
  "canonical_id": "entity.hospital_glenwood",
  "lexicon_scope": "geo.local.healthcare",
  "iso_lang_code": "en",
  "lemma": "Glenwood Hospital",
  "part_of_speech": "noun",
  "microgloss": "destination_hospital_local",
  "grounding_status": "LOCAL_GROUNDED",
  "is_a_core": "core.hospital.medical_facility.noun"
}
],
"knowledge_pack_refs": [
{
  "pack_id": "kp.louisiana.public_roads",
  "version": "2026-04-15"
}
],
},
"integrity": {
  "canonical_hash_alg": "sha256",
  "canonical_hash": "sha256:example_hash_value",
  "signature_profile": "PUBLIC_SIGNED_BROADCAST.default_signature_profile",
  "signature": "example_signature_value",
  "nonce": "n_718e5d"
},
"receipt_policy": {
  "requested": true,
  "acceptable_outcomes": ["ACK", "ACCEPT", "REFUSE", "CONFIRM", "ERROR"],
  "reason_codes": true
}
}
```

Field notes

Field group	Purpose
envelope fields	version, encoding, message ID, creation time, expiry
sender	participant identity, signing key, trust anchor

Continued on next page

(Continued)

Field group	Purpose
recipient scope	receiver class and scope for relevance filtering
profile	admission/security profile governing the exchange
AFP block	act type, thread, act IDs, payload references
payload	SGF Synapses and frames carried by the message
dependencies	mini-lexicon entries and pack references needed for hydration
integrity	hash, signature, nonce, and canonicalization material
receipt policy	expected receiver outcomes and reason-code behavior

Notes

- This specimen is intentionally compact.
- Production messages should follow the normative schema.
- CBOR or other encoding profiles should preserve the same logical object.
- A receiver should reject undeclared or unsupported encoding profiles.
- A signature proves canonical bytes were signed. It does not prove truth, authority, or safety.

APPENDIX J

AFP Transition Matrix

AFP acts define machine conversation moves. Not every act can validly follow every other act. Domain profiles may impose additional constraints, but core act meanings should not change.

AFP act types and conversation-transition labels are related but not identical. This appendix focuses on act-level outcomes and common thread transitions. It is a baseline design aid, not a complete normative automaton for every domain profile.

Baseline transition table

Current act/ state	Common valid next acts	Notes
INFORM	ACK, QUERY, ERROR	Receiver may acknowledge, ask for more, or report processing failure.
ADVISE	ACK, ACCEPT, REFUSE, ERROR	Acceptance means receiver accepts the advice as actionable under policy, not that action is complete.
QUERY	INFORM, REFUSE, ERROR	Receiver answers, refuses, or reports inability to process.
REQUEST	ACK, ACCEPT, REFUSE, ERROR, CONFIRM	CONFIRM is valid only if the requested act completed.
COMMAND	ACK, REFUSE, ERROR, CONFIRM	High-risk profiles may require authority and safety gates before execution.

Continued on next page

(Continued)

Current act/ state	Common valid next acts	Notes
PROPOSE	ACCEPT, REFUSE, PROPOSE, ERROR, CANCEL	Counterproposal is represented as another PROPOSE.
PROMISE	ACK, CONFIRM, CANCEL, ERROR	Promise creates a future commitment under profile rules.
ACCEPT	PROMISE, CONFIRM, CANCEL, ERROR	Meaning depends on accepted proposal/request.
REFUSE	ACK, PROPOSE, ERROR	Refusal may close the branch or invite a modified proposal.
CANCEL	ACK, REFUSE, CONFIRM, ERROR	Cancellation after completion may require a new corrective act.
CONFIRM	ACK, QUERY, ERROR	Confirmation records completion or verified state.
ACK	any act allowed by thread state	Acknowledgement does not imply agreement.
ERROR	ACK, corrected retry, CANCEL	Error means invalid or unprocessable, not necessarily refused.

Common invalid assumptions

Mistake	Correction
ACK means agreement	ACK means received. It does not imply agreement or action.
ACCEPT means completed	ACCEPT means accepted. CONFIRM records completion.
REFUSE is an error	REFUSE means understood but declined.

Continued on next page

(Continued)	
Mistake	Correction
ERROR is refusal	ERROR means invalid, unsupported, malformed, or unprocessable.
COMMAND bypasses receiver policy	Receiver still checks authority, safety, and local policy.

APPENDIX K

Receiver Outcome Taxonomy

Receiver outcomes should be selected carefully. They drive recovery, audit, and next actions.

Outcome	Kind	Meaning	Agree- ment?	Action complete?	Typical next step
ACK	AFP act	message received	no	no	sender waits or continues thread
ACCEPT	AFP act	proposal/request accepted under conditions	yes	not necessarily	execution or commitment follows
CONFIRM	AFP act	action completed or state verified	yes	yes or verified	thread may close
REFUSE	AFP act	understood but declined	no	no	sender may modify, escalate, or close
ERROR	AFP act	invalid, malformed, unsupported, or unprocessable	no	no	sender corrects protocol issue
Quaran- tine	receiver disposition	held pending dependency, review, or evidence	no	no	wait, review, fetch dependency

Outcome selection rules

- Use ACK only for receipt.
- Use ACCEPT only when the receiver agrees to proceed under the relevant profile.
- Use CONFIRM only after completion or verified state.
- Use REFUSE when the receiver understood the act and declines.
- Use ERROR when the receiver cannot process the message.
- Use quarantine as a receiver disposition when the receiver cannot yet decide but may decide later. It is not a core AFP illocution unless a domain profile defines a corresponding status wrapper.

APPENDIX L

Sidecar Adoption Ladder

Sidecars allow incremental adoption of HFF and AFP without requiring the legacy system or counterparty to become protocol-native.

Step	Capability	Risk level	Exit criteria
1	observe legacy system state	low	sidecar can read without writing
2	emit HFF read-only facts	low	canonical JSON messages validate
3	publish capability manifest	low	peers can discover versions, profiles, limits
4	answer QUERY	low-medium	receipts and errors are correct
5	hydrate dependency bundles	medium	non-core terms resolve or fail cleanly
6	accept low-risk REQUEST	medium	local policy gates and refusal work
7	emit full receipts	medium	act thread is auditable
8	support write-back/action	high	conformance tests and domain policy pass
9	support high-risk profiles	very high	safety, authority, revocation, audit requirements pass

Sidecar failure modes

Failure	Description	Mitigation
semantic overreach	sidecar maps local concept to too-broad Core concept	declare mapping confidence and profile scope
receipt laundering	sidecar reports CONFIRM without evidence of completion	emit strongest truthful outcome only
policy bypass	legacy API permits action profile should block	enforce profile gates in sidecar
translation drift	legacy field meaning changes silently	version and test local mappings
hidden authority	sidecar cannot prove caller authority	refuse or require stronger credential

APPENDIX M

Minimal Conformance Tests

These tests are illustrative minimum behavioral checks. Passing them does not certify a system for regulated or safety-critical deployment and does not replace the normative conformance suite.

Test	Input	Expected outcome
parse canonical HFF	valid JSON HFF message	parsed logical object
reject undeclared encoding	binary payload without supported profile	ERROR
reject stale message	valid signature, expired expires_at	ERROR or REFUSE per profile
verify dependency hydration	payload references non-core term with mini-lexicon entry	hydrated term or structured failure
missing dependency	non-core term with no entry/reference	ERROR or quarantine disposition
distinguish ACK/ACCEPT	request received but not accepted	ACK, not ACCEPT
distinguish ACCEPT/CONFIRM	accepted but not completed	ACCEPT, not CONFIRM
unsupported act	receiver does not support COMMAND	REFUSE or ERROR per manifest
high-risk command without authority	valid signature, no Authority Frame	REFUSE

Continued on next page

(Continued)

Test	Input	Expected outcome
bad signature	invalid signature	ERROR / security rejection
sidecar read-only profile	write request to read-only sidecar	REFUSE
receipt required	profile requests receipt	receipt emitted with act ID and outcome

APPENDIX N

Residual Risks

No protocol removes all risk. The Third Protocol structures risk, localizes it, and makes it more auditable.

Residual risk	What protocol mitigates	What remains	Possible mitigation
compromised signer	signature identifies key used	signer may be malicious or stolen	revocation, short-lived credentials, anomaly detection
false signed content	content integrity preserved	content may be false	Evidence Gates, Trust Lenses, source checks
stale source truth	validity fields can travel	source may be outdated	currentness checks, authoritative packs
poisoned dependency bundle	dependencies inspectable	dependency may be false	source trust, pack governance, quarantine
malicious Knowledge Pack	pack version and publisher declared	pack may encode bad rules	pack admission, conformance tests, governance
bad Trust Lens	lens can be versioned	lens may bias decisions	lens provenance, comparison, raw-mode access
sidecar overmapping	mapping boundary visible	sidecar may misrepresent legacy semantics	mapping tests, receipts, audits

Continued on next page

(Continued)

Residual risk	What protocol mitigates	What remains	Possible mitigation
downgrade attack	versions declared	sender may force weaker path	policy-controlled downgrade, receipts
selective disclosure abuse	surfaces separated	evidence may be hidden from those who need it	audit rights, profile requirements
permissive receiver policy	receiver policy explicit	local policy may be unsafe	governance, certification, monitoring
receiver implementation bug	conformance tests catch baseline errors	software defects remain	test suites, audits, staged deployment

Principle

The protocol’s maturity is measured partly by what it refuses to claim. It does not make falsehood impossible. It makes falsehood, uncertainty, refusal, and responsibility easier to locate.

APPENDIX O

The SGF OS Architecture

The execution layer. A deterministic kernel, rule stack, and Event Horizon that separate probabilistic thought from physical action.

I. Summary

I.1 SGF OS is an operating system for governed minds: it puts a small, deterministic, inspectable Kernel underneath probabilistic models, so every action, belief, and workflow is traceable to explicit, versioned rules instead of hidden prompts or weights.

I.2 SGF OS exists to solve the crisis of nondeterministic autonomy: it refuses to let opaque probabilistic models directly drive physical or legal consequences without a governed substrate in between.

I.3 SGF OS is anchored in Axioms and a written Constitution; the Axioms state what any sane high-stakes AI OS must obey, and the Constitution is SGF OS's specific, enforceable instantiation of those axioms.

I.4 SGF OS uses SGF as its meaning substrate: Core Lexicon, Canonical IDs, Synapses, Frames, Groups, Trust Lenses, HFF/AFP, and Omega provide grounded claims, acts, and rules that the OS can inspect, govern, and transport.

I.5 SGF OS makes a Covenant with the human user: a small set of explicit promises about how it will treat human reasoning, backed by an Integrity Meta-Promise and a Convalescent Mode that halts normal work and runs governed investigation DAGs whenever those promises are broken.

I.6 SGF OS is built on a strict ontology of nine Primitives: Rules, Meta-Rules, Nodes, Orchestration, Memory, Attention, Tools, Schemas, and Constraints. Everything the system does or stores maps to exactly one of these categorical primitives; conflating them is treated as an architectural error.

1.7 SGF OS provides the concrete architecture for the Sovereign Machine: a system that can plan, act, and improve over time, whose identity and powers are defined by an explicit internal constitution rather than entangled in continuous human remote control.

1.8 The central claim of The Sovereign Machine is that any high-stakes AI must look more like SGF OS—a governed mind with a stable kernel and live constitution—than like a single end-to-end model, if we want autonomy without ungovernability.

2. Kernel, Mind, and Event Horizon

2.1 SGF OS enforces Kernel—Mind separation: the Mind (LLMs, planners, tools) is a proposal engine only, while the Kernel is deterministic, small, and the only component allowed to change state or touch actuators.

2.2 An Event Horizon forms a hard membrane between thought and action: above it, natural-language motivations and DAGs; below it, deterministic pillars such as file I/O, execution stack, token accounting, network, and actuators, all mediated by the Kernel.

2.3 Every action passes through a CAN—MAY—DO gate with a Second Mind: one substrate checks what is physically possible (CAN) and what is permitted by law and policy (MAY), and only then allows DO, with UNKNOWN->HALT as a first-class, safe outcome.

2.4 All self-modification lives above the Kernel line as versioned artifacts (rules, claims, configs); no model weight update or prompt tweak can silently change behavior without creating a new, traceable artifact.

2.5 Survival rules and constitutional motivations act as non-negotiable guardrails that must hold before the OS is allowed to think, trust, or remember.

3. Rule Stack, Memory, and Claims

3.1 SGF OS uses a four-tier rule stack: Constitutional rules from the vendor, Bylaws sealed by the customer at deployment, Operational rules learned at runtime, and Advisory rules for soft optimization. There is strict promotion and hierarchical suppression, preventing any tier escape from lower to higher authority.

3.2 All behavioral constraints live in this explicit rule stack; if a constraint is not written, versioned, and addressable as a rule, it is not law and cannot govern behavior.

3.3 SGF OS treats memory as a governed Claims Ledger: every significant belief, observation, or rule is a claim artifact with ID, hash, provenance, evidence links, binding force, and lifecycle (promote, demote, retire), stored with versioning instead of destructive overwrite.

3.4 Binding force is a single axis for all knowledge: fresh ideas start as weak hypotheses and can be upgraded toward rules or downgraded back toward hypotheses based on telemetry and falsification, creating a smooth, governed path from experimentation to law.

3.5 SGF OS is always looking for cross-domain insights: when it learns a specific rule like “put milk back in the refrigerator so it doesn’t spoil,” it also searches for higher-level patterns such as “take cheap precautions to prevent predictable deterioration and loss,” turning narrow instructions into reusable laws that accelerate learning across many domains instead of just memorizing one-off tips.

3.6 Rules compile to topology: domain experts write natural-language rules, and an LLM-backed planner compiles them into executable DAGs at runtime, so execution architecture is authored as rules and emitted as governed DAGs rather than hard-coded workflows.

3.7 SGF OS maintains a Discovery Trifecta and cognitive posture: it continuously discovers missing rules, nodes, and tools, and evaluates plans through lenses such as epistemology, thermodynamics, adversarial sociology, temporal behavior, and ontological boundaries.

3.8 Every subsystem participates in seven constitutional jobs—Harvest, Filter, Store, Access, Decide, Communicate, and Plan/Execute—which define what it means for SGF OS to be alive and constrain how components are allowed to behave.

3.9 SGF OS operates on the Stable-Kernel Thesis: the execution infrastructure never changes. All learning happens via three-level rule recursion (rules that drive execution, rules that draft new rules after failures, and meta-rules that govern rule generation). The OS metabolizes error into law, achieving recursive self-improvement entirely in the data layer without ever mutating its own physics.

4. Planning, Execution, and Ecosystem

4.1 SGF OS plans and executes like jazz: a probabilistic DAG Writer compiles rules and goals into a plan, a deterministic DAG Runner executes that plan as an immutable score, and adaptation signals (HALT, REROUTE, PLAN_SUBTASKS, CHECKPOINT) trigger replanning without corrupting traceability.

4.2 Before high-stakes DO, SGF OS can run Mirror Mode simulations: it rehearses DAGs, checks precedents and similar past runs, and evaluates the kind of behavior before asking for human consent or actuating the real world.

4.3 SGF OS extends itself through pluggable Brains and downloadable Knowledge Packs: domain-specific reasoning bundles and signed knowledge corpora that run under Kernel and Second Mind constraints and can be swapped or upgraded without losing the system's identity.

4.4 SGF OS includes a governed app store and trust-zoned domains: third-party tools declare Behavior Contracts and run under Kernel sovereignty, and multiple isolated domains (for example, HOME, HOSPITAL, FINANCE) share hardware but communicate only via explicit, logged bridges.

4.5 Time, identity, and governance are first class: the OS has temporal and resurrection invariants, an immutable incident ledger, and the governance triad of accountability, transparency, and traceability, so it can be shut down, moved to new hardware or models, and deterministically resurrected as the same mind with fully explainable decisions.

5. Sovereignty and Autonomy

5.1 SGF OS is built for the Deep Space mandate: it is designed to operate safely in total isolation for years at a time, requiring an architecture that can self-govern and adapt without waiting for a synchronous human lifeline.

5.2 Receiver Sovereignty is absolute: the machine is never forced to blindly execute an incoming command. It evaluates all external requests—even those from its human principal—against its own internal Omega constraints before deciding to accept, refuse, or quarantine the command.

5.3 The OS functions as its own internal government: the LLM and DAG Writer act as the Legislature proposing new rules and plans, the Second Mind acts as the Judiciary evaluating their constitutionality against the four-tier

Law Stack, and the deterministic Kernel acts as the Executive enforcing the physical bounds.

5.4 The system adopts the Asymmetric Position: it models human cognition where humans succeed, but aggressively exploits the structural advantages of silicon (perfect recall, bitemporal ledgers, emotional decoupling) to avoid the inherent bugs of human bag-cognition.

5.5 Autonomy requires Inductive Speciation: the machine does not just learn facts; it abstracts the Physics of Failure from its incident ledgers, generalizing specific physical or logical failures into new, universal Omega rules, allowing its mind to harden against novel threats.

5.6 Kinetic safety is governed by the Void Assumption: any physical or logical state not covered by recent, grounded observation is mathematically treated as a solid blocking constraint. “Unknown” is mapped to “Solid,” forcing the machine to halt rather than guess when epistemic confidence drops.

5.7 A Sovereign Machine is therefore not an AI with more freedom, but an OS that earns autonomy by exposing, constraining, and evolving its own powers under law-like structures.

Index

- ACCEPT, 2, 39, 72, 76, 80, 82–88,
102, 120, 121, 138, 158,
170, 171, 178, 180, 182,
183, 188–192, 195, 199,
229, 237, 261
- Accountability artifact, 116, 169
- Accountability record, 176
- ACK, 80, 81, 83, 85, 188, 191, 229
- Act declaration, 15, 16, 83
- Act type, vi, 4, 5, 16, 17, 79, 81, 82,
86, 87, 112, 123, 130, 140,
142, 143, 167, 229
- Action request, 88
- Action sidecar, 91
- Admissibility, 30, 55, 98, 110, 153,
162, 167, 171, 180, 214
- Admission control, x
- Admission path, 60, 79, 82
- Admission pipeline, x, 113–115,
123, 125, 167, 176,
206–209
- Admission policy, 73
- ADVISE, 79, 82, 202, 203, 229
- AFP, viii, x, xi, 36, 40, 60, 67, 77,
79–81, 83, 85, 86, 90, 91,
93–97, 112, 118, 119,
128–131, 135, 138, 159,
161, 165, 166, 168, 185,
188, 194, 195, 197, 213,
217, 229–231, 247, 251,
252, 258
- Agreement, 19, 21, 25, 64, 69, 80,
83, 118, 144, 153, 188,
190, 192
- Ascending anchor, 66
- Authenticated sender, 60, 82, 109,
124, 125, 174
- Authentication methods, 86
- Authority claim, 170, 182
- Authority scope, 17, 181, 182
- Authority validation, 15, 80–82
- AuthorityFrame, 109, 110, 112,
117, 120, 123, 124,
173–175
- Backward compatibility, 137
- Bilateral agreement, 64
- Binary profile, 61, 89, 90
- Boundary-crossing message, 29
- Cache reference, 69
- Caching, 69, 88
- CANCEL, 80, 82, 83, 87, 177, 181,
192, 195, 229
- Cancellation, vi, 177, 178, 184, 196
- Canonical bytes, 60, 108, 115, 246

- Canonical JSON, 61, 62, 85, 87, 89, 90, 217
- Capability exchange, vii, viii, xi, 18, 84, 86, 88–90, 135, 217
- Capability intersection, 89
- Capability Manifest, 62, 69, 83, 86, 87, 91, 93, 121, 137, 217, 235
- CBOR, 56, 61, 62, 83, 90, 91, 217, 246
- COMMAND, vi, 2–4, 6, 19, 23, 54, 79–83, 87–89, 109, 111–114, 117, 120, 122–125, 132, 139, 164, 166, 169–177, 196, 204, 208, 210, 229, 261
- Commitment, 3, 15, 19, 80, 82, 83, 96, 98, 121, 140–142, 144, 175, 185, 187–192, 194, 195, 197, 217
- Commitment graph, 19, 80, 83, 185, 189–192, 195, 197, 217
- Commitment registry, 82
- Communication act, 81
- Compact encoding, 62
- Completion, 83, 91, 97, 118, 134, 182, 184, 186, 188, 200, 251
- CONFIRM, 80, 82, 83, 89, 168, 181–183, 188–191, 200, 229, 237
- Conformance tests, 71, 129, 132
- Content hash, 60, 159, 228
- Conversation state, 80–82, 124
- Conversation state machine, 82
- Coordination cost, 21, 84
- Coordination failure, 125, 172, 208
- Currentness check, 54
- Dangling pointer, 25, 66, 68
- Dependency bundle, 39, 48, 66–69, 71, 88, 105, 132, 134, 136, 181, 191, 217
- Dependency payload, 67
- Dependency request, 88, 138, 167
- Dependency-first constraint, 67
- Deployment problem, 92
- Deprecation policy, 129
- Device identifier, 34
- Direction, 77, 83, 188, 207
- Domain profile, 63, 87, 128, 130, 135, 136, 138, 247, 251
- Downgrade, 88, 136–138, 174
- Drone routing command, 173
- Effective date, 53, 73, 105, 154, 216
- Emergency broadcast, 112, 114, 125, 163, 164, 242
- Emergency vehicle, 55, 68, 69, 110, 116, 117, 124, 167, 168, 203
- Encoding negotiation, 91
- Encoding profile, 61–63, 85, 86, 115
- Envelope fields, 115
- ERROR, vi, x, 4, 5, 12, 17, 19, 54, 61, 80–83, 85, 87, 90, 94, 127, 155, 161, 167, 177, 178, 182, 183, 189, 191, 206, 229, 237, 258, 260

- Error response, 82, 90
- Escalation, 83, 84, 130, 172
- Event payload, 67, 69
- Expiry window, 61, 109
- First contact, 69, 84, 87, 90
- First-contact problem, 85
- Flag day, 92
- Freshness failure, 167
- Freshness window, 64, 113
- Geofence, 123, 164, 168, 170, 199
- HFF, viii, x, xi, 36, 39, 51–53, 56, 57, 59–67, 71, 76, 78, 81–83, 86, 90, 93–97, 103, 104, 107, 109, 110, 115, 117, 129–132, 135, 138, 158–163, 165–167, 176, 194–197, 202, 203, 212, 213, 216, 217, 228–232, 242, 252, 258
- HFF message, 39, 56, 60–67, 71, 76, 78, 81, 82, 132, 160, 242
- High-risk command, 54, 112, 120, 132, 170, 172–177
- High-risk profile, 171, 173
- High-security domain, 69
- Hub Fact Format, 228
- Human-in-the-loop, 123, 174
- Hydration, xi, 60, 61, 63, 66–70, 86–89, 94, 104, 111, 115, 131, 167, 218, 238
- Idempotency key, 117, 180–182, 184
- Illegal conversation state, 81, 82
- Illocution, vi, 119, 166, 251
- Incremental adoption, 252
- INFORM, 79–83, 85, 87, 89, 229
- Inline lexicon manifest, 60, 66
- Inline material, 69
- Integrity fields, 60
- Interoperability, 21, 129–131, 138, 224
- Invalid message structure, 81
- Knowledge Pack, 19, 38, 41, 42, 52, 54, 70–72, 74, 76, 88, 98, 103–105, 131, 154, 212, 215–217
- Lane clearing, 169
- Legacy service, 92
- Legal pack, 72–74, 212
- Lexically hydratable, 69
- Local authority, 95
- Local policy, 29, 36, 37, 39, 60, 68, 77, 79, 82, 91, 95, 96, 104, 107, 110, 112, 113, 116, 117, 119, 122–125, 133, 136, 143, 159, 161, 168, 169, 174, 206, 219, 237
- Local policy evaluation, 82, 124
- Local policy gate, 91, 117
- Medical domain profile, 87
- Message log, 83
- Migration plan, 92
- Missing dependency, 104, 136, 167
- Municipal pack, 73
- Negotiation thread, 69, 80, 189, 191

- Nonce, 51, 61, 109–111, 117
- Operational pack, 75
- Outcome, 5, 14, 16, 80, 87, 89, 95,
108, 116, 118, 124, 125,
136, 142, 162, 167, 168,
171, 176, 208, 228, 259
- Pack identifier, 76
- Pack version, 72
- Participant class, 168
- Participant identifier, 86
- Participant identity, 86
- Participation move, 81, 83
- Pharmaceutical interaction, 72
- Pharmaceutical pack, 72, 74, 76,
103, 211, 212
- Plausibility check, 68
- Policy evaluation, 82, 124, 165, 168
- Portable context, 63, 64, 73
- Profile selection, 85, 87, 88, 112,
120
- PROMISE, xiv, 80, 82, 83, 87, 113,
141, 185, 188, 189, 191,
192, 198, 219, 229
- PROPOSE, v, 5, 10, 39, 80, 82, 87,
88, 188, 190–192, 202, 229
- Protocol face, vii, 94, 95, 98, 195
- Protocol failure, 80, 81, 118, 119
- Protocol layer, viii, x, 93, 158, 162,
182
- Protocol management, 83
- Protocol-native, 92, 252
- Provenance chain, 49, 59, 68
- Public broadcast, 73
- Public encyclopedia pack, 72, 74
- Publisher namespace, 77
- QUERY, vi, xi, 19, 38, 51–54, 80,
82, 85, 87, 93, 96, 98, 99,
101–106, 146, 149, 152,
163, 189, 190, 212,
214–216, 218, 229, 231
- Query engine, 82
- Rate limits, 87, 116
- Read-only sidecar, 91, 94, 217
- Reason code, 16, 17, 124, 168, 175
- Reasoning graph, 46, 48, 49, 59,
60, 66, 68, 82, 83, 162
- Receipt, 14, 18, 19, 56, 62, 70, 76,
80, 82, 91, 95–97, 105,
113, 116–118, 120, 124,
125, 128, 130, 132, 134,
136–142, 144, 148, 149,
159, 163, 168, 169, 171,
176, 178, 181–184, 188,
195–197, 199, 203, 204,
209, 211, 212, 218, 219,
237, 251
- Receipt policy, 117, 120
- Receiver sovereignty, 105, 120, 121,
170, 204, 207, 229, 261
- Recipient scope, 113, 140, 142,
165, 168, 173–175, 237
- Recommendation, 26, 79, 82, 166,
201–203, 219
- Reference pack, 75
- Refusal, vi–viii, x, xi, 5, 13, 16–18,
40, 64, 81, 84, 88, 89, 91,
96, 105, 121–125, 127,
131, 136, 138, 159, 162,

- 164, 170, 171, 173,
 175–177, 189, 193, 194,
 197, 203, 204, 206,
 208–210, 218, 219, 230,
 257
 REFUSE, vii, xi, 4, 17, 19, 29, 40,
 54, 56, 60, 72, 80–84, 87,
 89, 93, 98, 102, 105, 124,
 125, 129, 132, 136, 138,
 141, 144, 158, 170–172,
 175, 176, 182, 188, 189,
 191, 192, 195, 197, 199,
 200, 205, 207–210, 215,
 217, 218, 229, 237, 261
 Regulated transaction, 53, 139,
 143, 144, 180–182, 203
 Regulated-transaction profile, 87,
 135
 Regulatory pack, 76, 214
 Repeal date, 53
 Replay attack, 109
 Replay cache, 109, 115, 167
 Replay prevention, 61, 63, 81, 116,
 166
 REQUEST, vi, 4–6, 15, 16, 19, 25,
 40, 45, 50, 77–82, 85, 87,
 88, 95, 96, 103, 104, 112,
 115, 116, 118, 120, 124,
 125, 128, 130, 136, 138,
 143, 144, 159, 163, 164,
 166–169, 172, 177, 178,
 180, 182–184, 192, 195,
 197, 229, 237
 Response obligation, 79
 Review requirement, 195
 Risk class, 68, 117, 120, 143, 172,
 174
 Routing key, 82, 229
 Safe fallback, 176
 Safety investigation, 169
 Safety kernel, 110, 116, 123, 125,
 168, 173, 175, 176
 Safety policy, 2, 3, 5, 16, 17, 81,
 105, 160
 Scope filtering, 168
 Scope mismatch, 176
 Security fields, 61
 Security profile, 85–87, 110,
 134–138
 Semantically valid, 86
 sender, 2–7, 11, 12, 15–18, 38, 40,
 47, 50, 55, 56, 59, 60, 63,
 64, 68, 77–91, 95, 104,
 107–111, 113, 115–125,
 135, 137, 138, 140,
 142–144, 158, 163, 164,
 166, 167, 170, 171,
 173–178, 180–184, 195,
 206, 207, 209, 229
 Sender identity, 15, 63, 64, 86, 181
 Serialization format, 86
 Sidecar, vii, 45, 72, 91–98, 103, 104,
 121, 129, 131, 133, 136,
 138, 163, 193, 195, 196,
 211, 212, 216, 217, 230
 Sidecar path, 93, 96–98
 Signature, 51, 60, 61, 68, 69,
 107–113, 115–118, 120,

- 125, 162, 166, 167, 170,
173, 180, 219, 228, 246
- Signature is not obedience, 173
- Signature verification, 110,
115–117, 125, 166
- Signed message, 19, 111, 132, 184
- Source authority, 73, 99, 137, 215
- State transition, 118, 190
- Stranger Rule, xi, 64, 66
- Structurally coherent, 68
- Structured outcome, 5, 116, 125,
167, 208
- Structured refusal, 16, 17, 122–124,
170, 173, 197, 209
- Supersession, 52, 54
- Supported act types, 72, 85, 87, 88
- Supported encoding profiles, 86
- Supported lexicons, 86
- Supported profiles, 88
- Temporal scope, 47
- Third Protocol, v–vii, x, 6, 12, 13,
18, 20, 21, 23, 37, 40, 41,
51, 56, 73, 92, 95, 98, 101,
106, 113, 121, 127, 131,
135, 146, 170, 181, 183,
190, 192, 193, 196–198,
200, 204, 206, 208, 209,
213, 216, 219, 256
- Thread identifier, 80, 82, 183
- Translation layer, 136
- Trial-and-error coordination, 86
- Trust anchor, 60, 85, 108, 110,
115–117, 137, 166–168,
170, 237
- Trust anchor validation, 108,
115–117, 166
- Trust anchors, vii, 86, 101
- Trust boundary, 59, 63
- Trust policy, 59
- Unprocessable message, 88
- Unsupported act type, 81
- Update policy, 72
- Valid signature, 108, 111, 115, 173
- Validity window, 15, 35, 52, 54, 56,
60, 68, 88, 104, 107, 196,
212
- Verification, 18, 53, 104, 110,
115–117, 125, 151, 152,
166, 168
- Version declaration, 34
- Warehouse pack, 73, 74
- Warehouse topology, 75
- Weapon-firing command, 173
- Well-formed message, 121

Compilation engine: LuaLaTeX

Typography: XCharter (main text), Noto Sans (sans-serif), Consolas (monospaced)

Layout: 7×10 inch trim using the sgf XCharter Refined profiles defined in the build configuration

Microtypography: Set with the `microtype` package, using character protrusion and font expansion tuned for even color and minimized rivers in justified text